

Desenvolvimento JAVA WEB

WBA0179_v1.1

Desenvolvimento JAVA WEB

Autor: Bruno Zolotareff dos Santos

Como citar este documento: SANTOS, B.Z. *Desenvolvimento JAVA WEB*. Valinhos: 2016.

Sumário

Apresentação da Disciplina	03
Unidade 1: Introdução às Tecnologias *Java Enterprise Edition*	06
Assista suas aulas	20
Unidade 2: *Web Container* Necessidades de uma aplicação Web	27
Assista suas aulas	48
Unidade 3: Persistência e Reaproveitamento de Código na utilização de sistemas *Web*	55
Assista suas aulas	83
Unidade 4: Configurando o Hibernate e Integração JSF e JPA	90
Assista suas aulas	132

Desenvolvimento JAVA WEB

Autor: Bruno Zolotareff dos Santos

Como citar este documento: SANTOS, B.Z. *Desenvolvimento JAVA WEB*. Valinhos: 2016.

Sumário

Unidade 5: Inclusão de Práticas de Validação, Requisição, outras Linguagens e Construção de Projeto (Prático-Conceitual)	139
Assista suas aulas	162
Unidade 6: JSF com PrimeFaces	169
Assista suas aulas	181
Unidade 7: Temas do PrimeFaces, apresentação e utilização	188
Assista suas aulas	205
Unidade 8: JSF com JPA e MVC apresentação e utilização	212
Assista suas aulas	239

Apresentação da Disciplina

A tecnologia Web é uma das principais ferramentas no mundo corporativo que utiliza a rede de computadores como meio de comunicação capaz de proporcionar a interconectividade e transações de informações de instituições financeiras, militares, hospitais, dentre outros.

A disciplina foi estruturada para um aprofundamento nas tecnologias necessárias para o desenvolvimento de aplicativos Web, utilizando o JAVA como ferramenta. Portanto, serão apresentados os principais conceitos da importância da tecnologia e como utilizá-la.

Entender métodos práticos e ágeis para o desenvolvimento de projetos Web e estimular boas práticas de programação utilizando padrões das especificações do JAVA e da engenharia de software fazem parte do entendimento do conteúdo programado.

Contudo, a disciplina abordará os conceitos de desenvolvimento necessários para uma aplicação, utilizando o *Java Enterprise Edition* (JEE) e todos os recursos necessários para realizar uma programação de software distribuída.

Além disso, a abordagem da disciplina proporcionará a utilização de padrões e APIs necessários para o desenvolvimento utilizando persistência de dados. Assim, poderá ter o domínio de tecnologias como JSP/JSF e Servlet para o desenvolvimento de páginas Web.

A disciplina proporciona um entendimento maior do assunto em relação a tecnologias

Web, tais como: HTML, XML, JAAS, AJAX, que são abordadas para o conhecimento geral.

Portanto, o desenvolvimento começa com uma introdução a tecnologia JAVA e explora as especificações do JEE. Em seguida, você conhecerá as ferramentas necessárias para o desenvolvimento e como configurar o ambiente de desenvolvimento.

Unidade 1

Introdução às Tecnologias *Java Enterprise Edition*

Objetivos

1. Visão geral da tecnologia *Java Enterprise Edition*.
2. Entender os princípios de desenvolvimento para aplicações Web.
3. Desenvolver aplicações utilizando técnicas modernas com Java.
4. Entender as arquiteturas de padrões de desenvolvimento de software.

Introdução

O desenvolvimento Web é um tema relevante no mercado mundial de tecnologia, sendo que *Java Enterprise Edition* (JEE) destaca-se, principalmente, em aplicações corporativas pela robustez e aplicabilidade.

Desde o advento da Web, Java, desenvolvida pela equipe de **James Gosling** e Sun Microsystems, é uma das tecnologias mais utilizadas por apresentar diversas vantagens, principalmente por ter uma plataforma independente (CAELUM, 2016).

Para saber mais

James Gosling foi o líder do projeto para o desenvolvimento de uma tecnologia que pudesse ser utilizada em dispositivos como geladeiras, rádios e televisores. Essa tecnologia, inicialmente baseada em *SmallTalk*, foi batizada pelo nome de AOK (Carvalho em inglês) e posteriormente necessitou ser renomeada para Java, pois já havia uma empresa com esse nome (ORACLE, 2016).

Por um lado, JEE segue todas as normas e padrões exigidos pela engenharia de software proporcionando qualidade assegurada em suas aplicações em projetos Web (PRESSMAN, 2016). Por outro, é flexível em aceitar diversos frameworks e não é projetada sob um único padrão de desenvolvimento Web.

JEE é uma tecnologia cliente/servidor que funciona sob um servidor Web e é conhecida como aplicação distribuída. Assim, são usados alguns protocolos no processo de comunicação.

Portanto, o mercado tecnológico disponibiliza diversos **servidores Web** para JAVA, destaque para: TOMCAT, Jetty, Glassfish, JBOSS, WebSphere e WebLogic.



Link

Aplicações Web utilizam diversos servidores que podem ser usados para diferentes tecnologias. Disponível em: <http://pt.slideshare.net/raphaelamarques/servidor-web-8225869>. Acesso em: 12 jul. 2016.

No entanto, há uma classificação básica dentre esses servidores Web que é (CAELUM, 2016):

- *Web container*: servidor que suporta algumas especificações do JEE como: JSP (*JavaServer Page*), Servlet, JSTL (JSP *Standard Tag Library*) e JSF (*JavaServer Faces*). Esse tipo de servidor não possui toda a especificação do JEE completo e é aconselhável para aplicações que não utilizam tudo do JEE. Destacam-se os servidores: TOMCAT e Jetty.
- *Web application*: possui todas as especificações do JEE e há diversos servidores: Glassfish, JBOSS, WebSphere e WebLogic. Alguns desses servidores são gratuitos, destaca-se o Glassfish

que a SUN/ORACLE produziu e é considerado um dos lideres do mercado.

JSP é o padrão de páginas dinâmicas do JEE que possui extensão .jsp. Semelhante a uma página de HTML, possui a capacidade de processar códigos Java dentro da própria página, chamados de <scriplets> (ORACLE_2, 2016).

JSF (*JavaServer Faces*) é um padrão mais atual das especificações JEE que possibilita a criação de interfaces nas páginas de maneira ágil (_____, 2016).

JSTL é um mecanismo de recuperação de dados que pode acessar arquivos que contenham XML e é recomendado pela W3C (EDUARDO, 2016).

Entretanto, o servidor mais conhecido é o TOMCAT, que suporta a maioria das aplicações de projetos Web com JEE e é um servidor gratuito e mantido pela Apache (APACHE, 2016).

A IBM é uma das principais empresas que contribui para o desenvolvimento da tecnologia Java e, ainda, ajudou no servidor Glassfish junto com a Oracle. Porém, seu servidor de aplicação é o Websphere (IBM, 2016).

Portanto, há diversas empresas que contribuem para o desenvolvimento da tecnologia Java e, principalmente, para JEE. Segundo a ORACLE (2016), a **tecnologia Java** também utiliza outros recursos e possui um conjunto de tecnologias.

Link

É possível consultar uma visão do framework do Java no site oficial da ORACLE. Disponível em: <http://docs.oracle.com/javase/8/docs/>. Acesso em: 10 jul. 2016.

Segundo a ORACLE (2016), há mais de quatro bilhões de dispositivos no mundo que utilizam Java, e esse número aumenta com o transcorrer dos anos, sendo que essa tecnologia é encontrada por toda parte, desde televisores, celulares, carros, robôs e na Web.

Há diversas aplicações que utilizam JEE e a tecnologia Java para conectar-se a outras tecnologias a fim de obter soluções de negócio, como **SOA** (Arquitetura Orientada a Serviços), que é um tipo de conceito de arquitetura de software.

Link

SOA é uma importante metodologia do mundo corporativo e conhecê-la poderá ser uma vantagem competitiva. Disponível em: <http://www.pcs.usp.br/~pcs5002/oasis/soa-rm-csbr.pdf>. Acesso em: 12 jul. 2016.

Todavia, o JEE utiliza servidores Web com alta escalabilidade, ou seja, suporta um grande número de dados e processos paralelos e, ainda, sua arquitetura de *container* apresenta vantagens por ser utilizado em qualquer plataforma. O universo Java e diversas empresas que utilizam essa **tecnolo-**

gia são pioneiros na apresentação de novas soluções.

Para saber mais

Empresas como a Apache desenvolvem ferramentas em Big Data para extrair e processar dados da Web. Essas ferramentas de Big Data são desenvolvidas em Java e utilizam recursos do JEE (APACHE, 2016).

1. CONCEITOS SOBRE DESENVOLVIMENTO BÁSICO EM JAVA

O Java apresenta dois principais produtos de plataforma: Java *Runtime Environment* (JRE) e Kit de Desenvolvimento Java (JDK).

O JRE fornece as bibliotecas, a Máquina Virtual Java e outros componentes para executar applets e aplicativos escritos na linguagem de programação Java.

É também o alicerce para as tecnologias em Java *Web* (JEE) para o desenvolvimento e implantação de software empresarial. O JRE não contém ferramentas e serviços de utilidade pública, tais como compiladores e depuradores de desenvolvimento de applets e aplicações.

Java *Development Kit* (JDK) é um conjunto do JRE que contém tudo o que está na JRE, além de ferramentas, tais como os compiladores e depuradores necessários para o desenvolvimento de applets e aplicações.

Para saber mais

Diversos assuntos referentes ao Java estão disponíveis no site oficial da ORACLE, desde ambientes de desenvolvimento até produtos, como, por exemplo, o servidor Web e JDK/JRE em diferentes versões e plataformas necessárias para o desenvolvedor e usuário (ORACLE, 2016).

Para programar em Java, é aconselhável utilizar o **Netbeans** ou Eclipse JEE, que são IDEs (*Integrated Development Environment*). O ambiente deve estar configurado e com *plug-ins* necessários. Serão utilizadas as duas IDEs, porém o **Netbeans** completo abrange todos os exemplos das lições.

Link

Site oficial da ORACLE para download da IDE com JDK. Disponível em: <http://www.oracle.com/technetwork/pt/java/javase/downloads/jdk-netbeans-jsp-142931.html>. Acesso em: 10 jul. 2016.

1.1 Descrições de especificações JEE

Há um grande conjunto de especificações do Java Enterprise Edition (JEE), e algumas *Application Programming Interfaces* (APIs) podem ser destacadas para ter uma melhor visão da tecnologia. As APIs no (**Quadro 1**) destacam-se dentre as disponibilizadas pelo Java Enterprise:

Quadro 1 – Especificação de APIs do JEE

APIs	Descrição
JavaServer Pages (JSP) Java Server Faces (JSF)	(páginas Web)
Enterprise Javabeans Components (EJB) Java Persistence API (JPA)	(objetos distribuídos, persistência de dados)
Java API for XML Web Services (JAX-WS) Java API for XML Binding (JAX-B)	(arquivos XML e webservices)
Java Authentication and Authorization Service (JAAS)	(Segurança)
Java Transaction API (JTA)	(controle de transação no contêiner)
Java Message Service (JMS)	(troca de mensagens assíncronas)
Java Naming and Directory Interface (JNDI)	(espaço de nomes e objetos)
Java Management Extensions (JMX)	(administração da sua aplicação e estatísticas sobre a mesma)

Fonte: Adaptado de descrição original da (CAELUM, 2016) e (ORACLE, 2016).

Você inicialmente utilizará páginas JSPs e *Servlets* para o desenvolvimento e posteriormente será empregado JSF e PrimeFaces para o entendimento dos diferentes tipos de aplicações em camada.

Serão estudados outros conceitos da tecnologia Web, dentre eles HTML e XML (*stands for EXtensible Markup Language*), bem como camada de persistência, ou seja, conexão com dados utilizando padrão *Data Access Object* (DAO) e com *Java Persistence* API (JPA) e Hibernate.

Glossário

Netbeans: é uma IDE que engloba diversas tecnologias.

Persistência: é o nome dado para a conexão com alguma base de dados; em Java, são utilizadas especificações como o JPA, que é uma API responsável em fazer o processo de mapeamento objeto-relacional.

.NET: é a tecnologia da *Microsoft* que utiliza um framework para execução de sistemas e aplicação.

Questão para reflexão



A tecnologia *Java Enterprise Edition* (JEE) contém especificações padronizadas e há um arcabouço cultural pelos desenvolvedores e empresas que a utilizam. Reflita como essa tecnologia Web pode trazer vantagens comerciais e faça uma pesquisa de empresas que a utilizam.

Considerações Finais

- Conhecer uma tecnologia Web, que tem grande aplicabilidade e é consolidada no mercado de trabalho, pode ser uma vantagem;
- Entender os principais conceitos envolvidos no ambiente de desenvolvimento de *software* é essencial para um profissional Web;
- Estudar as principais especificações do *Java Enterprise Edition* (JEE) ampliará o conhecimento relacionado com a tecnologia estudada;
- Fatores de aplicação em JEE envolvem uma programação distribuída que utiliza servidores *Web* específicos.

Referências

APACHE. Servidor TOMCAT. Disponível em: <http://tomcat.apache.org/>. Acesso em: 30 jul. 2016.

OASIS. SOA. Disponível em: <http://www.pcs.usp.br/~pcs5002/oasis/soa-rm-csbr.pdf>. Acesso em: 12 jul. 2016.

CAELUM. O que é Java? Disponível em: <https://www.caelum.com.br/apostila-java-orientacao-objetos/o-que-e-java/#2-3-maquina-virtual>. Acesso em: 12 jul. 2016.

EDUARDO, Carlos. Introdução-JSTL-Java. Disponível em: <http://www.devmedia.com.br/introducao-jstl-java/23582>. Acesso em: 12 jul. 2016.

IBM. Websphere. Disponível em: <https://www-01.ibm.com/software/br/websphere/>. Acesso em: 12 jul. 2016.

JAVASE. Framework Java. Disponível em: <http://docs.oracle.com/javase/8/docs/>. Acesso em: 12 jul. 2016.

JDK. Java Development Kit (JDK). Disponível em: <http://www.oracle.com/technetwork/pt/java/javase/downloads/index.html>. Acesso em: 12 jun. 2016.

MARQUES, Rafael. Servidor Web. Disponível em: <http://pt.slideshare.net/raphaelamarques/servidor-web-8225869>. Acesso em: 12 jul. 2016.

ORACLE. The History of Java. Disponível em: <Technologyhttp://www.oracle.com/technetwork/java/javase/overview/javahistory-index-198355.html>. Acesso em: 12 jul. 2016.

ORACLE_2. JavaServer Faces Technology. Disponível em: <http://www.oracle.com/technetwork/java/javaee/overview/index.html>. Acesso em: 12 jul. 2016.

PRESSMAN, Roger; MAXIM, Bruce. **Engenharia de Software**. 8. ed. New York, McGraw Hill Brasil, 2016.

Assista a suas aulas



Aula 1 - Tema: Introdução às Tecnologias Java *Enterprise Edition.* Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/fa5f4117b3a87f5c21e084c183ff079d>.



Aula 1 - Tema: Introdução às Tecnologias Java *Enterprise Edition.* Bloco II

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/f0fec5076fced0bccac5b68c6c28b3d5>.

Questão 1

1. Qual desses procedimentos relaciona-se a uma aplicação Web dinâmica?

a) Uma aplicação é interpretada/compilada no browser.

b) Aplicação dinâmica é interpretada do lado do servidor.

c) Para aplicação dinâmica, é necessário um servidor Web.

d) A aplicação dinâmica é interpretada pelo browser.

e) A aplicação é interpretada no servidor.

Questão 2

2. Qual foi o propósito da criação do Java por James Gosling?

a) Aplicação para vídeo games.

b) Aplicativos para *Smarthphones*.

c) Aplicação para dispositivos encaixados.

d) Aplicação Web.

e) Aplicação Desktop.

Questão 3

3. Para programação distribuída com JEE, quais são os requisitos?

a) JDK + IIS + Visual Studio.

b) JDK + APACHE TOMCAT + Eclipse/Netbeans.

c) JDK + JRE + Eclipse.

d) JSE + JRE + Netbeans.

e) JSE + JDK + IIS + Eclipse/Netbeans.

Questão 4

4. Quais desses servidores *Web* suportam JEE?

a) IIS

b) APACHE

c) MySQL

d) FTP Server

e) Glassfish

Questão 5

5. Quais dessas tecnologias representa melhor o sistema de mensageria no JEE?

a) JSE

b) JRE

c) JSF

d) JTA

e) JMS

Gabarito

1. Resposta: C.

Qualquer aplicação dinâmica em um projeto Web é preciso ter um servidor Web, se não irá compilar. Uma aplicação estática não precisa de um servidor e é interpretada ao lado do cliente.

2. Resposta: C.

O Java foi originalmente projetado para dispositivos eletrodomésticos baseado na linguagem SmallTalk e que mais tarde foi utilizado para Web.

3. Resposta: B.

O Java *Development* Kit possui todas as especificações para o desenvolvimento em Java. Porém, é preciso ter um servidor *Web* e uma IDE para auxiliar no desenvolvimento do software.

4. Resposta: E.

Glassfish é um servidor Web que suporta aplicações *Java Enterprise Edition*.

5. Resposta: D.

O *Java Message Service* é a tecnologia específica para tratar serviços de mensagens.

Unidade 2

Web Container Necessidades de uma aplicação Web

Objetivos

1. Conceitos de Web container;
2. Ciclo de vida de um *Servlet*;
3. Introdução ao *JavaServer Pages* – JSP.

Introdução

A tecnologia Web é considerada de grande importância por utilizar protocolos de rede como HTTP (*Hypertext Transfer Protocol*). O funcionamento de aplicações *Web* cresce com o uso de dispositivos móveis, tais como tablets e smartphones que acessam a Internet.

As aplicações Web utilizam a arquitetura cliente/servidor, que engloba um conjunto de tecnologias desenvolvidas sob padrões definidos por instituições como **IEEE** (Instituto de Engenheiros Eletricistas e Eletrônicos).

Para saber mais

IEEE é a maior organização profissional que se dedica ao avanço tecnológico e responsável por diversos padrões. Disponível em: <http://www.ieee.org.br/>. Acesso em: 10 jul. 2016.

Os servidores mais utilizados são: Apache e IIS (*Internet Information Services*) (LINHARES et al., 2014), sendo que há diversos outros servidores Web com tecnologias diferentes.

Para você compreender o funcionamento do ciclo de vida de um Servlet, é preciso saber como funciona um Web *container* e, para isso, iremos estudar o servidor Web Apache TOMCAT e explorar suas funcionalidades.

A empresa responsável em manter o servidor TOMCAT é a: *Apache Software Foundation* (ASF), que é uma organização sem fins lucrativos que disponibiliza toda documentação e assuntos relevantes a essa tecnologia, sendo que está disponível gratuitamente no **site oficial da apache** (TOMCAT, 2016).

Link

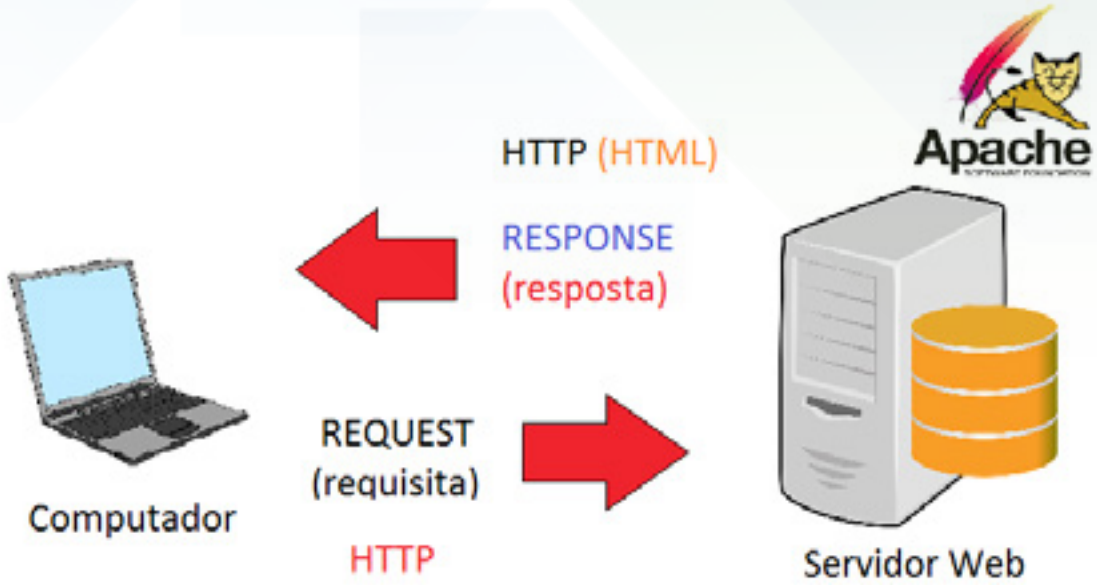
A Apache é uma das maiores empresas que disponibiliza diversos produtos para *download* e uso gratuito. É possível fazer o *download* do servidor TOMCAT pelo site. Disponível em: <https://tomcat.apache.org/download-80.cgi>. Acesso em: 20 jul. 2016.

1 Entendendo a arquitetura do TOMCAT

A arquitetura básica do Apache TOMCAT é o processamento em seu *container* das especificações: *JavaServer Pages* e *Servlets* e funciona com a versão do Java de acordo com a versão do TOMCAT.

Uma arquitetura Web está baseada em uma aplicação cliente/servidor, em que há requisições e resposta do servidor (*request/response)* como ilustrado na **(Figura 1):**

Figura 1 – Processamento servidor TOMCAT



Fonte: Elaborada pelo autor.

As páginas no browser do computador mandam ou requisitam informações que são processadas pelo recipiente do servidor Web. Dentro desse processo, são utilizados o *Servlet*, conhecido como um "servidorzinho", e a página HTML ou JSP.

Os códigos dentro dessas páginas utilizam em sua ação os *Servlets* criados no projeto; as páginas podem estar em HTML ou JSP. Entretanto, páginas HTML não processam códigos em Java

diretamente em sua página, como *scriptlets* (códigos em Java), mas chamam o *Servlets* no escopo dos formulários.

2. SERVLETS

Para entender o funcionamento do *Servlet*, é preciso ter conhecimento de como funciona um **protocolo HTTP**, que é responsável principalmente pela requisição feita pelo browser e resposta pelo servidor Web.

Link

A troca de informação pela Web utiliza protocolos que são necessários para a comunicação que envolve métodos dentro da rede de computadores. Conhecer esse funcionamento é fundamental para aplicações Web. Disponível em: <https://www.youtube.com/watch?v=94cwg-Rbm-NY>. Acesso em: 20 jul. 2016.

Serão utilizados os métodos GET e POST para a maioria das aplicações por oferecer as especificações necessárias para o funcionamento dos exemplos que serão demonstrados durante seu aprendizado.

Para saber mais

Java possui mais de um milhão de APIs (*Application Programming Interface*) para diversas aplicações (conexão de banco de dados, autenticação, interface visual etc.). A maioria dessas APIs está disponível para consulta. Disponível em: <https://docs.oracle.com/javase/7/docs/api/>. Acesso em: 20 jul. 2016.

Para o funcionamento do *Servlets* e JSPs, o servidor Web deve estar configurado adequadamente no Netbeans ou em outra IDE (*Integrated Development Environment*) para desenvolvimento Web.

Link

Para configuração do servidor Web TOMCAT no ambiente Eclipse, você deve seguir alguns passos necessários descritos no site a seguir: Disponível em: <http://www.ibm.com/developerworks/br/library/os-eclipse-tomcat/index_217033.html>. Acesso em: 20 jul. 2016.

A classe descrita a seguir é a formação da estrutura básica de um *Servlet* com construtor e os métodos *GET* e *POST*:

```
package web.curso.jee;
import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

/**
 * Servlet implementation class CursoServlet
 */
@WebServlet("/CursoServlet")
public class CursoServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
```

Pacote de onde se encontra o Servlet

APIs importadas para o Servlet

Comentários

Nome do Servlet

Herda a classe HttpServlet (API)

```
public CursoServlet() {
    super();
    // TODO Auto-generated constructor stub
}
    protected void doGET(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
        // TODO Auto-generated method stub
        response.GETWriter().append("Served at: ").append(request.GETContextPath());
    }
    protected void doPOST(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {
        // TODO Auto-generated method stub
        doGET(request, response);   }
}
```

Construtor

Método doGet

Método doPost

Todas as APIs necessárias com o protocolo HTTP são importadas automaticamente, e todos seus ***Servlets*** devem estar identificados e dentro de pacotes para estar organizado e seguindo as especificações da engenharia de software. Os métodos *GET e POST* agem de maneira semelhante, porém o GET deixa as informações explícitas na URL (*Uniform Resource Locator*) do navegador.

Será utilizado o método POST como demonstração do funcionamento do *Servlet,* que deverá ser a ação dentro de um formulário de uma página JSP. A seguir, você verá como imprimir uma mensagem no browser quando requisitado o *Servlet*, que foi criado com nome **cursoServlet.java**.



Link

Para saber mais sobre as especificações da tecnologia de Servlet, você pode acessar toda documentação das versões da JSRs (Java Specification Requests). Disponível em: <https://jcp.org/en/jsr/detail?id=315>. Acesso em: 20 jul. 2016.

```
        protected void doPOST(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {
            doGET(request, response);

   PrintWriter out = response.getWriter();

    out.println("Olá JAVA");

        }
}
```

out é um objeto declarado para saída vinda do import da API de

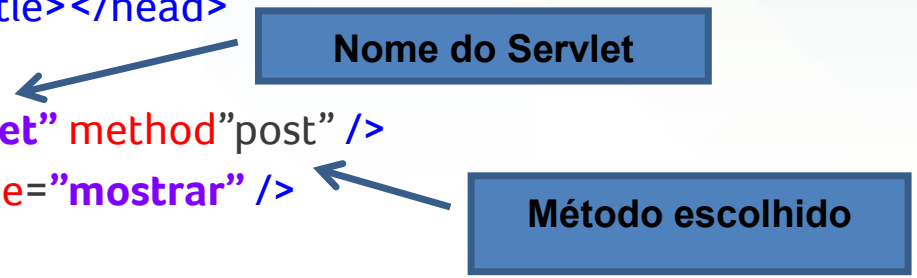
java.io.PrintWriter

Irá imprimir Olá JAVA

Com essa atualização em **cursoServlet** no método POST, será possível apresentar os dados em nosso browser. Para isso, deve ser chamado a página JSP no browser de nome: **index.jsp**, que contém o formulário, apenas para demonstrar como chamar esse *Servlet* na execução como no código a seguir:

Index.jsp

```
<html><head>
<meta http-equiv=”Content-Type” content=”text/html; charset=ISO-8859-1”>
<title>Insert title here</title></head>
<body>
<form action=”CursoServlet” method”post” />
<input type=”submit” value=”mostrar” />
</form>
</body>
</html>
```

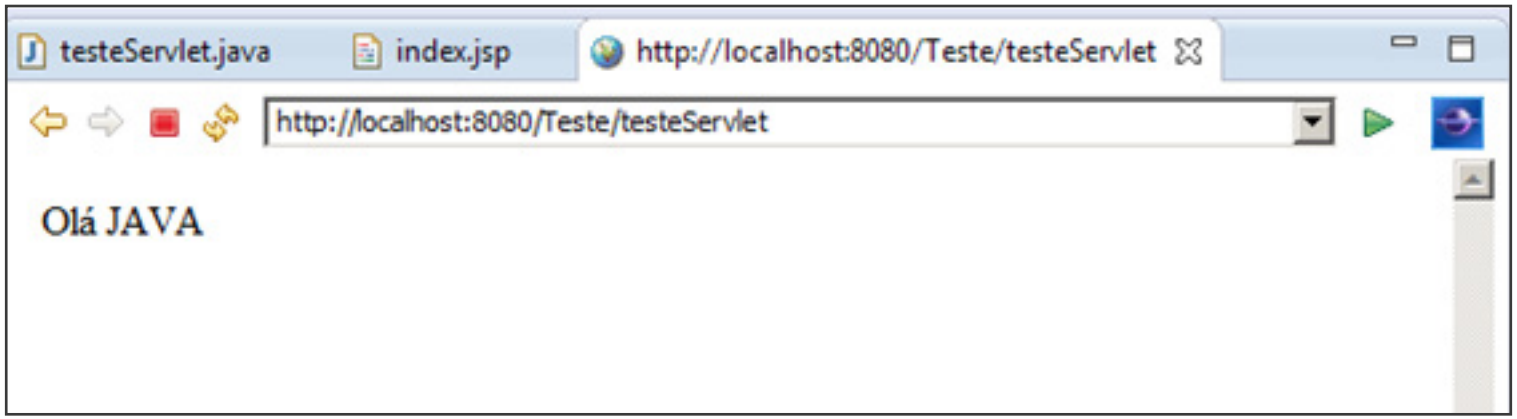


O resultado ao carregar o projeto será como na **Figura 2.** Caso ocorra algum erro, verifique o código e se configurou todo o ambiente do Eclipse ou Netbeans para o funcionamento correto, pois qualquer detalhe que faltar pode resultar erros.

Para saber mais

Em versões anteriores do *Servlet*, o mapeamento dessas classes era feito em um arquivo XML. Ainda há muitas aplicações que utilizam essas especificações. Disponível em: <http://blog.caelum.com.br/java-ee6-comecando-com-as-servlets-3-0/>. Acesso em: 21 jul. 2016.

Figura 2 – Resultado da demonstração com *Servlet*



Fonte: Elaborado pelo autor.

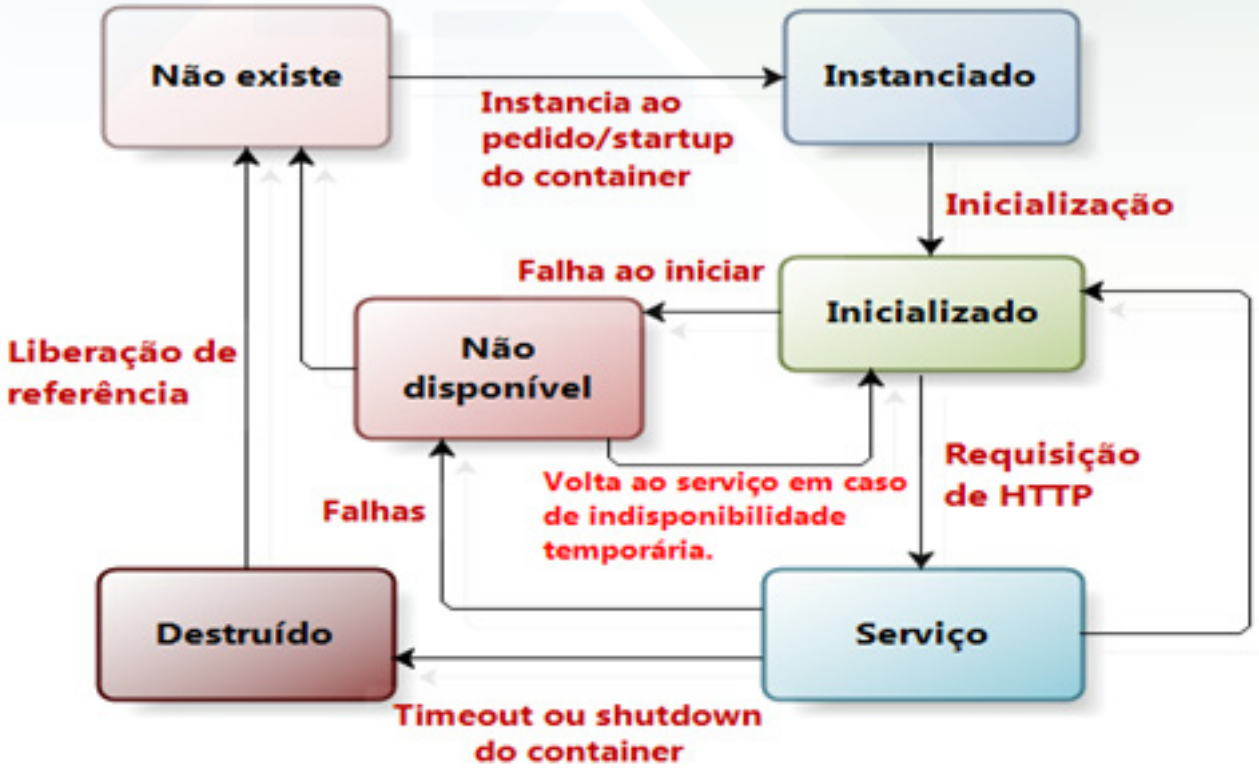
Lembre-se de que o mapeamento do *Servlet* é realizado por um arquivo **web.xml** nas especificações anteriores ao JEE 6. A partir das novas especificações, é utilizada a seguinte notação para o nome da classe:




```
@WebServlet("/CursoServlet")
public class CursoServlet extends HttpServlet {
```

Todo esse processo é realizado pelo *container* do servidor Web e engloba um ciclo de vida do *Servlet* que é requisitado. A **Figura 3** auxilia no entendimento de como funciona o ciclo de vida do *Servlet* segundo as especificações da JSR (JCP, 2016).

Figura 3 – Ciclo de vida do *Servlet*



Fonte: Adaptado de SAUVÉ (2016).

Segundo o professor Sauvé (2016), o ciclo de vida de um *Servlet* passa pelas seguintes fases apresentadas no **Quadro 2:**

Quadro 2 – Descrição das fases do um *Servlet*

Fases	Descrição
Instanciação	**O Web *container* cria uma instância do servlet**
Inicialização	**O Web *container* chama o método init() do servlet**
Serviço	**Se o *container* receber um pedido para o servlet, o método service() do servlet é chamado**
Destruição	**Antes de destruir a instância, o Web *container* chama o método destroy() do servlet**
Não disponível	**A instância é destruída e marcada para coleta de lixo**

Fonte: Adaptado de (SAUVÉ, 2016).

O processo do ciclo de vida do Web *container* possui todas as fases especificadas no quadro, sendo importante conhecer o funcionamento do *Servlet* para compreender o que acontece em seu projeto Web.

3. Introdução a *Java Server Pages* (JSP)

Páginas JSP (*JavaServer Pages*) possuem semelhanças às páginas HTML (*HyperText Markup Language*) e aceitam *scriptlet* que são códigos em Java no escopo da página (HUNTER, 2001 p. 37-42).

Os *scriptlets* utilizam esses símbolos **<% %>** dentro da página JSP, sendo que todo código em Java é colocado dentro dessas diretivas. As páginas JSP contêm outras diretivas importantes de

referência para usar recursos como JSTL (*JavaServer Pages Standard Template Library)* e a linguagem EL (*Expression Language*) (LUCKOW, 2010 p. 74-75).

No exemplo a seguir, você verá um escopo de uma página JSP com *scriptlets*, que nesse caso não utiliza o *Servlet* em sua requisição para processar e carregar o resultado no browser.

```
<html>
<head>
<title>Insert title here</title>
</head>
<body>
<%
if (request.getParameter(“nome”) == null){
   out.println(“Olá Java”);
} else {
   out.println(“olá, “ + request.getParameter(“nome”));
} %>
</body>
</html>
```



A tecnologia JSP é utilizada, principalmente, para separar a camada de apresentação da camada lógica da programação (HUNTER, 2001 p. 4-21).

Glossário

Container: recipiente.

TOMCAT: servidor Web.

MVC: *Model View Controller (Modelo Visão Controle).*

Questão para reflexão



Qual vantagem você encontra em utilizar notações para mapear *Servlet* em sua aplicação ao invés de utilizar um arquivo **web.xml** para fazer essa tarefa?

Considerações Finais

- Funcionamento de um Web container;
- Servidores Web;
- Ciclo de vida do *Servlet*;
- Funcionamento de páginas JSP.

Referências

HUNTER, Jason; CRAWFORD, William. **Java servlet programming**. “O’Reilly Media, Inc.”, 2001.

LINHARES, E.S. et al. TÓPICOS AVANÇADOS COMPARATIVOS ENTRE OS SERVIDORES WEB APACHE E IIS. **Revista Computação Aplicada – UNG**, v. 3, n. 1, p. 35-39, 2014.

LUCKOW, Décio Heinzelmann; MELO, Alexandre Altair. **Programação Java para a WEB**. São Paulo: Novatec, 2010.

SAUVE, Jacques Philippe. Introdução a Containers Web: A Primeira Aplicação Web com Servlets. Disponível em: <http://www.dsc.ufcg.edu.br/~jacques/cursos/j2ee/html/servlets/intro.htm>. Acesso em: 12 jul. 2016.

TOMCA. Apache Software Fundation. Disponível em: <https://tomcat.apache.org>. Acesso em: 12 jul. 2016.

Assista a suas aulas



Aula 2 - Tema: Web Container: Necessidades de uma aplicação Web. Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/311c2ee98b93faaad9081ab0c0372ae0>.



Aula 2 - Tema: Web Container: Necessidades de uma aplicação Web. Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/63543fe861828c82a6ac622a0a940217>.

Questão 1

1. Qual dessas opções é considerado um protocolo Web?

a) HTTP.

b) IEEE.

c) IOT.

d) POST.

e) GET.

Questão 2

2. Qual dessas alternativas se refere a um Web *container*?

a) É um servidor de requisição HTTP.

b) É a aplicação do *Servlet* com o JSP.

c) É conhecido também como um recipiente de *Servlet* .

d) É reconhecido com um componente do *Servlet* .

e) É interpretado como uma parte do ciclo do *Servlet* .

Questão 3

3. Qual desses servidores Web não suportam JEE?

a) Apache TOMCAT

b) JBOSS

c) Web Sphere

d) IIS

e) GlassFish

Questão 4

4. Qual desses métodos existe no *Servlet* e apresenta os dados na URL no browser quando processado?

a) *POST*

b) *GET*

c) *processRequest*

d) *delete*

e) DAO

Questão 5

5. O mapeamento automático com anotações foi implementado a partir de qual versão do JEE?

a) Versão 1.4 Java para Web

b) Versão 2.0 J2EE

c) Versão 5 JEE

d) Versão 6 JEE

e) Versão 7 JEE

Gabarito

1. Resposta: A.

HTTP é a única opção que representa um protocolo as alternativas não são protocolos.

2. Resposta: C.

Um Web *container* é um recipiente de *Servlet* que contém todas as especificações necessárias.

3. Resposta: D.

O servidor IIS é o servidor da Microsoft que não possui a tecnologia da Web *container*.

4. Resposta: B.

Não há o cuidado de ocultar informações no método *GET* como no *POST* que oculta os dados.

5. Resposta: D.

Foi implementado a partir da versão 6 do JEE como uma das melhorias na nova especificação da ORACLE/Sun.

Unidade 3

Persistência e Reaproveitamento de Código na utilização de sistemas *Web*

Objetivos

1. Persistência.
2. Reaproveitar dados.
3. Padronização.

Introdução

Em sua aplicação JEE, é possível se conectar a qualquer Sistema Gerenciador de Banco de Dados (SGBD) que contenha o *Driver* disponível para essa ação. Para conectar-se a uma base de dados, é preciso utilizar APIs que façam essa ligação e possibilitem fazer ações como: gravar, consultar, editar e excluir.

Segundo as especificações do Java, o JDBC (*Java Database Connectivity*) é capaz de fazer essa ligação de conexão com SGBD (JCP, 2016). A empresa ORACLE possui o SGBD mais famoso, que é o ORACLE (mesmo nome da companhia), e a empresa também possui um dos SGBD gratuitos mais utilizados, que é o **MySQL**.

Para saber mais

Há diversas ferramentas administrativas gratuitas para o MySQL no site da ORACLE. Além disso, você pode baixar o servidor MySQL diretamente do site. Disponível em: <http://www.oracle.com/br/products/mysql/overview/index.html>. Acesso em: 20 jul. 2016.

Para conectar-se e efetuar as ações do chamado CRUD (CREATE, READ, UPDATE E DELETE), será apresentado o padrão de conexão DAO (*Data Access Object*) junto com o banco de dados MySQL.

Com esse padrão, é possível ter um aproveitamento melhor no código e reutilizar sem precisar reescrever ou validar novamente. O padrão DAO utiliza os paradigmas da

programação orientada a objetos, que facilitam na herança e abstração das classes (CORDEIRO, 2014, p. 7-15).

Essa camada de persistência possui uma estrutura padrão, e, no exemplo a seguir, serão utilizadas as seguintes classes:

- Conexão com banco de dados: dentro dessa classe, há especificações de qual banco de dados será utilizado e a referência do *Driver* e o endereço do servidor que utiliza o JDBC como API, e é responsável em fazer a ligação.
- *GETTERS e SETTERS*: há uma classe que representa os mesmos atributos da tabela do banco de dados; nessa classe, são gerados os *GETTERS e SETTERS* para ações de transações de dados.
- CRUD: essa classe contém os métodos necessários para fazer as operações: gravar, consultar, editar e excluir.

Antes de começar a programar, você deve criar um banco de dados com nome: **cadastro** e uma tabela chamada **users** com os seguintes atributos (id_usuario, login, senha e email). Segue o código SQL para criar os requisitos no MySQL:

```
CREATE DATABASE cadastro;
```

Nome do banco de dados

```
CREATE TABLE users(
id_usuario int not null auto_increment,
login varchar(100) null,
senha varchar(100) null,
email varchar(160) null,
Primary key(id_usuario));
```

Nome da tabela e atributos

Para utilizar o MySQL no projeto com banco de dados, é preciso baixar o *Driver* de conexão no site. Esse arquivo **jar** deve ser colocado dentro de bibliotecas do projeto para que funcione.

Link

Você necessita baixar o arquivo e descompactar para ser utilizado dentro do projeto. Disponível em: <https://dev.mysql.com/downloads/connector/j/5.0.html>. Acesso em: 20 jul. 2016.

As classes estão dentro do pacote **bd.jee.connexao**, e, após criar um banco de dados e a tabela como indicado anteriormente, será possível desenvolver os códigos a seguir da classe **Conexão.java**:

```
package bd.jee.conexao;

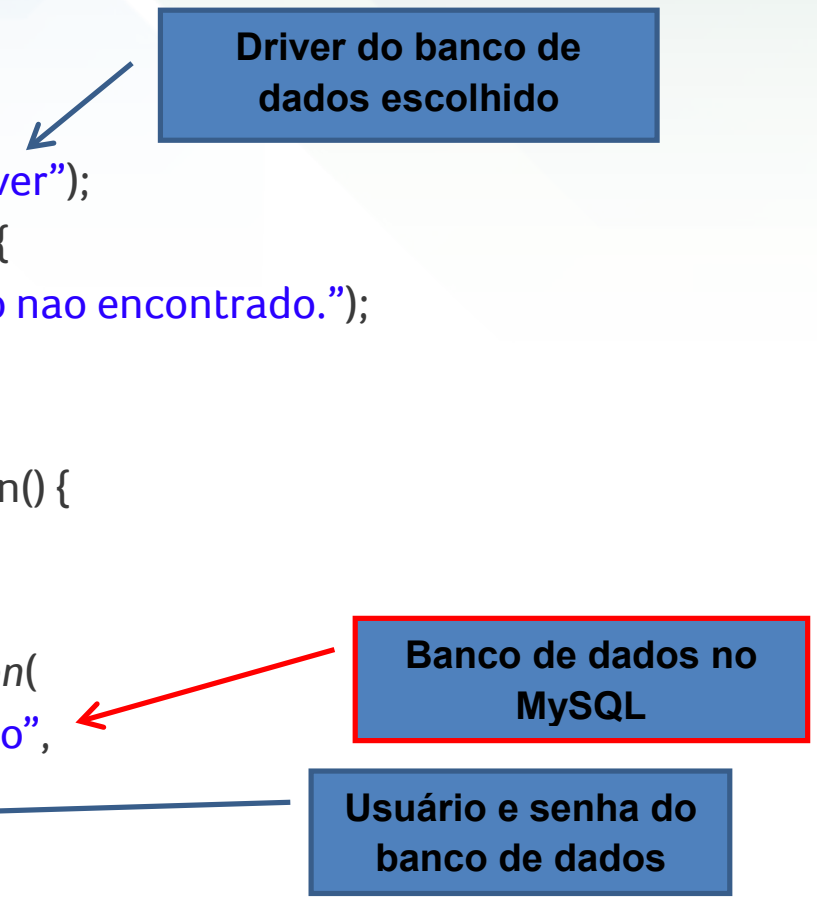
import java.io.*;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public final class Conexao {
    private Conexao(){
   PreparedStatement pstmt = null;
```



APIs necessárias para o funcionamento com banco de dados

```
}
static {
   try {
      //Carrega o driver do banco
      Class.forName("com.mysql.jdbc.Driver");
   } catch (ClassNotFoundException ex) {
      System.out.println("Driver do banco nao encontrado.");
      System.exit( -1);   }
}
public static Connection getConnection() {
   Connection conn = null;
   try {
      conn = DriverManager.getConnection(
            "jdbc:mysql://localhost/cadastro",
            "root",
            "1234"
            );
```



```
        } catch (SQLException ex) {
            System.err.print("Erro ao obter conexao: " + ex.getMessage());
        }
        return conn;   }
}
```

A próxima classe a ser criada é a classe **Dados.java** que representará os atributos do banco de dados para manipular e encapsular as informações.

```
package bd.jee.conexao;

public class Dados {
    private int id_usuario;
    private String login;
    private String senha;
    private String email;
```



```
//Gerar getters e Setters
}
```

Deve gerar os Getters e Setters em sua aplicação

Para completar o DAO, a próxima classe **DadosDao.java** terá todos os métodos necessários para o CRUD.

```
package bd.jee.conexao;

import java.sql.*;

public class DadosDAO {

    private static DadosDAO instance;

   static {
      instance = new DadosDAO();
   }


   public DadosDAO() {}
```

APIs necessárias para o funcionamento com banco de dados

Colocar todas como na classe anterior

```
public static DadosDAO getInstance() {
   return instance;
}
public Dados read(Dados dados) {

   Dados dadosAux = new Dados();
   Connection conn = Conexao.getConnection();
   try {
      Statement stmt = conn.createStatement();
      ResultSet rs = stmt.executeQuery(
            "SELECT * FROM users WHERE id_usuario = " +  dados.getId_usuario());
      if (rs.next()) {
         dadosAux = carregaDados(dados, rs);
      } else {
         dadosAux = null;
      }
      conn.close();
```

Método para ler os dados

```
    } catch (SQLException ex) {
        ex.printStackTrace();
    }
    return dadosAux;    }
```

Método para carregar os dados

```
private Dados carregaDados(Dados dados, ResultSet rs) throws SQLException {

    Dados dadosAux = new Dados();
    dadosAux.setId_usuario(rs.getString("Id_usuario"));
    dadosAux.setLogin(rs.getString("login"));
    dadosAux.setSenha(rs.getString("senha"));
    dadosAux.setEmail(rs.getString("email"));

    return dadosAux;
}
```

Método para apagar os dados

```
public boolean delete(Dados dados) {
```

```
    boolean retorno = true;
    Connection conn = Conexao.getConnection();
    try {
      Statement stmt = conn.createStatement();
      stmt.executeUpdate("DELETE FROM users WHERE id_usuario = " + dados.getId_usuario());
      conn.close();
    } catch (SQLException ex) {
      retorno = false;
      ex.printStackTrace();
    }
    return retorno;
}
public boolean create(Dados dados) {

    Connection conn = Conexao.getConnection();
    PreparedStatement pstmt;
```

Método para criar os dados

```
boolean retorno = true;
try {
   pstmt = conn.prepareStatement(
    “INSERT INTO users (id_usuario, login, senha, email) VALUES (?, ?, ?, ?)”);

   pstmt.setString(1, dados.getId_usuario());
   pstmt.setString(2, dados.getLogin());
   pstmt.setString(3, dados.getSenha());
   pstmt.setString(4, dados.getEmail());

   pstmt.executeUpdate();
   conn.close();
} catch (SQLException ex) {
   retorno = false;
   ex.printStackTrace();
}
return retorno;
```

}

Método para atualizar os dados

```
public boolean update(Dados dados) {

   boolean retorno = true;
   Connection conn = Conexao.getConnection();
   PreparedStatement pstmt;
   try {
      pstmt = conn.prepareStatement(
            "UPDATE users SET "+ "nome=?, idade = ? "+"WHERE codigo = ?");
    pstmt.setString(1, dados.getId_usuario());
    pstmt.setString(2, dados.getLogin());
    pstmt.setString(3, dados.getSenha());
    pstmt.setString(4, dados.getEmail());
    pstmt.executeUpdate();
      conn.close();
   } catch (SQLException ex) {
```

```
            retorno = false;
            ex.printStackTrace();
        }
        return retorno;
    }
}
```

O **método DAO** é um padrão de conexão (*Design Pattern*) e há variações desse padrão que podem ser implementadas em um projeto dependendo da aplicação e do propósito. Nas especificações do site da ORACLE, é possível conhecer esses outros modelos.

Link

Há diversas representações do padrão DAO identificados com diagrama de classe para um melhor entendimento no site da ORACLE, que envolve especificações do JEE. Disponível em: <http://www.oracle.com/technetwork/java/dataaccessobject-138824.html>. Acesso em: 20 jul. 2016.

O padrão DAO facilita a reutilização do mesmo código em todo o projeto e é uma das opções que poderá escolher se estiver de acordo com as especificações de seu projeto. O padrão DAO abstrai e encapsula todos os dados para a utilização dos dados (ORACLE, 2016).

Para saber mais

Há uma organização responsável pela padronização da UML e que disponibiliza manuais e outros materiais para estudo. Disponível em: <http://www.omg.org/spec/UML/>. Acesso em: 20 jul. 2016.

1. Aplicando padrão DAO

Assim que o padrão DAO estiver pronto, você poderá utilizar junto às páginas JSP e *Servlets*. No exemplo proposto, será aplicado o padrão DAO para fazer as seguintes ações: gravar (*CREATE*).

Para saber mais

Há diversos padrões conhecidos como *Design Pattern* que fazem parte da UML. Disponível em: <https://www.youtube.com/watch?v=-N1AbH-Z1_yg>. Acesso em: 20 jul. 2016.

Com o exemplo utilizando a ação de cadastrar para demonstrar, você será capaz de desenvolver outras ações do CRUD. No projeto, iremos utilizar uma página **cadastro.**

jsp e desenvolver o formulário:

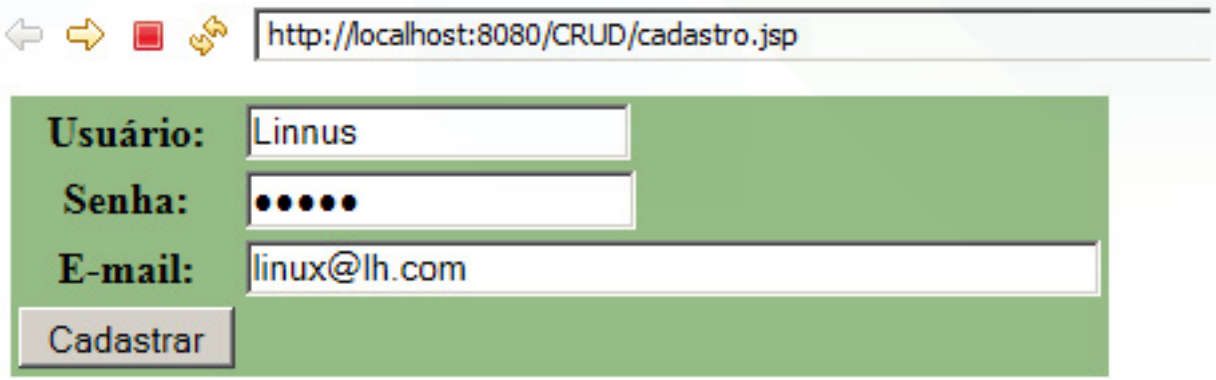
```
<html><head>
<title>Cadastro Web</title>
</head>
<body>
<FORM action="cadastroServlet" method="post">
<TABLE style="background-color: #95BB85;" WIDTH="40%" >
   <TR>
    <TH width="50%">Usuário:</TH>
<TD width="50%"><INPUT NAME="login" TYPE="text" maxlength="10" size="20%"></TD>
     </tr>
     <TR>
     <TH width="50%">Senha:</TH>
<TD  width="50%"><INPUT  NAME="senha"  TYPE="password"  maxlength="10"  size="20%"></
TD>
     </tr>
```



```
      <TR>
      <TH width=”50%”>E-mail:</TH>
<TD width=”50%”><INPUT NAME=”email” TYPE=”text” maxlength=”30” size=”50%”></TD>
      </tr>
<TD width=”50%”><INPUT TYPE=”submit” VALUE=”Cadastrar” ></TD> </tr>
  </TABLE>
   </FORM>
</body>
</html>
```

O formulário apresenta como resultado a **Figura 4:**

Figura 4 – Resultado ao carregar página cadastro.jsp.



Fonte: Elaborada pelo autor.

Para colocar as ações no botão, você deverá criar um *Servlet* com nome **cadastroServlet** e escolher o método POST que terá todos os códigos necessários em sua programação. Dentro do método POST, deverá ter os seguintes códigos:

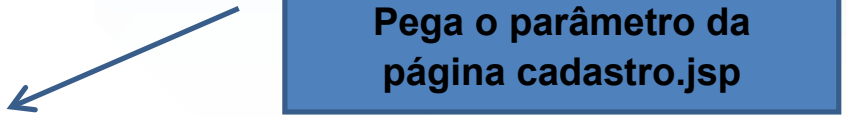
```
protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
```

```
//Declara session
HttpSession session = request.getSession(true);

DadosDAO dao = new DadosDAO();
Dados dado = new Dados();

 //Pega argumentos da página jsp
String login = request.getParameter("login");
String senha = request.getParameter("senha");
String email = request.getParameter("email");

//coloca valores para manipulae
dado.setLogin(login);
dado.setSenha(senha);
dado.setEmail(email);
//método para gravar do DAO
dao.create(dado);
```



Sessão para manipular os dados entre as páginas JSP

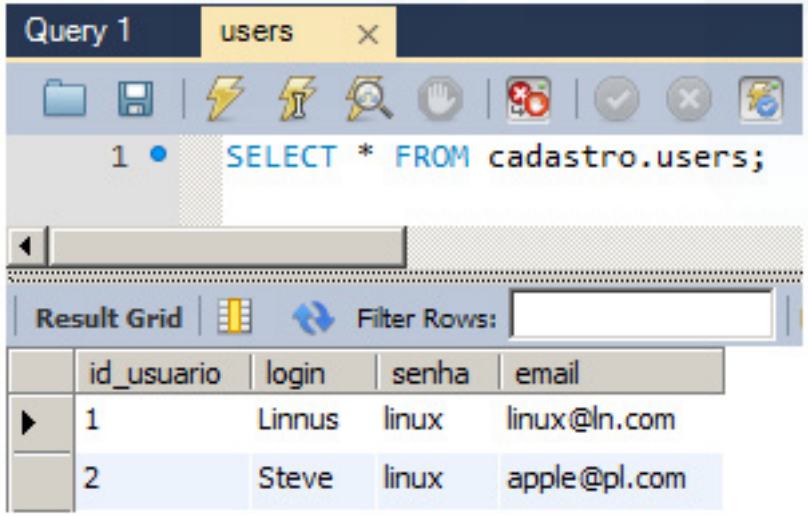
```
//session para manipular os dados entre as páginas JSP
 session.setAttribute("ulogin",dado.getLogin());
 session.setAttribute("usenha", dado.getSenha());
 session.setAttribute("uemail", dado.getEmail());

//Redireciona para outra página
 response.sendRedirect("resultado.jsp");
}
```

Deverá redirecionar para essa nova página JSP.

Esse código capturará o que foi digitado nos campos do formulário na página **cadastro.jsp** e chamará o **cadastroServlet** para fazer a ação de cadastrar. Dentro do código, você pode perceber que os parâmetros são colocados em variáveis e o método CREATE (que irá gravar) é chamado pelo objeto instânciado: **dao** que contém os métodos de ação para ser gravado no banco de dados conforme a **(Figura 5)**. Além disso, é utilizada a API de sessão que serve para navegar entre as sessões de uma página para outra e manipular os dados.

Figura 5 – Consulta de cadastro no banco de dados



Fonte: Elaborada pelo autor.

Para verificar o funcionamento do recurso de sessão, uma página **resultado.jsp** deve ser criada para receber os dados para demonstração:

```
<html>
<head>
<title>Insert title here</title>
</head>
<body>
<%
    String login = (String) session.getAttribute("ulogin");
    String senha = (String) session.getAttribute("usenha");
    String email = (String) session.getAttribute("uemail");
%>
  <form>
Login: :<br /><input type="text" name="codigo" value="<%= login%>" /><br />
Senha: :<br /><input type="text" name="nome" value="<%= senha%>" /><br />
E-mail:<br /><input type="text" name="idade" value="<%= email%>" /><br />
   </form>
</body>
</html>
```

Usando a sessão feita no cadastroServlet

Imprime o valor da sessão

O resultado é a impressão e é apresentado em um novo formulário na página **resultado.jsp**, como na **Figura 6**. Portanto, a aplicação utilizou recursos de sessão para manipular os dados e o padrão DAO.

Figura 6 – Resultado do uso da sessão



Fonte: Elaborada pelo autor.

Link

Para saber mais sobre a utilização de sessão, há uma documentação completa no site da ORACLE. Disponível em: <https://docs.oracle.com/cd/E23943_01/web.1111/e13712/sessions.htm#WBAPP300>. Acesso em: 21 jul. 2016.

A utilização do DAO não é o único método empregado para persistência de dados e há alguns frameworks que fazem todo processo de mapeamento relacional com a base de dados. Você verá na próxima lição a configuração do framework Hibernate para completar seu conhecimento.

Glossário

Persistência: conexão ou ligação com dados.

Encapsular: abstrair ou ocultar dados.

DAO: padrão de persistência.

Questão para reflexão

Atualmente, o padrão DAO seria a melhor opção para a maioria dos projetos com muitas tabelas?



Considerações Finais

- Controle de dados;
- Gerenciar códigos com persistência;
- Encapsular e padronizar dados;
- Reaproveitar código.

Referências

CORDEIRO, Gilliard. **Aplicações Java para a web com JSF e JPA**. São Paulo: Editora Casa do Código, 2014.

ORACLE. Acesso a dados. Disponível em: <http://www.oracle.com/technetwork/java/dataaccessobject-138824.html>. Acesso em: 12 jul. 2016.

Assista a suas aulas



Aula 3 - Tema: Persistência e reaproveitamento de código na utilização de sistemas web. Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f-1d/31a607ed011f6acdf0a3dc9774d08900>.



Aula 3 - Tema: Persistência e reaproveitamento de código na utilização de sistemas web. Bloco II

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/aae74eb7cca73374e60e899238e33504>.

Questão 1

1. Qual Driver padrão do Java é utilizado para conexão de banco de dados?

a) ODBC.

b) JDBC.

c) TOPLINK.

d) MYSQL.ODBC.

e) DERBY.

Questão 2

2. Qual dessas alternativas refere-se ao Driver do MySQL?

a) com.mysql.odbc.Driver

b) orm.mysql.jdbc.Driver

c) mysql.jdbc.odbc.Driver

d) com.jdbc.odbc.Driver

e) com.mysql.jdbc.Driver

Questão 3

3. Qual dessas páginas aceita *scriptlets*?

a) ASP

b) ASP.NET

c) HTML

d) PHP

e) JSP

Questão 4

4. Qual dessas alternativas representa melhor o CRUD?

a) CREATE, ROAD, UPDATE E DELETE.

b) CREATE, READ, UPDATE E DELETE.

c) CREATE, READ, SELECT E DELETE.

d) CREATE, INSERT, UPDATE E DELETE.

e) CREATE, READE, UPDATED E DELETE.

Questão 5

5. Qual dessas API é utilizada para sessão em JEE?

a) javax.servlet.http.HttpSession.Sessiom

b) java.servlet.http.HttpSession

c) javax.servlet.http.HttpSession

d) javax.servlet.HttpSession

e) java.http.HttpSession

Gabarito

1. Resposta: B.

O padrão do JAVA é o JDBC que está nas especificações da tecnologia.

2. Resposta: E.

A declaração do Driver correto é **com.mysql.jdbc.Driver.**

3. Resposta: E.

Scriptlets é utilizado apenas na linguagem JSP.

4. Resposta: B.

As siglas do CRUD é a abreviação de *CREATE, READ, UPDATE E DELETE*.

5. Resposta: C.

A API utilizada em *Servlets* para o protocolo HTTP é javax.servlet.http.HttpSession.

Unidade 4

Configurando o Hibernate e Integração JSF e JPA

Objetivos

1. Entender o JPA Hibernate.
2. Configurar Hibernate no ambiente de programação.
3. Integrar JPA Hibernate com framework *JavaServer Faces*.

Introdução

A utilização de banco de dados relacionais em Java é um grande desafio por tratar-se de um mundo de objetos. Essa persistência é realizada com processo de ORM (mapeamento objeto-relacional) facilitando a realização de tarefas complexas, como manipulação de dados, associando o mundo de objetos ao estruturado (KONDA, 2014 p. 18).

Para fazer ORM, é possível utilizar especificações do framework JPA (*Java Persistence* API), que é da própria especificação do Java, e também outro framework conhecido como Hibernate, que utilizaremos para integrar em uma aplicação com JSF (*JavaServer Faces*).

1 JPA

A especificação do JPA é um mapeamento objeto-relacional. O JPA foi desenvolvido para o EJB 3.0 (*Enterprise JavaBean*) e faz parte do grupo do JSR 220 (*Java Specification Requests*) e estende-se seu uso além do JEE (*Java Enterprise Edition*), por exemplo, em programação para *Desktop* (ORACLE, 2016).

O **framework JPA** facilita o gerenciamento de dados em aplicações Java e consiste em quatro áreas: *the Java Persistence API; the query language; the Java Persistence Criteria API e object/relation mapping metadata* (_____, 2016).

Para saber mais

A ORACLE possui um tutorial completo referente ao JPA e toda a especificação JEE para consulta gratuita. Disponível em: <http://docs.oracle.com/javaee/6/tutorial/doc/bnbpz.html>. Acesso em: 30 jul. 2016.

O JPA faz o mapeamento de classes POJOS (*Plain Old Java Objects*), que são **Beans** de Entidade: classes que utilizam *Java Persistence metadata*. Esse mapeamento utiliza anotação **@Entity** na classe que deve ter a mesma estrutura da tabela do banco de dados (TULIO, 2016).

Diversos frameworks implementam o JPA, que é uma especificação para realizarem a persistência em um banco de dados, dentre eles: Hibernate, EclipseLink e TopLink.

2. HIBERNATE

Por muito tempo, os desenvolvedores em Java utilizaram a API JDBC (*Java Database Connectivity*) para fazer conexões com banco de dados relacionais (MySQL, SQL Server, FireBird etc). Porém, em projetos grandes a utilização do JDBC era muito trabalhosa para os desenvolvedores e havia muita repetição de código (KONDA, 2014 p. 20).

Para esse propósito, a **organização Hibernate** fez o framework ORM de forma gratuita e muitos desenvolvedores adotaram por ser de código aberto e estar disponível para baixar no site oficial (HIBERNATE, 2016).



Link

Para baixar o framework do Hibernate e outras ferramentas, acesse o site oficial. Disponível em: <http://hibernate.org/>. Acesso em: 30 jul. 2016.

O Hibernate é um dos frameworks mais populares dentre os programadores Java e foi adquirida pela **Red Hat**, que é uma das empresas de software que mantém um dos servidores Web mais conhecidos, JBOSS, e toda documentação disponível para o usuário.

Para saber mais

Red Hat é uma empresa proveniente de uma das versões mais conhecidas do Linux e é a mantenedora do framework Hibernate, que disponibiliza on-line e gratuitamente um guia oficial de ótima qualidade. Disponível em: <https://docs.jboss.org/hibernate/orm/5.1/userguide/html_single/Hibernate_User_Guide.html>. Acesso em: 30 jul. 2016.

3. JSF

Um dos frameworks mais utilizados para camada de visão é o JSF (*JavaServer Faces*), que entrou para especificação do JEE (*Java Enterprise Edition*). É um padrão de mercado em diversas empresas (EDS, EMC, HP, IBM,

Apache) e contribui com ferramentas integradas ao JSF, dentre as quais temos: MyFaces e o ICE Faces (LUCKOW, 2010 p.72-74).

O JSF utiliza padrões de projetos (*design patterns*) da **engenharia de software**, principalmente o modelo MVC (*Model View Controller*), que possui a ideia em dividir a aplicação em três camadas: modelo, visualização e controle (_____, 2016).

Será apresentado como integrar essas três tecnologias no ambiente de programação **Netbeans**, a fim de ter um melhor resultado no seu desenvolvimento em aplicações Web.



Link

Uma das principais referências em engenharia de software é Roger Pressman. Os conhecimentos dessa área são essenciais para um profissional na área de desenvolvimento em TI. Disponível em: <http://loja.grupoa.com.br/autor/roger-s-pressman.aspx>. Acesso em: 30 jul. 2016.

Para saber mais

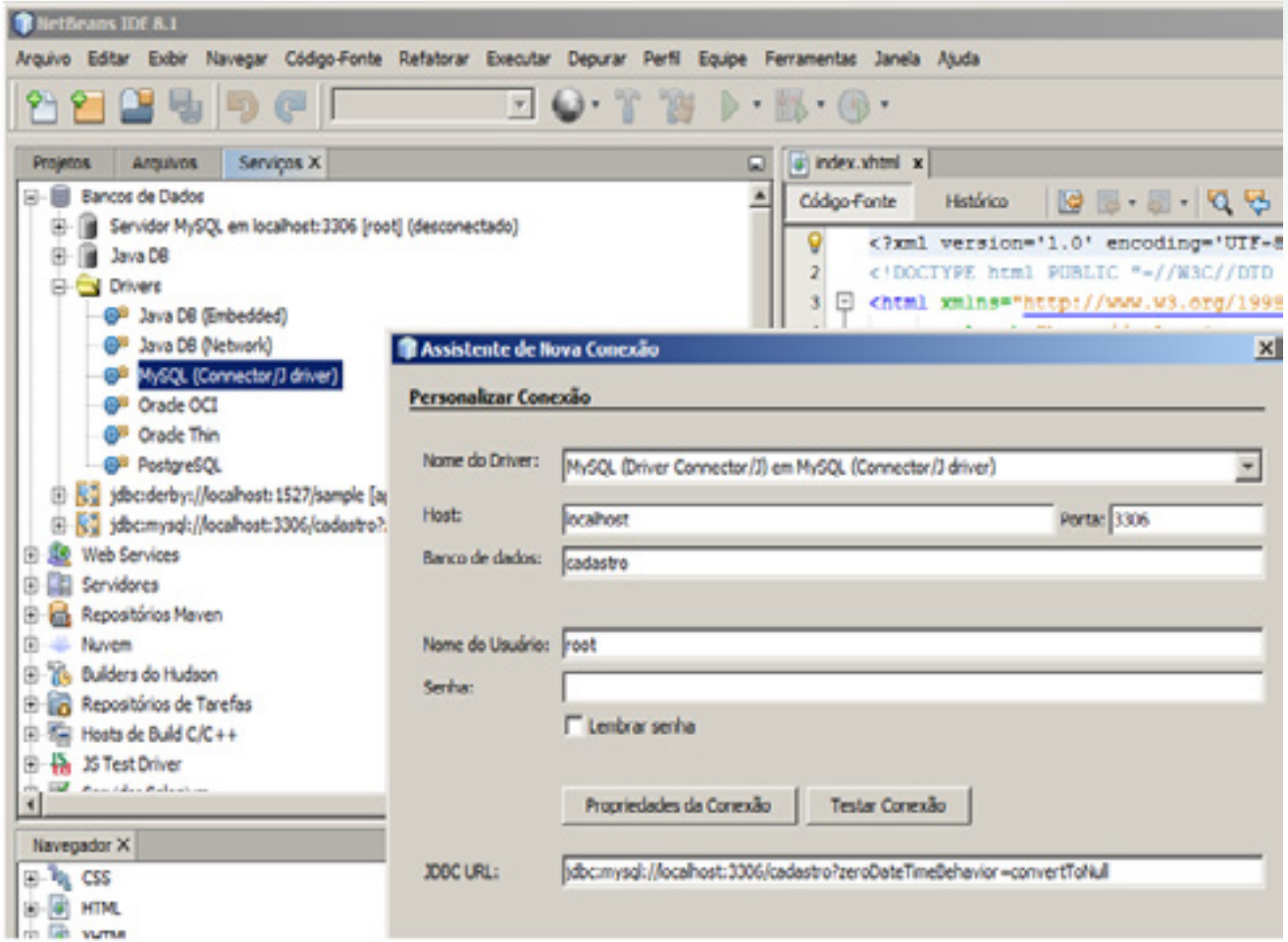
O Netbeans é um ambiente de desenvolvimento completo para tecnologia Java que possui diversos frameworks já disponibilizados como: Hibernate e JSF. Disponível em: <http://www.oracle.com/technetwork/pt/java/javase/downloads/jdk-netbeans-jsp-142931.html>. Acesso em: 2 ago. 2016.

4. Integrando JPA Hibernate com JSF

Para realizar a persistência, utilizaremos o mesmo banco de dados com nome: **cadastro** com a tabela **users** do capítulo anterior referente ao DAO. Inclua alguns registros na base de dados para ser apresentada em sua aplicação com JSF.

No Netbeans, você deve fazer a conexão com o banco de dados MySQL. Vá para serviços e, se não visualizar em seu Netbeans, vá ao menu em Ferramentas e encontrará serviços. Na tela de serviço dentro da pasta Drivers, você encontrará o MySQL (Connector) e deverá fazer a instância com seu banco de dados cadastro como na **Figura 7.**

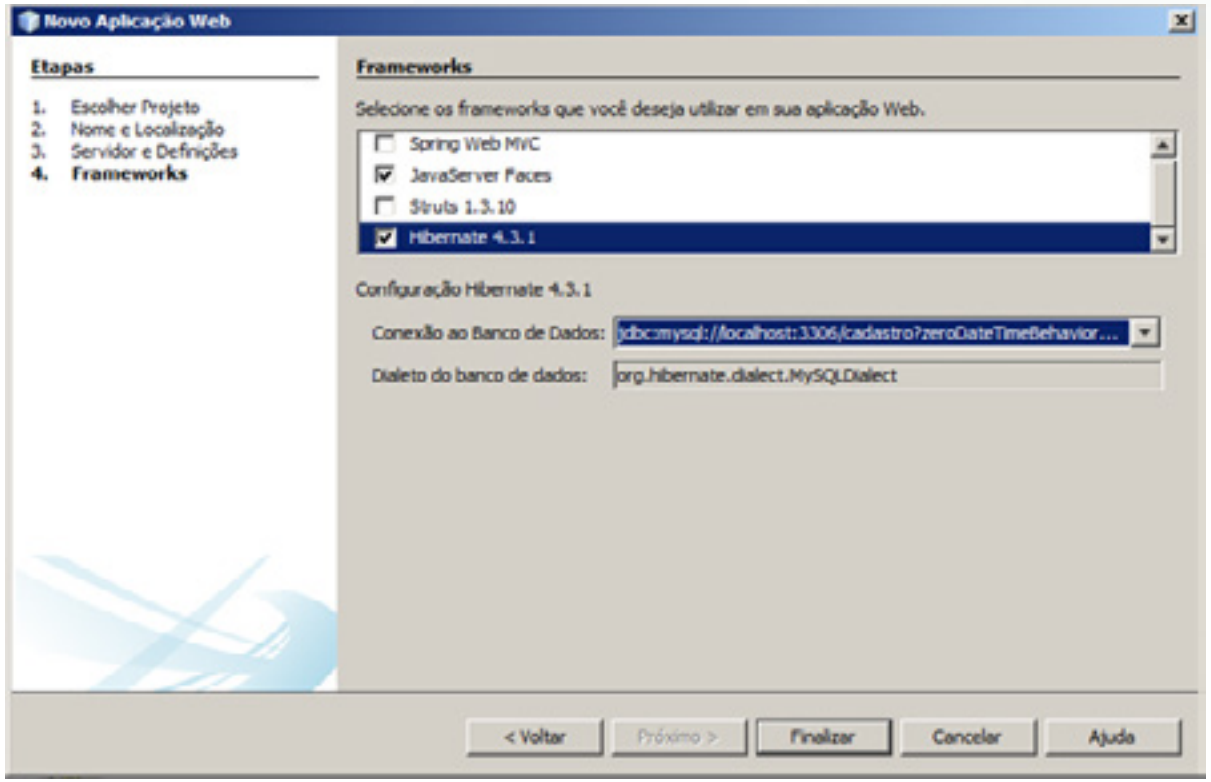
Figura 7 – Conexão com banco de dados



Fonte: Elaborada pelo autor.

Com a conexão estabelecida, você deverá criar um projeto Java *Web* com servidor GlassFish e, na tela de frameworks, escolher *JavaServer Faces* e Hibernate **(Figura 8).** Não se esqueça de escolher a conexão para o MySQL.

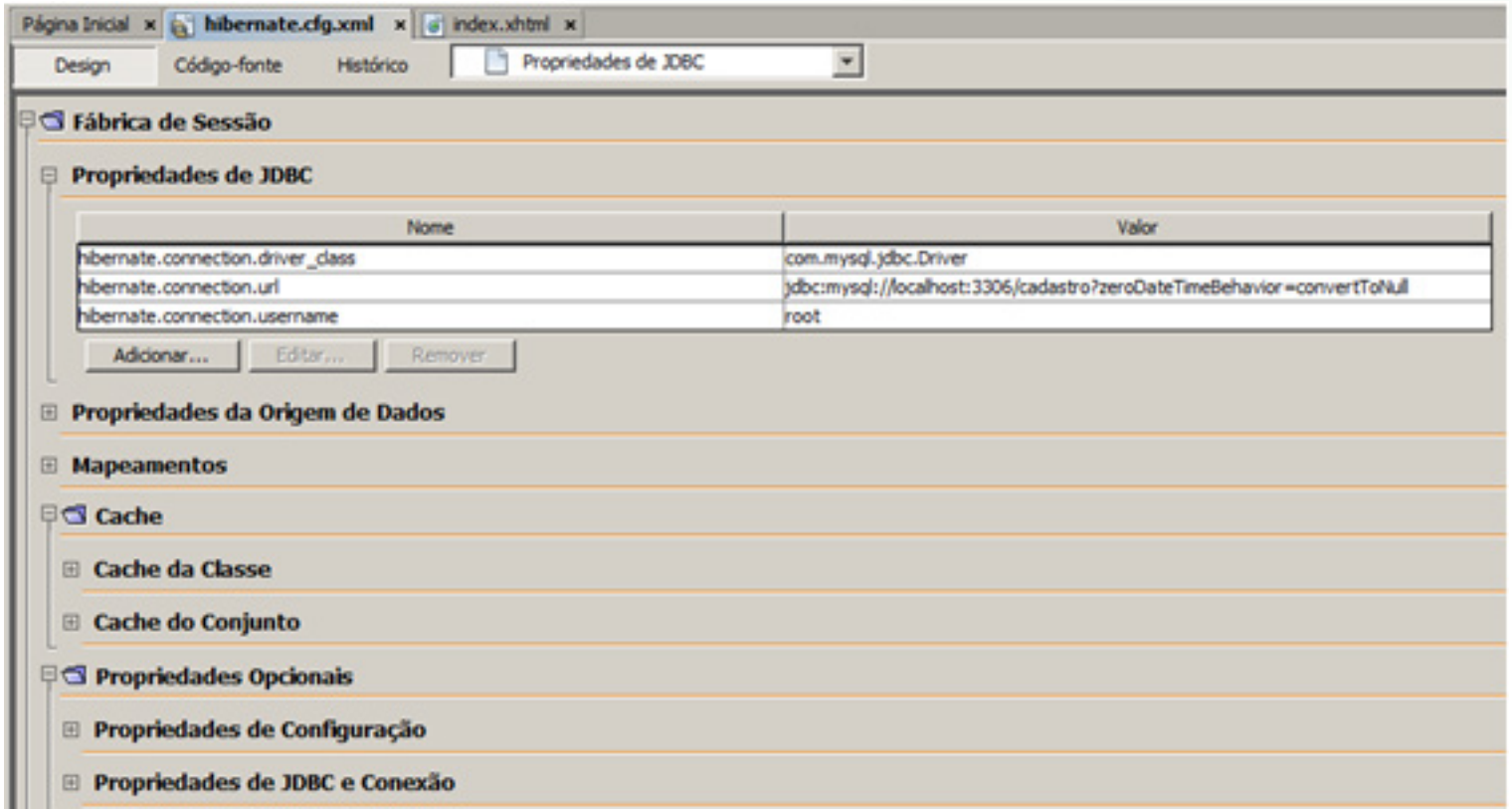
Figura 8 – Configurando projeto com JSF e Hibernate



Fonte: Elaborada pelo autor.

Ao criar o projeto, abrirá uma página **hibernate.cfg.xml (Figura 9)** contendo dados de conexão com o MySQL com usuário, senha (se houver) e uma página **index.xhtml**.

Figura 9 – Configuração hibernate.cfg.xml



Fonte: Elaborada pelo autor.

O código do arquivo XML deve ser:

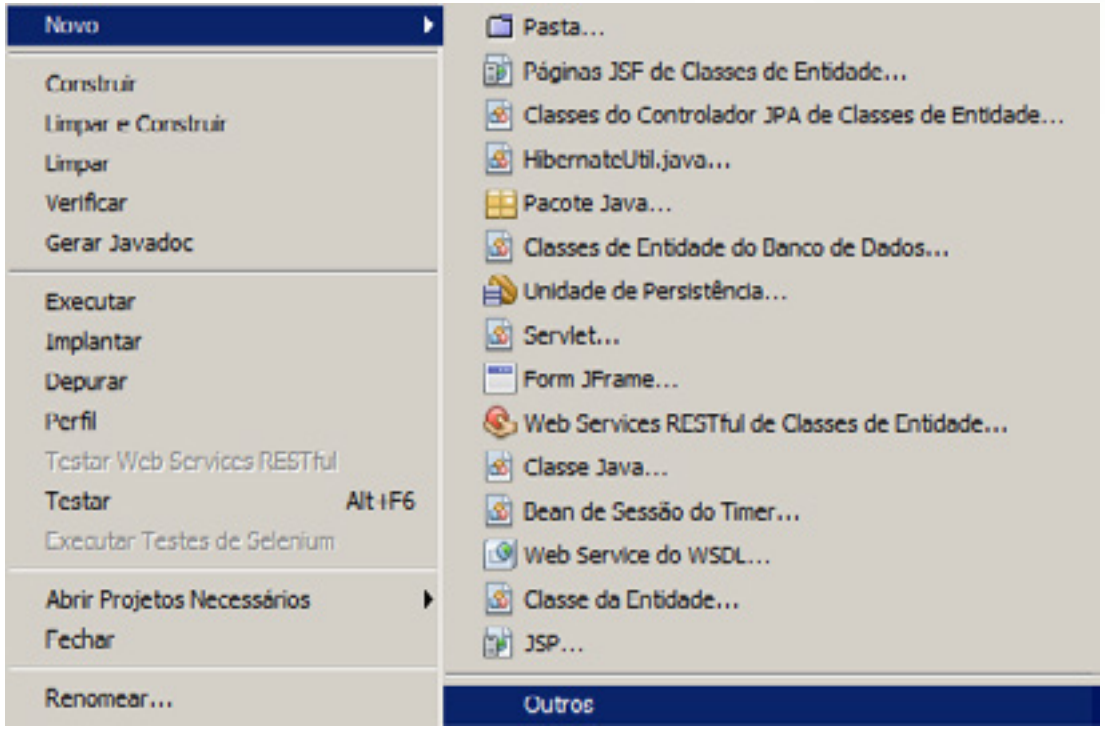
```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN" "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
 <session-factory>
  <property name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
  <property name="hibernate.connection.driver_class">com.mysql.jdbc.Driver</property>
  <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/cadastro?zeroDateTimeBehavior=convertToNull</property>
  <property name="hibernate.connection.username">root</property>
  <property name="hibernate.connection.password">suasenha</property>
 </session-factory>
</hibernate-configuration>
```



Caso houver senha, no banco de dados, aparecerá essa linha

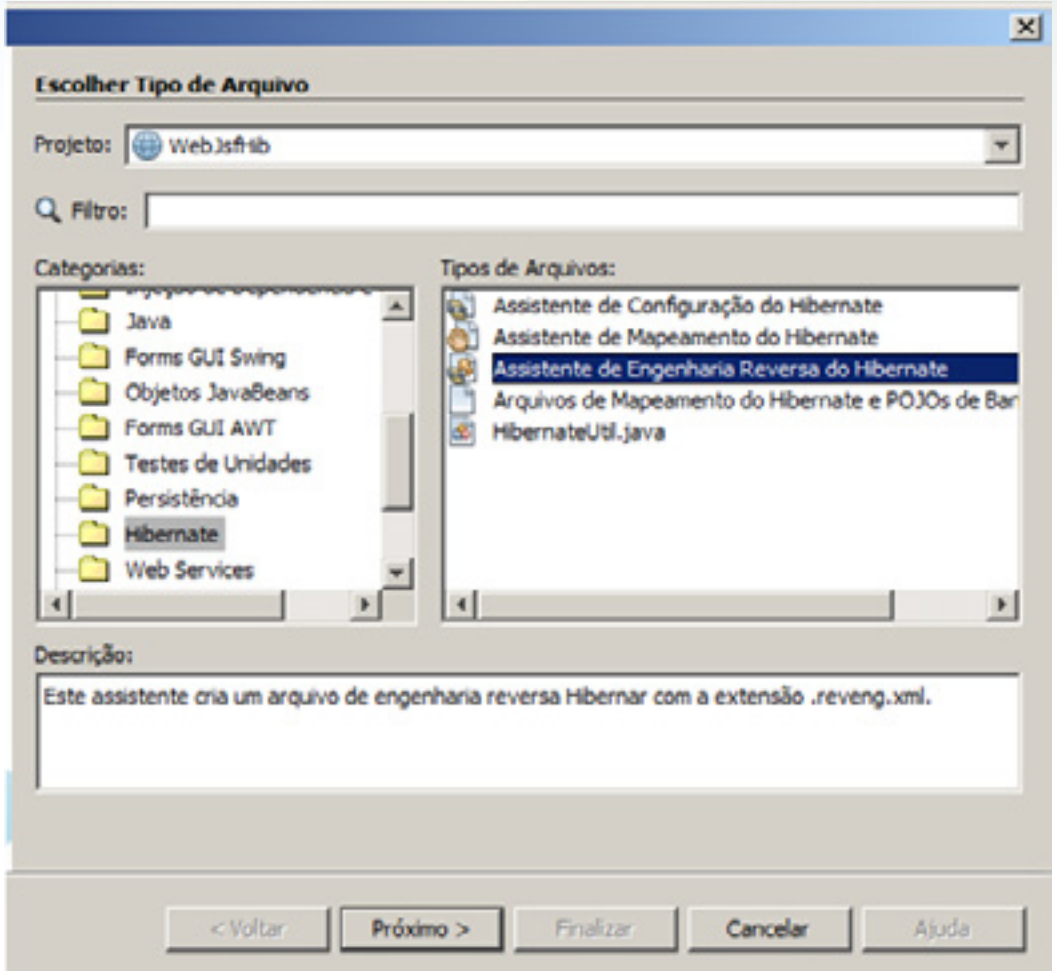
Com a conexão criada com Hibernate, o próximo passo é utilizar um: **Assistente de Engenharia Reversa do Hibernate.** Para isso, volte ao menu de opções do Netbeans e entre em: **Novo** e escolha a opção **outros (Figura 10)**: Em Tipos de Arquivos e escolha a opção do **Assistente de engenharia reversa (Figura 11).**

Figura 10 – Outras opções dentro do Netbeans



Fonte: Elaborada pelo autor.

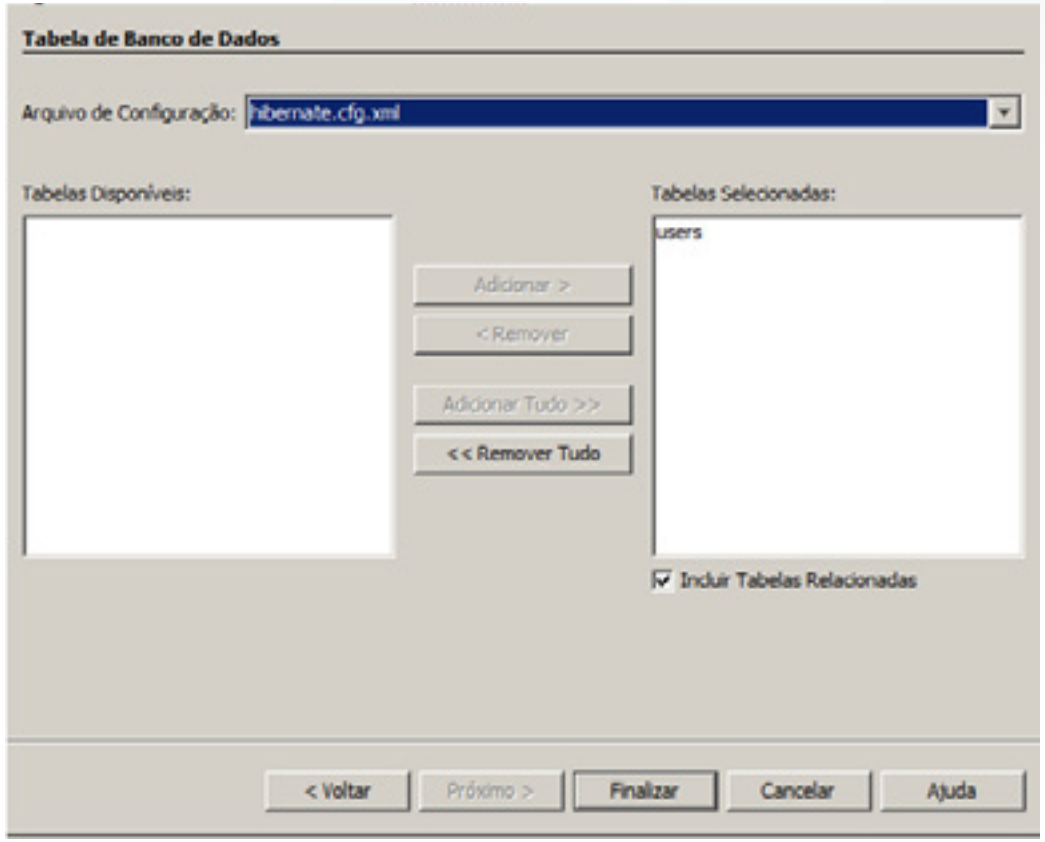
Figura 11 – Assistente do Hibernate



Fonte: Elaborada pelo autor.

Apenas avance até chegar à escolha de tabela que está no banco de dados e adicione a tabela escolhida como na **(Figura 12).**

Figura 12 – Escolha de tabela no Hibernate



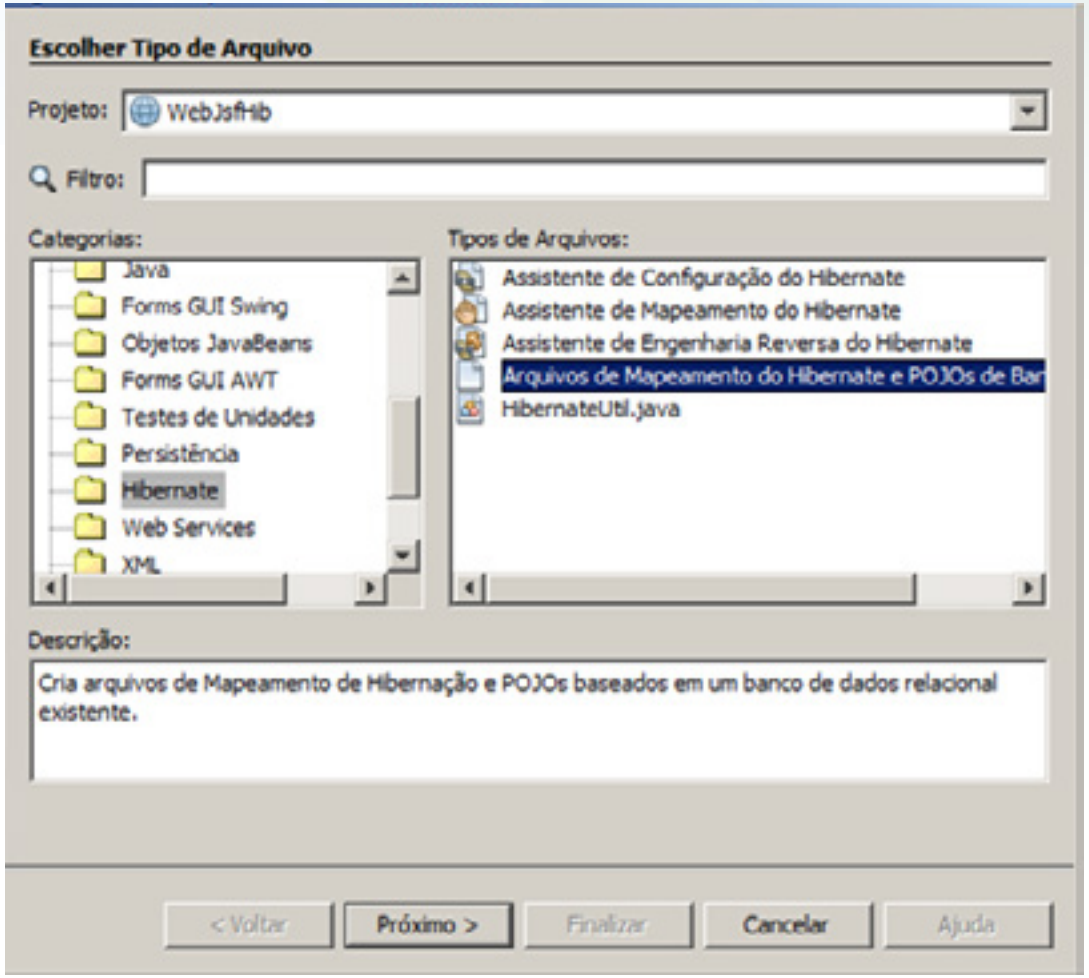
Fonte: Elaborada pelo autor.

Agora, o Hibernate gerará um arquivo chamado **hibernate.reveng.xml** com o seguinte código.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-reverse-engineering PUBLIC "-//Hibernate/Hibernate Reverse Engineering DTD 3.0//EN" "http://hibernate.sourceforge.net/hibernate-reverse-engineering-3.0.dtd">
<hibernate-reverse-engineering>
  <schema-selection match-catalog="cadastro"/>
  <table-filter match-name="users"/>
</hibernate-reverse-engineering>
```

Agora, você criará o POJO, que é a representação da persistência. Volte na pasta do Hibernate e escolha **(Figura 13)**: Arquivos de Mapeamento e POJOS de banco de dados.

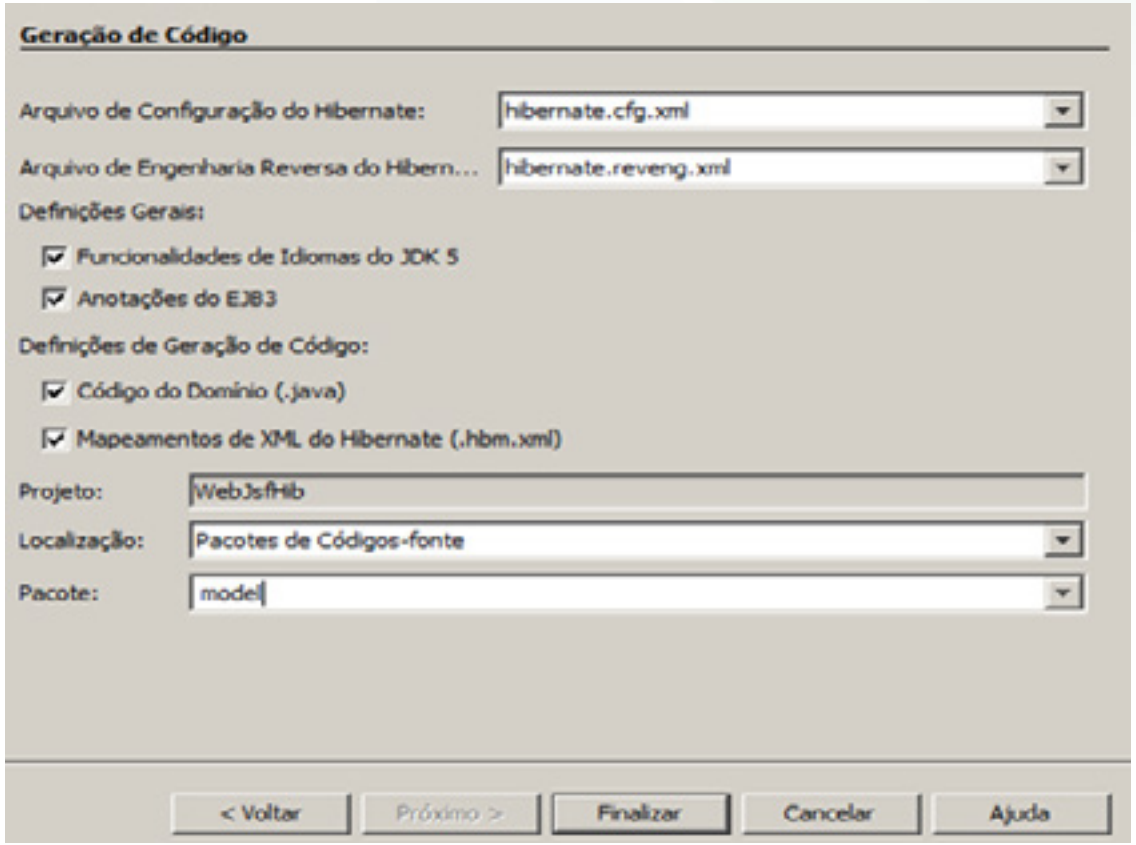
Figura 13 – Mapeamento do Hibernate



Fonte: Elaborada pelo autor.

Como na **Figura 14,** escolha o arquivo de engenharia reversa, marque todas as opções e nomeie o pacote de **model**.

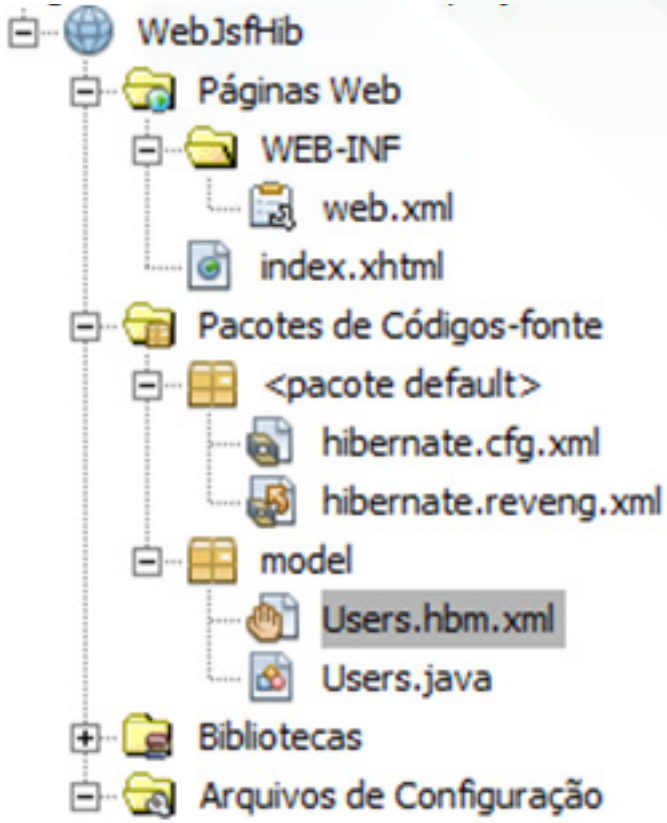
Figura 14 – Gerando POJO



Fonte: Elaborada pelo autor.

O seu projeto deverá ficar semelhante à **Figura 15:**

Figura 15 – Estrutura do projeto



Fonte: Elaborada pelo autor.

O arquivo **Users.hbm.xml** terá uma estrutura dos campos de sua tabela no banco de dados, como apresentado no código a seguir:

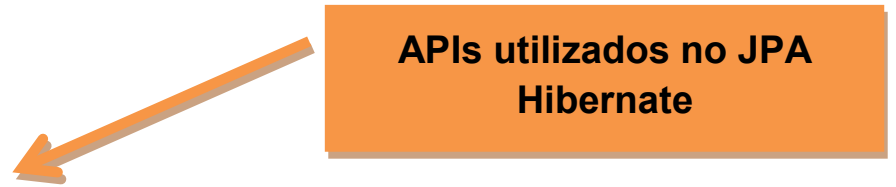
```
<?xml version="1.0"?>
<!-- Generated 02/08/2016 21:18:12 by Hibernate Tools 4.3.1 -->
<hibernate-mapping>
   <class name="model.Users" table="users" catalog="cadastro" optimistic-lock="version">
      <id name="idUsuario" type="java.lang.Integer">
         <column name="id_usuario" />
         <generator class="identity" />
      </id>
      <property name="login" type="string">
         <column name="login" length="100" />
      </property>
      <property name="senha" type="string">
         <column name="senha" length="100" />
      </property>
```

```
        <property name="email" type="string">
            <column name="email" length="160" />
        </property>
    </class>
</hibernate-mapping>
```

O arquivo **Users.java** contém todos *GETTERS* e *SETTERS* com as anotações do @Entity da JPA.

```
package model;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import static javax.persistence.GenerationType.IDENTITY;
import javax.persistence.Id;
import javax.persistence.Table;
```

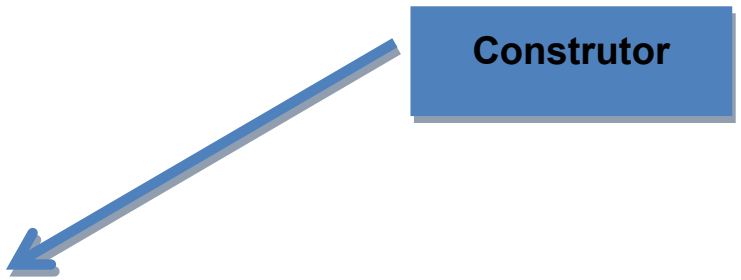





```
@Entity
@Table(name="users"
  ,catalog="cadastro"
)
public class Users  implements java.io.Serializable {

    private Integer idUsuario;
    private String login;
    private String senha;
    private String email;

  public Users() {
  }
  public Users(String login, String senha, String email) {
    this.login = login;
    this.senha = senha;
    this.email = email;
```



```
}
 @Id @GeneratedValue(strategy=IDENTITY)

 @Column(name="id_usuario", unique=true, nullable=false)
public Integer getIdUsuario() {
   return this.idUsuario;
}
 public void setIdUsuario(Integer idUsuario) {
   this.idUsuario = idUsuario;
}
 @Column(name="login", length=100)
public String getLogin() {
   return this.login;
}
 public void setLogin(String login) {
   this.login = login;
}
```



Anotação identificação da chave primária



GETTERS e SETTES

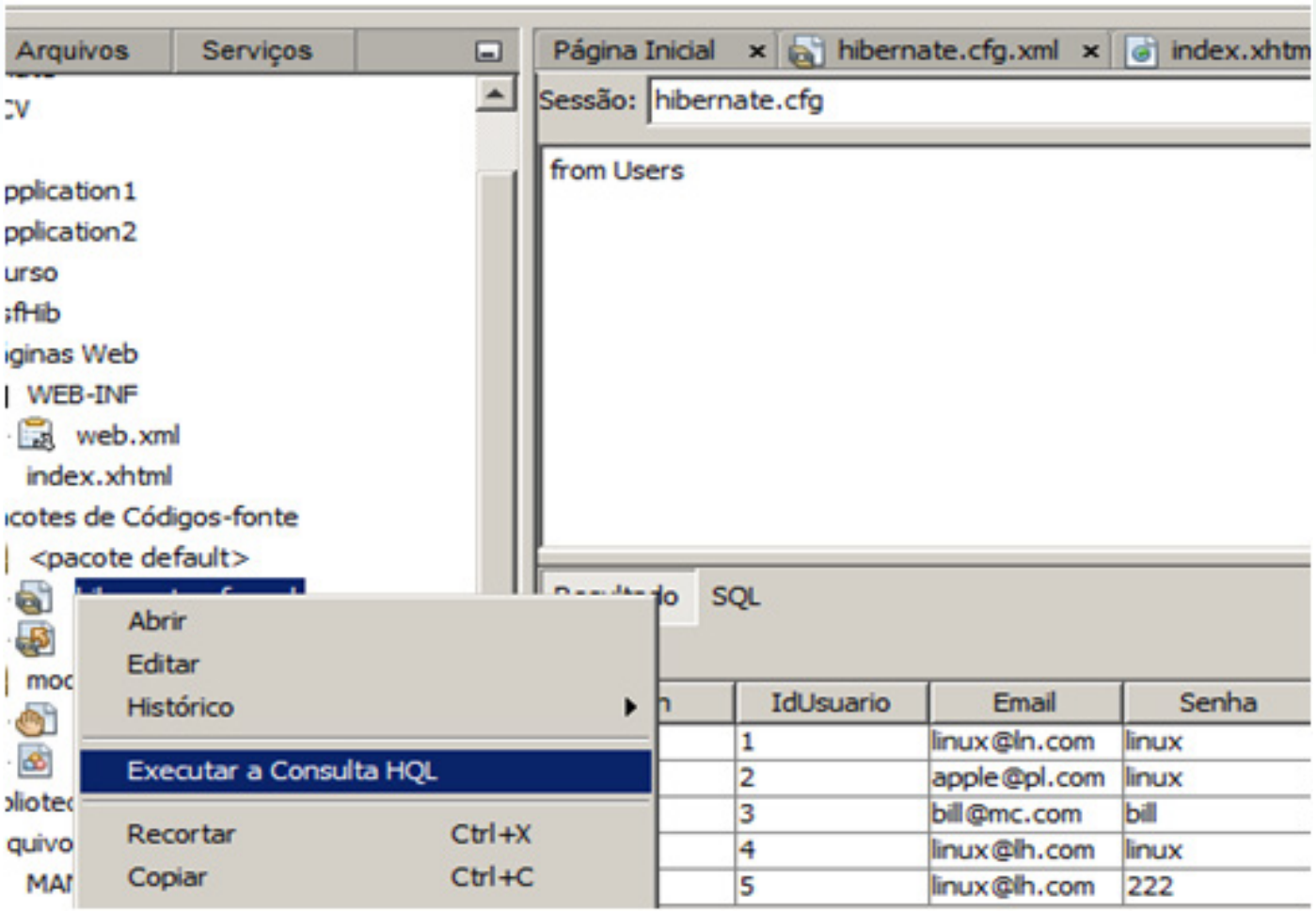
```
    @Column(name="senha", length=100)
    public String getSenha() {
        return this.senha;
    }
        public void setSenha(String senha) {
        this.senha = senha;
    }
    @Column(name="email", length=160)
    public String getEmail() {
        return this.email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
}
```

Com essa estrutura, é possível utilizar recursos da linguagem do Hibernate, o HQL. Para fazer consultas, por exemplo, clique com botão direito do mouse sobre o arquivo **hibernate.cfg.xml** e escolha a opção: Executar a consulta HQL **(Figura 16).**

Para saber mais

O Hibernate possui uma linguagem semelhante ao SQL que pode ser utilizada em sua programação. O HQL é uma das opções desse framework. Disponível em: <https://docs.jboss.org/hibernate/orm/3.5/reference/pt-BR/html/queryhql.html>. Acesso em: 2 ago. 2016.

Figura 16 – Consulta com HQL

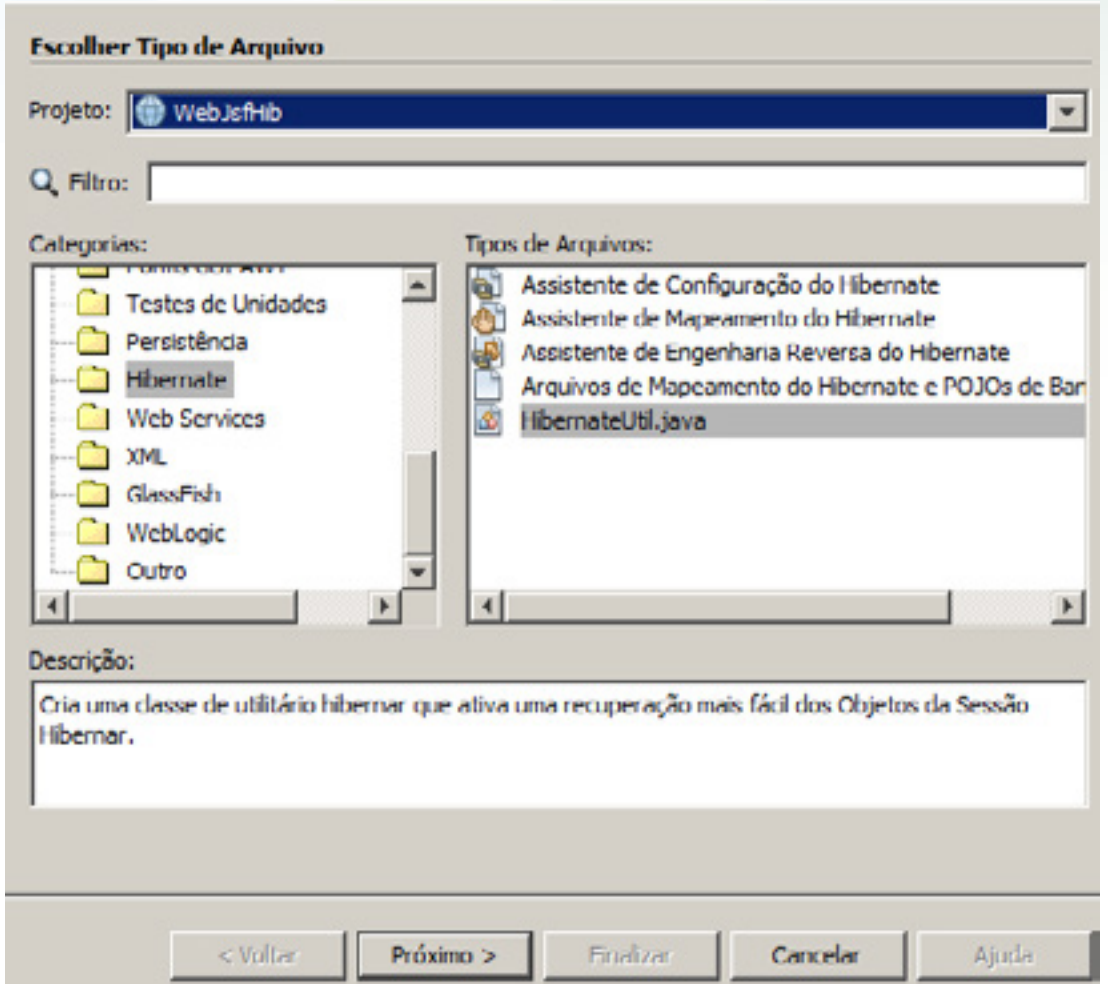


Fonte: Elaborada pelo autor.

Dentro da tela de consulta, poderá utilizar o HQL. Se, na primeira consulta, não funcionar, deverá atualizar o projeto (limpar e construir) e tentar novamente. Essa ação deverá consultar todos os dados inseridos em sua base na tabela **users**.

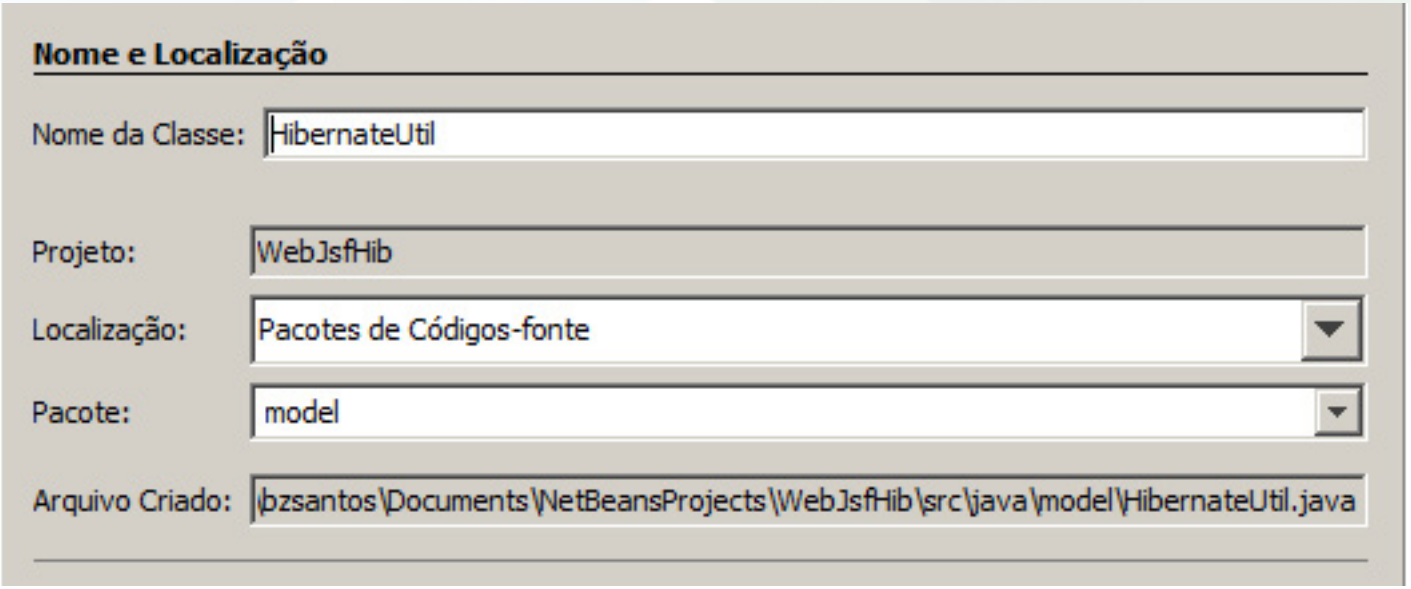
Agora, você configurará o arquivo **HibernateUtil.java**. Retorne à pasta do Hibernate, em que se encontram os arquivos de configuração, e escolha o arquivo como na **Figura 17**. Na próxima tela **(Figura 18)**, mude o nome de **NovoHibernateUtil** para **HibernateUtil** e coloque no mesmo pacote **model**.

Figura 17 – Inserir arquivo HibernateUtil



Fonte: Elaborada pelo autor.

Figura 18 – Nome e pacote do arquivo HibernateUtil



Fonte: Elaborada pelo autor.

Esse arquivo servirá para o Hibernate utilizar o recurso de sessão em suas transações, ou seja, para navegar de uma página para outra e utilizar dados. Dentro dessa classe, terá os seguintes códigos:

```
package model;

import org.hibernate.cfg.AnnotationConfiguration;
import org.hibernate.SessionFactory;

public class HibernateUtil {

  private static final SessionFactory sessionFactory;

  static {
    try {
      // Create the SessionFactory from standard (hibernate.cfg.xml)
      // config file.
      sessionFactory = new AnnotationConfiguration().configure().buildSessionFactory();
    } catch (Throwable ex) {
      // Log the exception.
      System.err.println("Initial SessionFactory creation failed." + ex);
```

A fábrica de sessão utiliza o arquivo hibernate.cfg.xml configurado para conexão no início da lição.

```
            throw new ExceptionInInitializerError(ex);
        }
    }

    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
}
```

Com a classe **HibernateUtil** configurada, agora você terá de criar uma classe com comandos que enviam informações para camada de visão que é utilizado o JSF. Na classe, coloque o nome de **Consulta.java** dentro de um novo pacote chamado **controller**. O seguinte código deve ter em sua classe:

```
package controller;
import model.HibernateUtil;
import model.Users;
import org.hibernate.SessionFactory;
```

```
public class Consulta {
    private Users user;

  public Users getUser() {
    return user;
  }

  public String sessao(){

    try{

    SessionFactory fab = HibernateUtil.getSessionFactory();
    fab.getCurrentSession().beginTransaction();
    this.user = (Users) fab.getCurrentSession().get(Users.class, 1);
    fab.getCurrentSession().getTransaction().commit();
    fab.getCurrentSession().close();
```

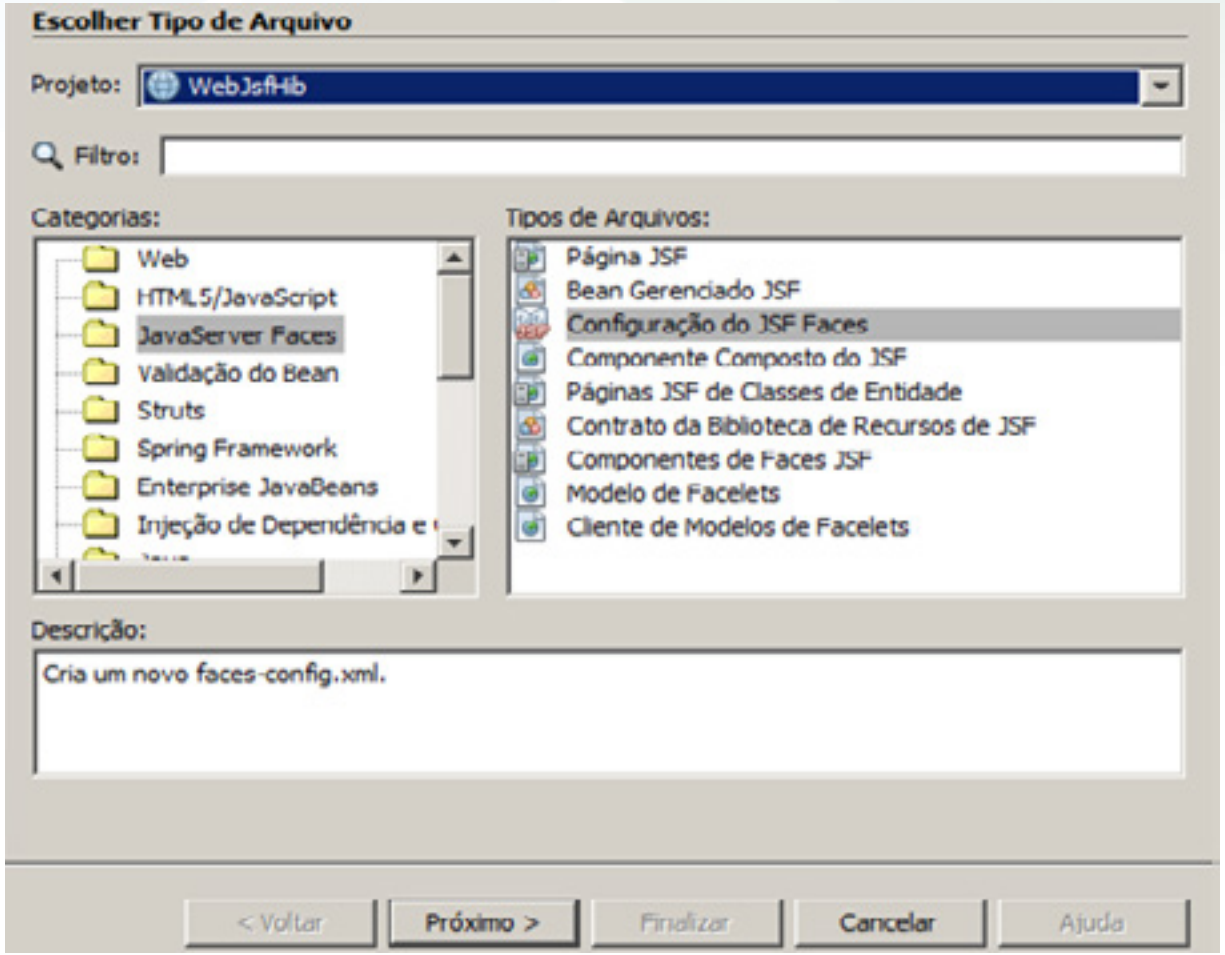
```
        } catch(Exception e){
            e.printStackTrace();
        }

        return null;
    }

}
```

Agora, você irá configurar o arquivo **faces-config.xml**. Escolha a opção, como na **Figura 19**, para adicionar o arquivo **xml** que terá um mapeamento de classe com **Beans**.

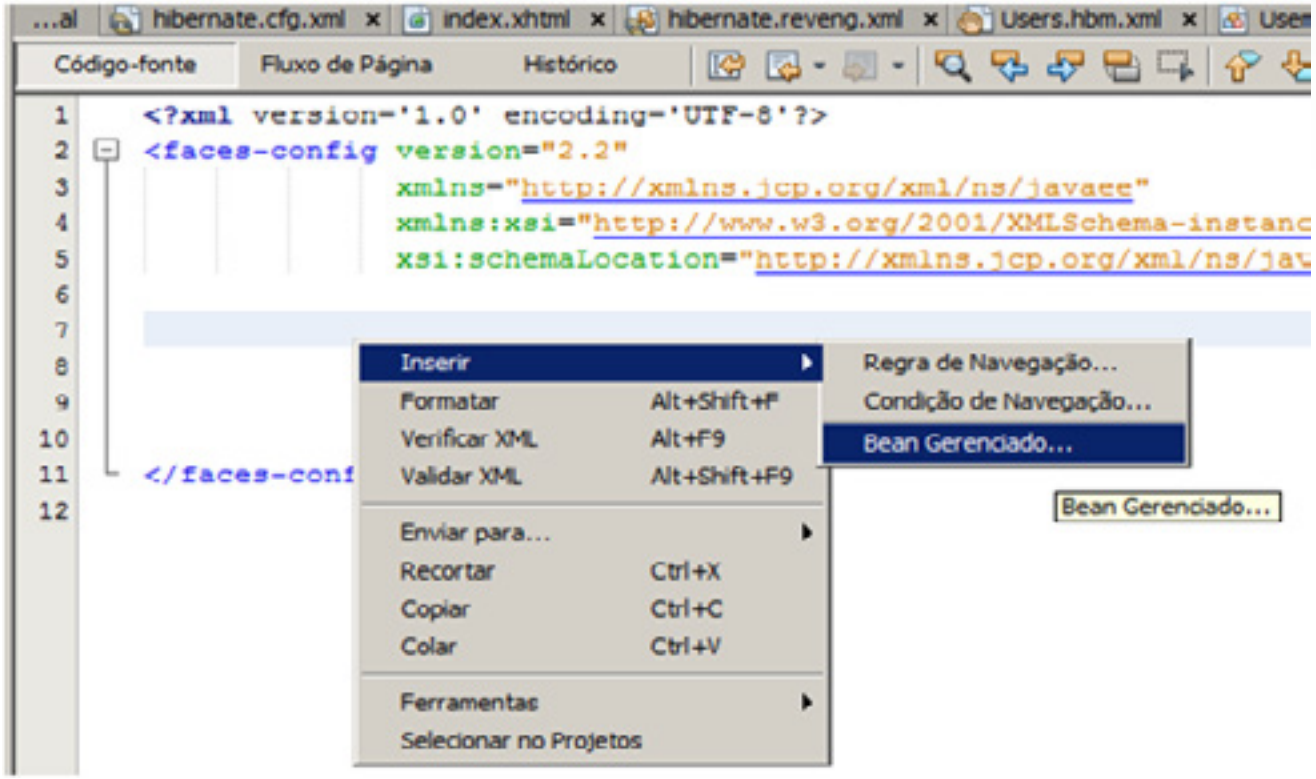
Figura 19 – Arquivo de configuração do JSF



Fonte: Elaborada pelo autor.

Quando o arquivo for criado, deverá inserir o **Bean** referente à classe que contém o acesso a sessão que permitirá navegar os dados na tabela **users** do seu banco cadastro no MySQL. Para inserir o **Bean (Figura 20)**, coloque o nome referente. No caso, será colocado o nome do **Bean** de **con**.

Figura 20 – Criando Bean no arquivo faces-config.xml.



Fonte: Elaborada pelo autor.

Você deve fornecer o nome sugerido ao **Bean** e localizar a classe **Consulta.java** para adicionar ao arquivo **xml**. Seu código deverá ficar semelhante ao descrito:

```
<?xml version='1.0' encoding='UTF-8'?>
<faces-config version="2.2"
        xmlns="http://xmlns.jcp.org/xml/ns/javaee"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee http://xmlns.jcp.org/xml/ns/
javaee/web-facesconfig_2_2.xsd">
   <managed-bean>
      <managed-bean-name>con</managed-bean-name>
      <managed-bean-class>controller.Consulta</managed-bean-class>
      <managed-bean-scope>request</managed-bean-scope>
   </managed-bean>
</faces-config>
```

Agora que configurou o seu arquivo **faces-config.xml**, você precisará atualizar seu arquivo **hibernate.cfg.xml** para que o framework entenda que utilizará a sessão.

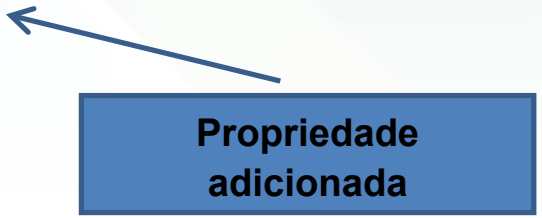
Deverá inserir uma linha que é uma propriedade referente à sessão:

```
<property name="hibernate.current_session_context_class">
org.hibernate.context.internal.ThreadLocalSessionContext</property>
```

Seu código deverá ficar semelhante ao código a seguir:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//
EN" "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
<session-factory>
<property name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
<property name="hibernate.connection.driver_class">com.mysql.jdbc.Driver</property>
<property     name="hibernate.connection.url">jdbc:mysql://localhost:3306/cadastro?zero-
```

```
DateTimeBehavior=convertToNull</property>
<property name="hibernate.connection.username">root</property>
<property    name="hibernate.current_session_context_class">org.hibernate.context.internal.ThreadLocalSessionContext</property>
<mapping resource="model/Users.hbm.xml"/>
</session-factory>
</hibernate-configuration>
```



Toda configuração do Hibernate está pronta, e, agora que você atualizou e criou o Bean no **aquivo xml** do JSF, é possível acessar os dados com Hibernate. Na página **index.xhtml**, faça uma atualização, como no código a seguir:

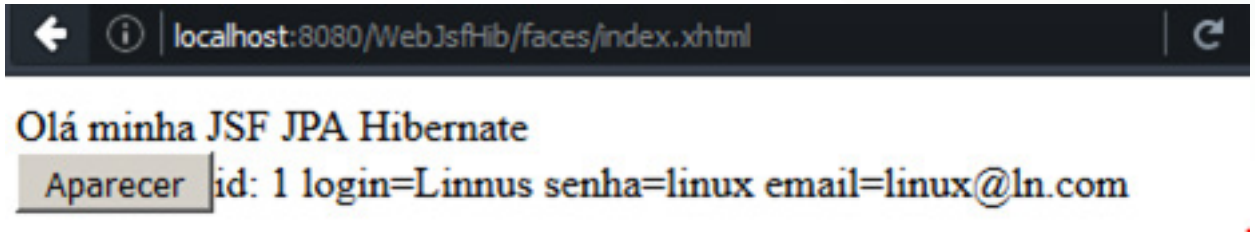
```
<?xml version='1.0' encoding='UTF-8' ?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:h="http://xmlns.jcp.org/jsf/html">
```

```
    <h:head>
      <title>Facelet Title</title>
    </h:head>
    <h:body>
       Olá minha JSF JPA Hibernate
      <h:form>
        <h:commandButton value="Aparecer" action="#{con.sessao()}" />
         <h:outputText id="teste" value="id: #{con.user.idUsuario} login=#{con.user.login} se-
nha=#{con.user.senha} email=#{con.user.email}" />
      </h:form>
    </h:body>
</html>
```

Na classe **Consulta.java**, foi determinado para trazer o primeiro registro gravado em sua base de dados. Se modificar esse registro para 2 ou 3, irá trazer os registros com esse número em sua: **id_usuario**.

Não se esqueça de incluir o *Driver* do MySQL nas Bibliotecas do projeto Web no Netbeans para que ele possa ler as requisições. Faça uma atualização (limpar e construir) em seu projeto e carregue para ver o resultado, que ficará como **na Figura 21**.

Figura 21 – Resultado da aplicação



Fonte: Elaborada pelo autor.

Com essa aplicação, você aprendeu a integrar as tecnologias JSF e JPA com Hibernate, que poderão servir como base para outras aplicações futuras. Há uma vasta documentação nos links disponibilizados para aperfeiçoar os estudos.

Glossário

JPA: *Java Persistence* API.

Hibernate: framework que utiliza JPA.

JSF: *JavaServer Faces*.

Questão para reflexão



Por que alguns desenvolvedores preferem utilizar JSP ao invés de JSF e qual dessas tecnologias você escolheria em sua aplicação?

Considerações Finais

- Persistência;
- frameworks que implementa o JPA;
- reaproveitamento de código;
- integração de tecnologias utilizando MVC.

Referências

KONDA, M. **Introdução ao Hibernate**. Boston: O'Reilly Media, 2014.

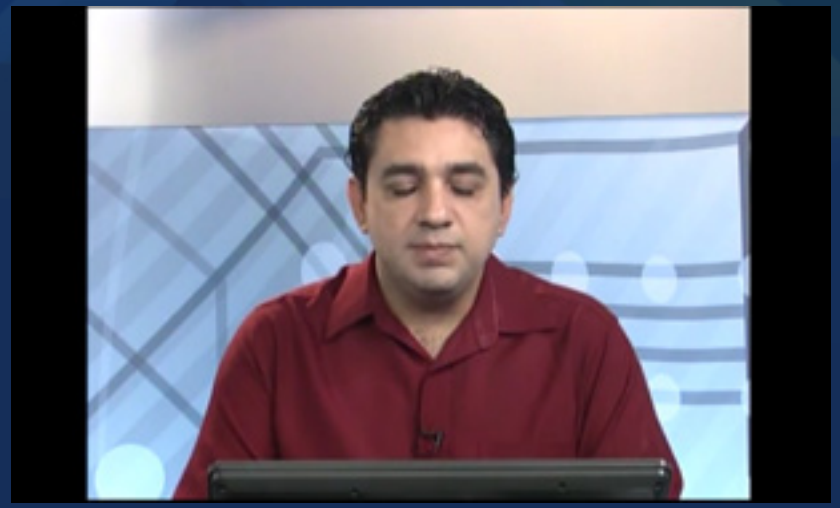
LUCKOW, Décio Heinzelmann; DE MELO, Alexandre Altair. **Programação Java para a WEB.** São Paulo: Novatec, 2010.

ORACLE. Persistência. Disponível em: <http://www.oracle.com/technetwork/java/javaee/tech/persistence-jsp-140049.html>. Acesso em jul. 2016.

_____. Tutorial JEE. Disponível em: <http://docs.oracle.com/javaee/6/tutorial/doc/bnbpz.html>. Acesso em: 30 ago. 2016.

TULIO. Aplicação CRUD com JSF2 + Hibernate + JPA + Primefaces. Disponível em: <http://www.lucianotulio.com.br/hibernate-primefaces/>. Acesso em: 30 ago. 2016.

Assista a suas aulas



Aula 4 - Tema: Configurando o hibernate e integração JSF e JPA. Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/28fe7804cccc06eda8da15b84d47025b>.



Aula 4 - Tema: Configurando o hibernate e integração JSF e JPA. Bloco II

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/670079be925bf67adfbc63ec6f8c22d7>.

Questão 1

1. Qual dessas especificações possui o JPA nativo?

a) JSP.

b) JSF.

c) EJB.

d) JSE.

e) JME.

Questão 2

2.Qual desses frameworks refere-se à camada *View* do MVC?

a) JSF.

b) Hibernate.

c) JPA.

d) TopLink.

e) EclipseLink.

Questão 3

3. No MVC, qual desses frameworks utiliza a camada do *Model*?

a) Hibernate.

b) JSF.

c) JSP.

d) HTML.

e) XHTML.

Questão 4

4. Qual dessas opções representa melhor a camada do *Controller* do MVC?

a) Pode ser considerado o *front-end* da aplicação.

b) É representado tudo o que está no *back-end* da aplicação.

c) Tudo o que controla a persistência.

d) Classe que contém a lógica de negócio que trocará informações com outras camadas.

e) É uma classe que configura o mapeamento de *Servlets* para apresentação.

Questão 5

5. Qual dessas empresas que mais contribui para o JSF?

a) Microsoft.

b) Apache.

c) Facebook.

d) HP.

e) Googl.

Gabarito

1. Resposta: C.

Toda especificação do JPA venho do EJB.

2. Resposta: A.

JSF, assim como o JSP, é a camada que apresenta os dados para o cliente.

3. Resposta: A.

Hibernate persiste os dados, ou seja, a conexão e manipulação dos dados é tratada na camada do Model no MVC que interagem com as outras camadas que são responsáveis pelas regras de negócio e apresentação das informações.

4. Resposta: D.

A camada *Controller* pertence ao MVC e nessa camada é desenvolvida a lógica de negócio, ou seja, as regras que serão feitas dentro do sistema, por exemplo: o que o cliente irá acessar na camada *View* não é o mesmo que o administrador irá acessar, há regras e é nessa camada que deve estabelecê-las.

5. Resposta: B.

A empresa Apache recebeu como doação da SUN Microsystems toda especificação do JSF e é uma das empresas que mais contribui para essa tecnologia.

Unidade 5

Inclusão de Práticas de Validação, Requisição, outras Linguagens e Construção de Projeto (Prático-Conceitual)

Objetivos

1. Práticas de validação.
2. Validadores.
3. Ajax com *JavaServer Pages.*

Introdução

A validação de dados utilizando a camada de apresentação facilita o processo de inserção de dados validados. No caso do framework JSF, os validadores podem ser configurados sem a necessidade de validar com a persistência.

Contudo, essa validação de dados realizada diretamente no cliente traz uma vantagem de não acessar o banco de dados toda vez que for requisitado na aplicação. Portanto, quando há muitos acessos ao servidor, a validação feita do lado do cliente não irá onerar a capacidade de comunicação do cliente e servidor.

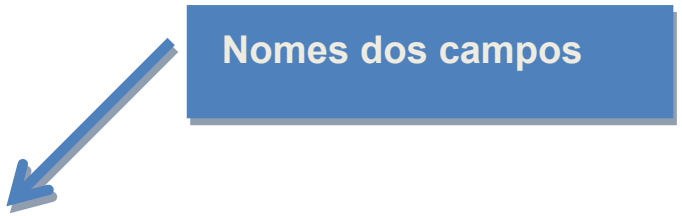
1 Práticas de Validação

Neste tema, você aprenderá como proceder a validação de dados diretamente no *front-end* (visão) de sua aplicação. Há diversas maneiras de validar dados, desde direto na persistência, por exemplo, com Hibernate que utiliza anotações no modelo de suas classes que trazem mensagens ou tratamento na requisição (HIBERNATE, 2016).

É possível elaborar métodos para validar campos ou inserir dados em formulários ou utilizar frameworks para validar dados para segurança, como o *spring security* empregado principalmente para validar *logins* e senha junto com a persistência utilizando Hibernate (LUCKOW, 2010 p.232-236).

Para essa aplicação, crie um projeto Web no Netbeans ou no Eclipse com framework JSF 2.x com nome: **WebValidador**, e na página **index.xhtml** crie um formulário com os seguintes campos: **Nome, Idade e Telefone** com Botão com valor:

```
<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:h="http://xmlns.jcp.org/jsf/html">
   <h:head>
      <title>Facelet Title</title>
   </h:head>
   <h:body>
      <h1>Cadastro de Usuário</h1>
      <hr/>
      <h:form>
            <h:panelGrid columns="2">
               <h:outputLabel value="Nome" for="nome" />
               <h:inputText id="nome" label="Nome" />
               <h:outputLabel value="Idade" for="idade" />
```



```
            <h:inputText id=”idade” label=”idade” />
            <h:outputLabel value=”Telefone” for=”telefone” />
            <h:inputText id=”telefone” label=”telefone”  />
            <h:commandButton value=”Validar” />
        </h:panelGrid>
    </h:form>
  </h:body>
</html>
```

O formulário da **(Figura 22)** gerado pelo código acima com JSF aceita qualquer tipo de caracteres e isso irá gerar erros posteriores de validação em sua base de dados, por exemplo, se um campo do banco de dados for inteiro e o cliente digitar letras, ocorrerá erros.

Figura 22 – Formulário de cadastro com JSF



Fonte: Elaborada pelo autor.

Para validar os campos, o framework do JSF possui recursos de validação desde suas primeiras versões 1.x até as mais atuais. Você atualizará as tags do JSF do código anterior.

No formulário de Cadastro de Usuários, serão implementados validadores para não aceitar números no campo do nome e números negativos ou letras no campo da idade.

Primeiro, colocaremos no campo nome um requerimento (**required=”true”**) e será adicionada a tag **<h:messages>** para enviar a mensagem para o cliente. Caso você não coloque a tag para enviar a mensagem, o cliente não saberá o que está acontecendo e simplesmente não irá cadastrar.

Atualize seu código anterior do formulário de Cadastro de Usuário como o código logo abaixo e teste sua aplicação sem a tag **<h:-messages>**, que não irá apresentar a mensagem e com a tag aparecerá a mensagem como na **(Figura 23).**

```
<html xmlns=”http://www.w3.org/1999/xhtml”
    xmlns:h=”http://xmlns.jcp.org/jsf/html”>
   <h:head>
      <title>Facelet Title</title>
   </h:head>
   <h:body>
      <h1>Cadastro de Usuário</h1>
      <hr/>
      <h:form>
         <h:messages>
            <h:panelGrid columns=”2”>
               <h:outputLabel value=”Nome” for=”nome” />
               <h:inputText id=”nome” label=”Nome” required=”true” />
               <h:outputLabel value=”Idade” for=”idade” />
               <h:inputText id=”idade” label=”idade” />
               <h:outputLabel value=”Telefone” for=”telefone” />
               <h:inputText id=”telefone” label=”telefone”  />
```

Tag <h:messages> para exibir mensagens caso o required for igual a “true”.

Deixar campo de preenchimento obrigatório.

```
            <h:commandButton value=”Validar” />
          </h:panelGrid>
        </h:messages>
      </h:form>
    </h:body>
</html>
```

Figura 23 – Validação do campo nome com mensagem



Fonte: Elaborada pelo autor.

A tag **<h:messages>** possui diversas variações de validação, por exemplo, para mudar a cor da mensagem para vermelho poderá atualizar seu código para:

```
<h:messages errorStyle=”color:red”>.
```

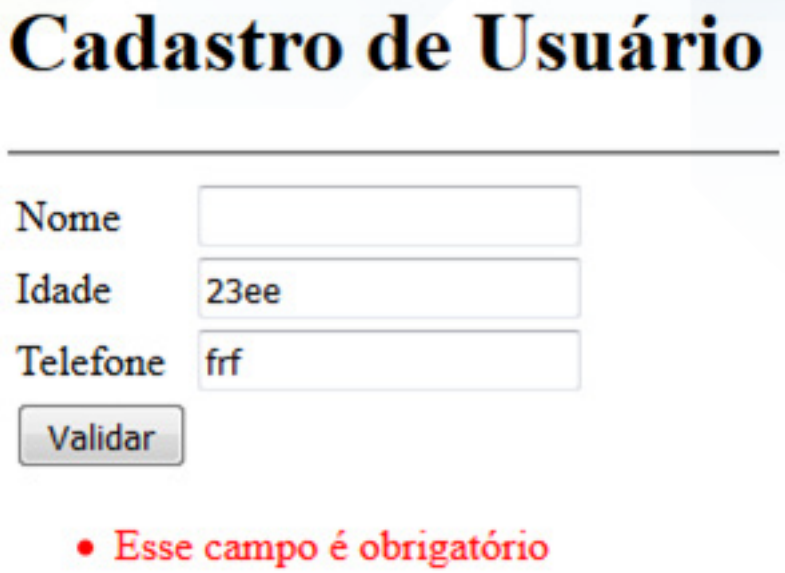
Personaliza a cor da mensagem requerida.

No campo de validação, é possível personalizar a mensagem **(Figura 24)** que será apresentada para o usuário inserindo o **requiredMessage**, como no código a seguir.

```
<h:inputText id=”nome” label=”Nome” required=”true” requiredMessage=”Esse campo é obrigatório” />
```

Propriedade para personalizar a mensagem exibida para o cliente.

Figura 24 – Mensagem personalizada



Fonte: Elaborada pelo autor.

2 Tipos de Validadores

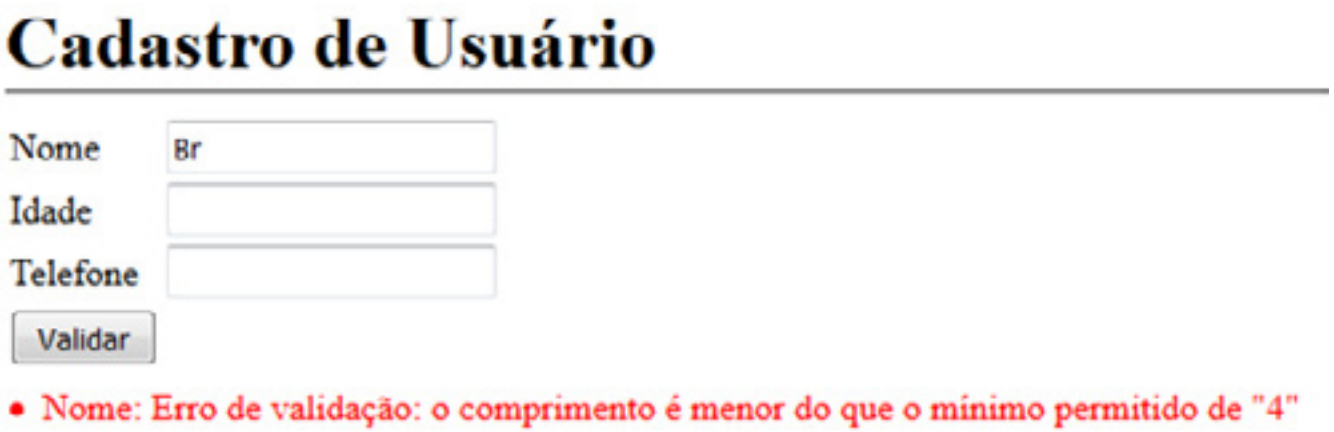
Os validadores de campos são utilizados para diversos propósitos, dentre eles: o tamanho do campo, tipo **double** e para validação de e-mail. Para validar os campos, você deverá atualizar os códigos utilizados nos validadores, como no exemplo a seguir:

```
<h:inputText id="nome" label="Nome" required="true" requiredMessage="Esse campo é obrigatório">
        <f:validateLength minimum="4" maximum="9" />
</h:inputText>
```

Validador de tamanho mínimo e máximo de caracteres.

Na **(Figura 25),** o campo nome é validado e o cadastro não ocorrerá, gerando uma mensagem para o usuário. No caso, exibe o mínimo de caracteres ou o máximo permitido.

Figura 25 – validação de tamanho de caracteres

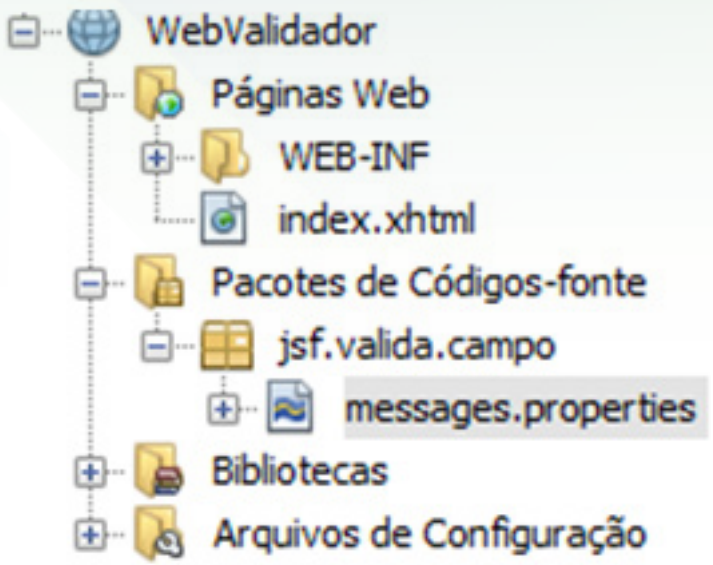


Fonte: Elaborada pelo autor.

No JSF, há outra maneira de validação utilizando um arquivo de propriedades que contém as mensagens que serão apresentadas nessas requisições. Dentro da tag de validação do código anterior, não há o **requiredMessage**, como no **<h:inputText>**.

Portanto, é possível chamar esse arquivo de propriedade para validação. Faça um pacote com nome **jsf.valida.campo** e adicione um arquivo de propriedades chamado **messages**, como na **Figura 26.**

Figura 26 – Arquivo de propriedade de validação



Fonte: Elaborada pelo autor.

Dentro do arquivo, coloque as validações requisitadas para exibir as mensagens de acordo com a condição. As mensagens que foram usadas até o momento devem colocar dentro do arquivo **messages.properties**:

javax.faces.component.UIInput.REQUIRED=''{0})'' campo obrigatório.

javax.faces.validator.LengthValidator.MINIM=Valor menor do que o mínimo permitido ''{0}''

javax.faces.validator.LengthValidator.MINIM=Valor maior do que o máximo permitido ''{0}''

Para o validador funcionar com o arquivo criado (**messages.properties**), você precisará modificar o arquivo de configuração **faces-config.xml** (ORACLE, 2016). Caso não houver o arquivo de configuração em seu projeto, insira um novo. Seu arquivo de configuração deverá ter os seguintes códigos:

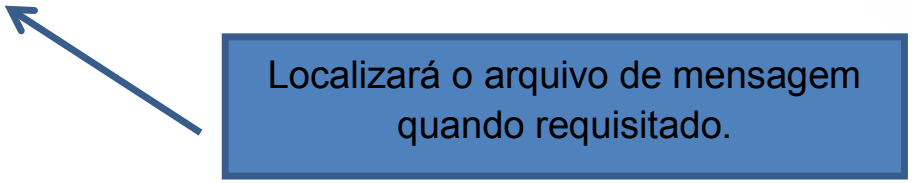
Para saber mais

É possível criar um dicionário de dados utilizando messages.properties com esquema de internacionalização de código. Disponível em: <https://docs.oracle.com/javase/tutorial/i18n/intro/steps.html>. Acesso em: 20 ago. 2016.

```
<?xml version='1.0' encoding='UTF-8'?>
<faces-config version="2.2"
        xmlns="http://xmlns.jcp.org/xml/ns/javaee"
```

```
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee http://xmlns.jcp.org/xml/ns/
javaee/web-facesconfig_2_2.xsd">

   <application>
     <message-bundle>jsf.valida.campo.messages</message-bundle>
   </application>
</faces-config>
```



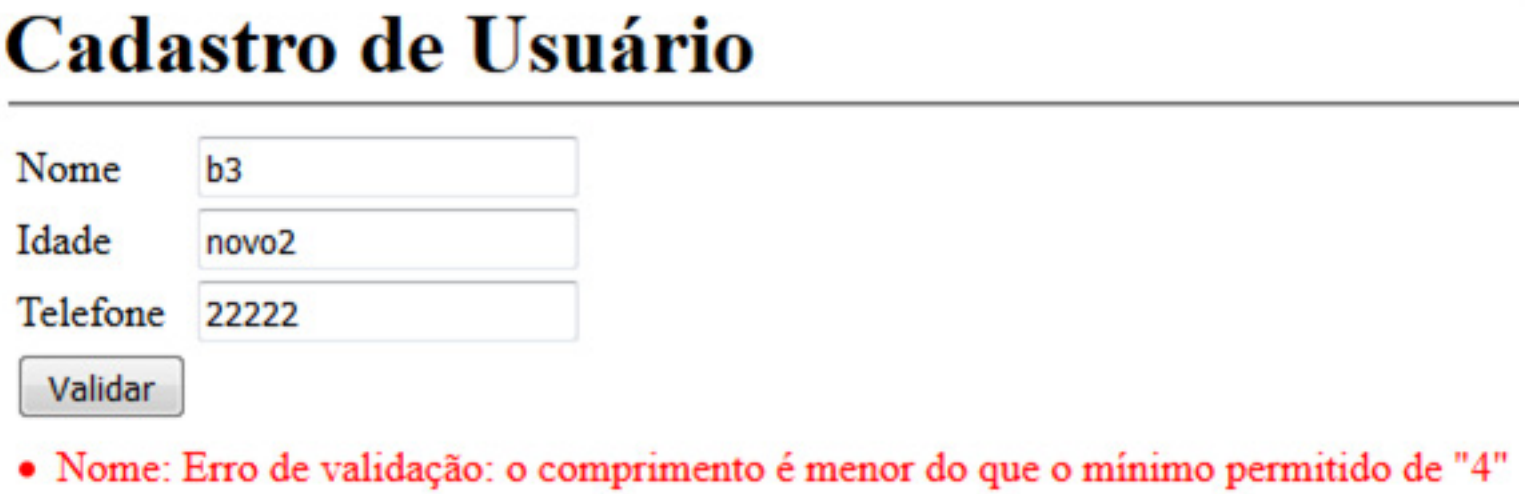
Você irá notar que não será preciso utilizar mais o: **requiredMessage="Esse campo é obrigatório"** dentro de seu código da página **index.xhtml**. Confira o código a seguir e veja o resultado, como apresentado na **Figura 27**.

```
<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:h="http://xmlns.jcp.org/jsf/html"
    xmlns:f="http://xmlns.jcp.org/jsf/core">
   <h:head>
```

```
    <title>Facelet Title</title>
</h:head>
<h:body>
    <h1>Cadastro de Usuário</h1>
    <hr/>
    <h:form>
        <h:messages errorStyle="color:red">
            <h:panelGrid columns="2">
                <h:outputLabel value="Nome" for="nome" />
                <h:inputText id="nome" label="Nome" required="true" >
                    <f:validateLength minimum="4" maximum="9" />
                </h:inputText>
                <h:outputLabel value="Idade" for="idade" />
                <h:inputText id="idade" label="idade" />
                <h:outputLabel value="Telefone" for="telefone" />
                <h:inputText id="telefone" label="telefone"  />
                <h:commandButton value="Validar" />
```

```
            </h:panelGrid>
        </h:messages>
    </h:form>
  </h:body>
</html>
```

Figura 27 – Validado com arquivo messages.properties



Fonte: Elaborada pelo autor.

O arquivo **messages.properties** trará o nome do campo que está sendo validado antes da mensagem, como identificado na **Figura 27**. Nesse arquivo, você poderá adicionar todo tipo de validação para sua aplicação.

Há muitos tipos de validações que poderão ser adicionadas no documento e isso dependerá do projeto que você estará desenvolvendo. Além de validações, é possível utilizar outros recursos e linguagens para otimizar sua aplicação. Uma das tecnologias mais utilizadas é conhecida como **Ajax,** que poderá ser um recurso em sua aplicação que reduzirá o acesso constante ao servidor.

Para saber mais

Ajax não era uma tecnologia, e sim um termo utilizado para descrever diversas tecnologias. Disponível em: <https://developer.mozilla.org/pt-BR/docs/AJAX>. Acesso em: 20 ago. 2016.

3 AJAX COM JAVASERVER PAGES

A utilização reduz a requisição de informações que acessam o servidor tornando mais eficiente, não onerando o processo de transação na comunicação. **Ajax** é o acrônimo para assíncrono JavaScript e XML uma tecnologia altamente responsiva para aplicações *Web*. Na plataforma do JEE, há suporte para utilização com JSF.

Link

Documentação sobre a utilização de Ajax. Disponível em: <http://www.w3schools.com/ajax/>. Acesso em: 20 ago. 2016.

A tecnologia do Ajax não se limita apenas ao JavaScript e XML, podendo ser utilizados outros tipos nesse conjunto. Em uma requisição do cliente, o servidor manda respostas de atualização para o ***Document Object Model*** **(DOM)** em formato XML ou em formato JSON.

Link

Documentação referente ao uso de DOM. Disponível em: <https://www.w3.org/DOM/>. Acesso em: 20 ago. 2016.

É possível utilizar o Ajax como um recurso embutido (*built-in*) com biblioteca de JavaScript, que faz parte das bibliotecas nativas do JSF. Pode-se utilizar os códigos de Ajax em JSF em aplicações Web da seguinte maneira:

- Usando a tag **f:ajax** como um componente do **Facelets** sem precisar configurar ou adicionar algum componente;
- utilizar API do JavaScript e o método **jsf.ajax.request()** diretamente na aplicação com possibilidade de customização de controle.

Para saber mais

Facelets é uma linguagem de descrição de página utilizado como *template* Web e utilizada junto com JSP e que se tornou um dos padrões do JEE. Disponível em: <http://www.devmedia.com.br/facelets-visao-geral-conceitos-utilizacao-e-criacao-de-componentes/14068>. Acesso em: 20 ago. 2016.

A utilização do Ajax dentro do JSF de forma *Built-in* torna o uso de suas tags uma forma fácil de implementação desse recurso, por exemplo, utilizando a tag **f:ajax** dentro do contexto do JSF:

```
<h:inputText value="#{bean.message}">
   <f:ajax />
</h:inputText>
```

Para cada aplicação de requisição, é possível utilizar o Ajax trazendo vantagens na performance de sua aplicação *Web*. Há uma grande variação nos recursos do Ajax dentro do JSF. O **tutorial de Ajax** aborda nas especificações do JEE uma referência completa para esse recurso.

Link

Referência na utilização de Ajax dentro do JSF. Disponível em: <http://docs.oracle.com/javaee/6/tutorial/doc/gkace.html>. Acesso em: 10 ago. 2016.

Portanto, é possível utilizar Ajax em sua aplicação para reduzir a troca de informação com servidor, a fim de melhorar a comunicação, diminuindo o acesso ao servidor e tendo uma melhor resposta para o cliente.

Glossário

DOM: *Document Object Model.*

JSON: *JavaScript Object Notation.*

AJAX: Asynchronous Javascript and XML.

Questão para reflexão

Por que utilizar Ajax em suas aplicações e quais seriam as vantagens?



Considerações Finais

- Validadores;
- JSF e recursos de componentes;
- Ajax em aplicações JSF;
- integração de tecnologias Web.

Referências

AJAX. Disponível em: <https://developer.mozilla.org/pt-BR/docs/AJAX>. Acesso em: 10 ago. 2016.

AJAX_2. Disponível em: <http://www.w3schools.com/ajax/>. Acesso em: 10 ago. 2016.

DEVMEDIA. Faceletes. Disponível em: <http://www.devmedia.com.br/facelets-visao-geral-conceitos-utilizacao-e-criacao-de-componentes/14068>. Acesso em: 10 ago. 2016.

DOM. Disponível em: <https://www.w3.org/DOM/>. Acesso em: 10 ago. 2016.

HIBERNATE. Disponível em:<http://hibernate.org/>>. Acesso em: 10 ago. 2016.

LUCKOW, Décio Heinzelmann; DE MELO, Alexandre Altair. **Programação Java para a WEB**. São Paulo: Novatec, 2010.

ORACLE. Persistência. Disponível em: <https://docs.oracle.com/javase/tutorial/i18n/intro/steps.html>. Acesso em: 30 jul. 2016.

ORACLE _2. Tutorial JEE. Disponível em: <http://docs.oracle.com/javaee/6/tutorial/doc/gkace.html>. Acesso em: 2 ago. 2016.

Assista a suas aulas



Aula 5 - Tema: Inclusão de práticas de validação, requisição, outras linguagens e construção de projeto. Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f-1d/6cc8ab9c4853d8e5203e596e8920361b>.



Aula 5 - Tema: Inclusão de práticas de validação, requisição, outras linguagens e construção de projeto. Bloco II

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/c23ec7ed6540c5d8bc71b5d36c897a2e>.

Questão 1

1. Qual dessas alternativas requisita ao cliente o preenchimento do campo obrigatório?

a) <h:inputText id=”nome” label=”Nome” required=”true” [...]

b) <h:inputText id=”nome” label=”Nome” required=”false” [...]

c) <h:inputText id=”nome” label=”Nome” [...]

d) <h:inputText id=”nome” label=”Nome” quired=”true” [...]

e) <h:inputText id=”nome” label=”Nome” require=”true” [...]

Questão 2

2. É possível ler um arquivo messages.properties para validação. Em qual documento é declarado para ser lido na sua aplicação?

a) index.jsp.

b) web.xml

c) faces.xml

d) config.xml

e) faces-config.xml

Questão 3

3. Em qual dessas tags é declarado o arquivo messages.properties?

a) <message>jsf.valida.campo.messages</message>

b) <message-bundle>jsf.valida.campo.messages</message-bundle>

c) <bundle>jsf.valida.campo.messages</bundle>

d) < application>jsf.valida.campo.messages</application>

e) < faces-config>jsf.valida.campo.messages</faces-config>

Questão 4

4. Qual dessas alternativas é correta no uso do Ajax com JSF?

a) É possível utilizar o Ajax como um recurso embutido (built-in) com bibliotecas de JSON que faz parte do JSF.

b) É possível utilizar o Ajax como um recurso embutido do framework com bibliotecas de JavaScript que faz parte das biblioteca nativas do JSP.

c) É possível utilizar o Ajax como um recurso importado com bibliotecas de JavaScript que faz parte do JSF.

d) É possível utilizar o Ajax como um recurso embutido (*built-in*) com bibliotecas de JavaScript que faz parte das biblioteca nativas do JSF.

e) É possível utilizar o Ajax como um recurso inserido com bibliotecas que faz parte do JSF.

Questão 5

5. Para utilizar Ajax com JSF, qual desses procedimentos de tag é correto?

a) tag f:ajax como um componente do Facelets.

b) tag ajax como um componente do Facelets.

c) tag e:ajax como um componente do Facelets.

d) tag h:ajax como um componente do Facelets.

e) tag c:ajax como um componente do Facelets.

Gabarito

1. Resposta: A.

O required deve ser igual a '*true*', que significa que o campo é requerido.

2. Resposta: E.

O nome correto de configuração para JSF estabelecido pelas normas regulamentadas pela ORACLE como nome padrão do arquivo utilizado na elaboração dos padrões, os demais não estão completos ou não se referem ao JSF.

3. Resposta: B.

O nome da tag deve estar declarada dentro do <message-bundle> por padrão.

4. Resposta: D.

O Ajax se encontra embutido no JSF.

5. Resposta: A.

A tag utilizada é a f:ajax no JSF.

Unidade 6

JSF com PrimeFaces

Objetivos

JSF é uma tecnologia que incorpora características do padrão MVC, que separa a aplicação em três camadas: modelo, visualização e controle. Neste tema, serão abordados os tópicos referentes a:

1. PrimeFaces
2. MVC

Introdução

O JavaServer Faces (JSF), como já mencionado e utilizado em outros módulos, é um framework com diversos recursos, que faz parte da especificação do JEE (JSR-314) e utiliza padrões (JCP, 2016).

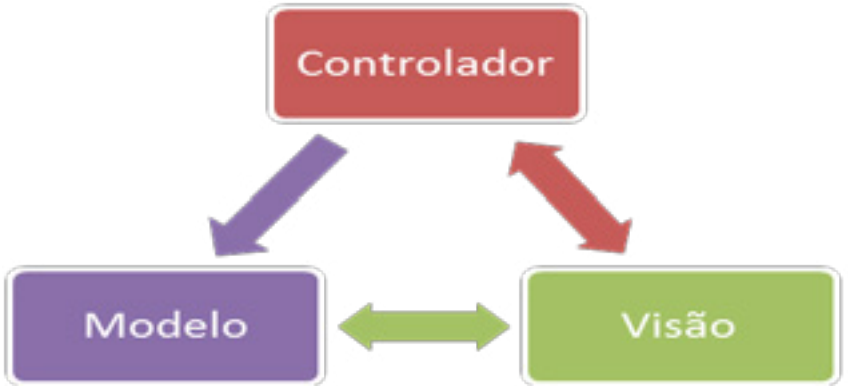
Essa tecnologia Web possui componentes visuais pré-definidos para facilitar o desenvolvimento em suas aplicações, e, dentre esses componentes pré-prontos, destacam-se: calendários, combo, caixa de textos e tabelas (CAELUM, 2016).

1 MVC

Os componentes do JSF possuem características de *Stateful*, ou seja, guardam o estado dos componentes das páginas automaticamente, evitando erros de navegação do cliente, principalmente quando o usuário tenta retornar para a sessão anterior (CORDEIRO, 2014 p.16).

Portanto, o JSF possui a característica de um framework MVC dentro das especificações do Java *Community Process* (JCP) e possui validadores e inclusão *built-in* do Ajax **(Figura 28)**.

Figura 28 – Representação da iteração dos objetos no MVC



Fonte: Elaborada pelo autor.

A arquitetura do JSF possui algumas especificações dentro dos padrões de desenvolvimento em camadas que separa a camada de negócios, dados e visão na aplicação. A ORACLE define a arquitetura aplicada ao JSF e o padrão conhecido como MVC (*Model, View e Controller*):

> [...] Uma das principais vantagens do JSF é que ele é ao mesmo tempo um padrão de interface de usuário Java Web, bem como um quadro que segue firmemente o padrão de design *Model-View-Controller* (MVC). Isso faz com que aplicativos JSF sejam muito mais gerenciáveis, porque o código de interface do usuário (*View*) é claramente separado dos dados e lógica (modelo). Para preparar o contexto JSF, que fornece acesso a dados de aplicação para as páginas, e para se proteger contra o acesso não autorizado ou impróprio das páginas, todas as interações do usuário com o aplicativo são manipuladas por um *front-end* "Faces" servlet (*Controller*) [...] (ORACLE, 2016).

A Microsoft faz referência ao padrão MVC original da comunidade *Smalltalk* em um de seus sites e descreve o padrão (MSDN, 2016):

Modelo: o modelo administra o comportamento e os dados do domínio da aplicação e responde aos pedidos de informação sobre o seu estado (geralmente a partir do ponto de vista) e às instruções para alterar o estado (geralmente a partir do controlador).

Visão: a visão gere a exibição de informações.

Controlador: o controlador interpreta as entradas de *mouse* e teclado do usuário, informando o modelo ou a visão para mudar conforme o caso.

O modelo MVC é utilizado em qualquer tipo de projeto em camadas, trazendo a vantagem de ser mais independente. Por exemplo: se sua aplicação for desenvolvida no padrão MVC, poderá trocar sua base de dados no MySQL para uma base no ORACLE ou DB2 da IBM.

Há muitas aplicações que começam com uma tecnologia e, por razões políticas ou por necessidade, o sistema gerenciador de banco de dados deve ser substituído, sendo que segue o mesmo princípio para outras camadas que podem sofrer mudanças sem atingir toda arquitetura.

O JSF foi desenvolvido sob os padrões da engenharia de software e se aplica ao modelo MVC. Para entender como o JSF se encaixa dentro do MVC, uma representação no **Quadro 3** pode nos auxiliar.

Quadro 3 – Modelo MVC no JSF

Modelo	Visão	Controlador
• **entidades e regras de negócio.**	• Componentes UI em páginas JSP/XHTML	• Faces Servlet (Front Controller)
• **Objetos gerais da aplicação, persistência.**	• Renderizadores (HTML, XML, bootstrap, jquery).	• Backing Bean (Page Controller ou Modelo)

Fonte: Adaptado de Casaopen (2016).

2 PRIMEFACES

É um conjunto de componentes desenvolvido com as especiações do JSF e utiliza de forma nativa o Ajax. Para usar o framework do **PrimeFaces**, você precisa ter instalado no mínimo o Java 5 e o JSF 1.2 (PALIARI, 2016). Entretanto, é recomendado instalar as últimas versões do framework disponíveis no site oficial.

Link

Site oficial do framework PrimeFaces. Disponível em: <http://www.primefaces.org/>. Acesso em: 10 ago. 2016.

O PrimeFaces não é o único framework que possui a mesma linha de componentes que trabalham com JSF. Há outras empresas que disponibilizam tecnologias semelhantes, dentre elas: **RichFaces** (JBoss) e ICEFaces(I-CEsoft).

Para saber mais

O projeto RichFaces é mantido pela JBoss que possui um dos servidores Web de aplicação mais conhecido que é mantida pela empresa Red Hat. Disponível em: <http://richfaces.jboss.org/>. Acesso em: 10 ago. 2016.

Destaque para o framework **ICEFaces**, que possui ferramentas visuais em uma palette para arrastar e soltar (*Drag and drop*) semelhantes ao antigo **Visual JSF** (ICEFACES, 2016).

Link

Projeto oficial do framework ICEFaces. Disponível em: <http://www.icesoft.org/java/projects/ICEfaces/overview.jsf>. Acesso em: 10 ago. 2016.

O framework PrimeFaces é mantido pela Prime Teknoloji da Turquia e assegura um desenvolvimento de forma leve e simples para o usuário (PRIMEFACES, 2016). Além disso, a biblioteca foi projetada para resolução de problemas de forma mais rápida.

O framework possui mais de cem componentes baseados em **Ajax**, *add-ons* para mobile em aplicações Web com aparência nativa e fácil integração com suporte para o Eclipse e o Netbeans (ORACLE_2, 2016).

Para saber mais

O JSF pode ser agregado a diferentes tipos de frameworks, dentre eles PrimeFaces, que possui uma coleção de componentes baseados em Ajax facilitando o uso no desenvolvimento.

Link

Projetos Mobile e templates em PrimeFaces. Disponível em: <http://www.primefaces.org/showcase/mobile/index.xhtml>. Acesso em: 10 ago. 2016.

O PrimeFaces pode ser adquirido diretamente do site e incluído no projeto como uma biblioteca. O framework apresenta a possibilidade de trocar o visual da aplicação e seus componentes podem ser utilizados em projetos **Mobile**.

A integração do PrimeFaces com o JSF pode trazer vantagens para o desenvolvedor ao utilizar componentes prontos, por exemplo: popular combos e preencher tabelas com dados.

Portanto, o conhecimento de novas tecnologias e o domínio do uso de padrões da engenharia de software, como o **MVC** poderá trazer benefícios na qualidade no desenvolvimento com JSF.

Glossário

MVC: *Document Object Model*.

SERVLET: “servidorzinho”.

FRONT-END: camada de visão do MVC.

Questão para reflexão

Por que utilizar diferentes frameworks com JSF e quais seriam as diferenças de utilizar na camada *View* forma e componentes baseados em Ajax e de aparência rica?



Considerações Finais

- MVC
- PrimeFaces
- Frameworks
- *Design Pattern*

Referências

CAELUM. Apostila de Java. Disponível em: <https://www.caelum.com.br/apostila-java-testes-js-f-web-services-design-patterns/introducao-ao-jsf-e-primefaces/#7-2-caracteristicas-do-jsf>. Acesso em: 30 ago. 2016.

CASAOPEN. JSF. Disponível em: <http://www.casaopen.com.br/2012/12/jsf-mvc.html>. Acesso em: 30 ago. 2016.

_____. Modelo MVC. Disponível em:<http://www.casaopen.com.br/2012/12/jsf-mvc.html>. Acesso em ago. 2016.

CORDEIRO, Gilliard. **Aplicações Java para a web com JSF e JPA**. São Paulo: Casa do Código, 2014.

ICEFACES. Componentes. Disponível em:<http://component-showcase.icesoft.org/component--showcase/showcase.iface>. Acesso em ago. 2016.

JCP. Java Specification Requests. Disponível em: <https://jcp.org/en/jsr/detail?id=314>. Acesso em: 30 ago. 2016.

MSDN. Padrão MVC. Disponível em: <https://msdn.microsoft.com/en-us/library/ff649643.aspx>. Acesso em: 10 ago. 2016.

ORACLE. Introdução ao JavaServer Faces. Disponível em: <http://www.oracle.com/technetwork/topics/index-090910.html>. Acesso em: 30 ago. 2016.

ORACLE_2. PrimeFaces. Disponível em: <http://www.oracle.com/technetwork/developer-tools/netbeans/community/customer-case-study-primefaces-1912066.pdf>. Acesso em: 30 ago. 2016.

PALIARI, Marcos. O que é PrimeFaces. Disponível em: <http://www.matera.com/br/2011/09/30/matera-utiliza-o-primefaces/>. Acesso em: 30 ago. 2016.

PRIMEFACES. Partners. Disponível em: <http://www.primefaces.org/partners>. Acesso em: 30 ago. 2016.

Assista a suas aulas



Aula 6 - Tema: JSF com Primefaces. Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/8681c9095736488a321e09c47e4ff5f2>.



Aula 6 - Tema: JSF com Primefaces. Bloco II

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/530495787d42eb573dd187bcb14c0961>.

Questão 1

1. Qual dessas alternativas possui a descrição da sigla MVC?

a) *Model View Controller.*

b) *Model Viewed Controlled.*

c) *Modeled View Controller.*

d) *Model View Control.*

e) *Model View Controll.*

Questão 2

2. Qual dessas empresas é responsável pelo ICEFaces?

a) Apache.

b) JBoss.

c) PrimeFaces.

d) RichFaces.

e) ICEsoft.

Questão 3

3. O padrão MVC tem origem de qual projeto?

a) Smalltalk.

b) JAVA.

c) Visual Studio.

d) NET.

e) UML.

Questão 4

4. Como é considerado o MVC dentro da engenharia de software?

a) O MVC é considerado um modelo de projeto (*Design Model*).

b) O MVC é considerado um padrão de projeto (*Design Model*).

c) O MVC é considerado um padrão de projeto (*Design Pattern*).

d) O MVC é considerado um projeto (*Design Model*).

e) O MVC é considerado um modelo de software (*Design Model*).

Questão 5

5. Qual framework pelo qual a empresa Prime Teknoloji da Turquia é responsável?

a) RichFaces.

b) PrimeFaces.

c) ICEFaces.

d) JavaServer Faces.

e) JavaServer Pages.

Gabarito

1. Resposta: A.

Essa é a descrição correta de cada sigla.

2. Resposta: E.

A empresa responsável é a ICEsoft.

3. Resposta: A.

O MVC começou com o projeto Smalltalk antes do JAVA.

4. Resposta: C.

É um padrão de projeto segundo a engenharia de software.

5. Resposta: B.

O PrimeFaces é de uma empresa da Turquia.

Unidade 7

Temas do PrimeFaces, apresentação e utilização

Objetivos

1. Temas
2. PrimeFaces
3. Configuração

Introdução

O framework PrimeFaces possui a característica de adicionar temas para trabalho em projetos Web; é um complemento que deixa o visual de sua aplicação moderno e com satisfatória aparência para o cliente.

No site oficial, é possível adquirir temas gratuitos para utilizar em sua aplicação. Há diversos temas a serem escolhidos no arcabouço de conteúdos do PrimeFaces. De acordo com a aplicação que irá desenvolver, poder-se-á selecionar a que se melhor encaixa com suas perspectivas.

Os temas do PrimeFaces possuem cores e efeitos que podem ser adicionados, trocados ou removidos de sua aplicação com facilidade utilizando o arquivo em **xml**. Assim, seu projeto vislumbra uma apresentável camada de visão e desenvolvimento mais produtível.

1 TEMAS DO PRIMEFACES

O framework PrimeFaces possui característica de adicionar temas para trabalho em projetos Web. No site oficial, é possível adquirir temas gratuitos para utilizar em sua aplicação.

Além disso, a utilização de temas com PrimeFaces é algo simples de implementar, como será mostrado adiante. No site oficial, são oferecidos outros temas pagos na categoria *premium*.

O PrimeFaces integra-se ao **ThemeRoller CSS framework**, que disponibiliza outros temas para utilizar em sua aplicação *Web*. No site do jQuery, poderá escolher diversas alternativas para utilizar em sua aplicação.

Link

Para saber mais sobre o ThemeRoller, acesse o site a seguir. Disponível em: <http://jqueryui.com/themeroller/>. Acesso em: 20 ago. 2016.

Além desses recursos, o PrimeFaces também se integra ao framework **Bootstarp**, que possui muitos temas gratuitos e é fácil de utilizar em projetos Web.

Para saber mais

O framework bootstrap pode ser utilizado em qualquer projeto Web. Disponível em: <http://getbootstrap.com.br/>. Acesso em: 20 ago. 2016.

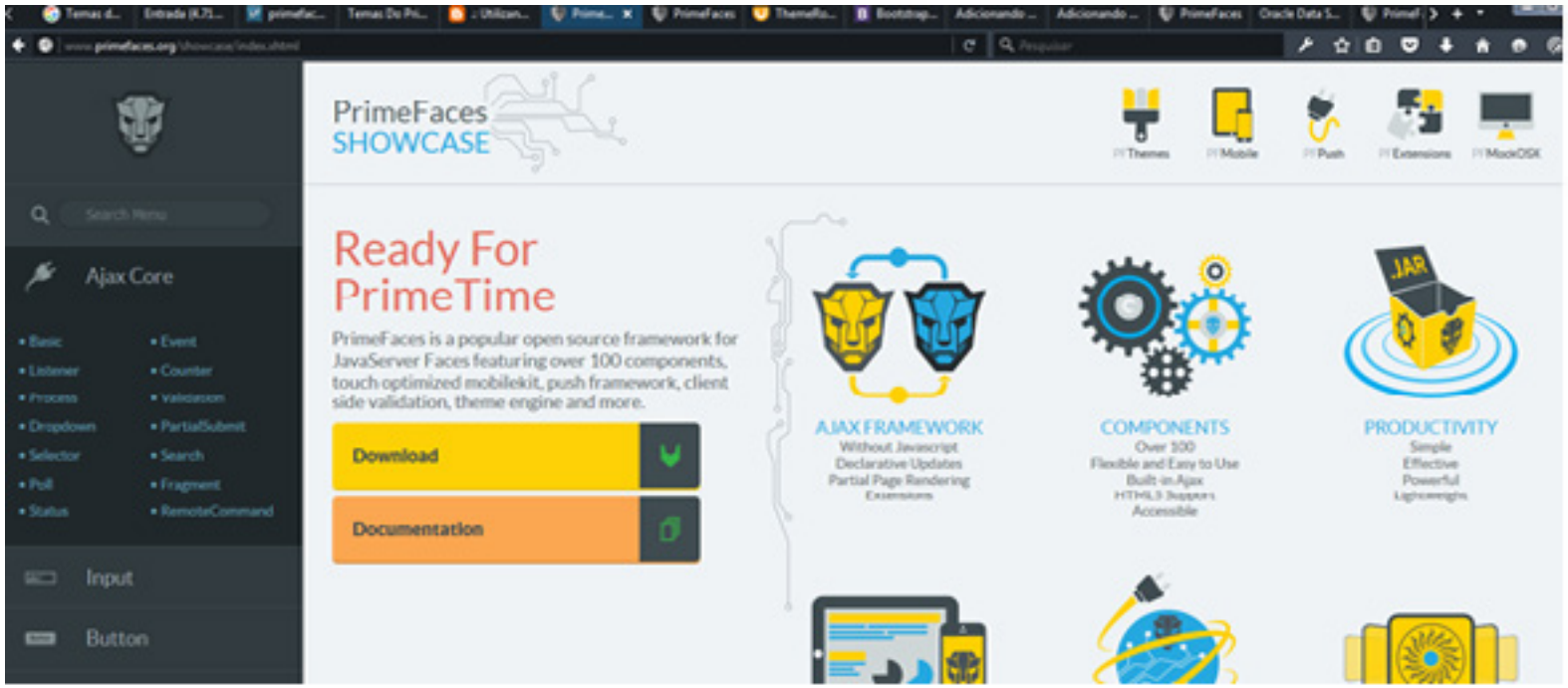
Para um maior conhecimento desse recurso do Primefaces, você irá conhecer cinco temas que poderão ser utilizados em sua aplicação. Esses temas podem ser desde o *layout* de sua aplicação até componentes, como calendários disponibilizados sob a licença Apache.

O site do PrimeFaces possui um **showcase** para você fazer download e utilizar os componentes **(Figura 29)**. Para escolher, entre no site, clique sobre os componentes e verá o código e o componente.

Link

Acesse o showcase. Disponível em: <http://www.primefaces.org/showcase/index.xhtml/>>. Acesso em: 20 ago. 2016.

Figura 29 – PrimeFaces showcase



Fonte: PrimeFaces (2016).

Após inserir o arquivo .jar do PrimeFaces em seu projeto, você precisa declarar no escopo do cabeçalho da página JSF os componentes do framework, como ilustra o código a seguir:

```
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="http://xmlns.jcp.org/jsf/html"
      xmlns:p="http://primefaces.org/ui">
```

Letra 'p' que representará os componentes do PrimeFaces

Com o framework do PrimeFaces dentro de seu projeto, é possível utilizar em suas páginas do JSF diversos temas em sua página xhtml do JSF que estão disponíveis dentro do **repositório oficial** na Web para fazer download.

Link

O repositório oficial do PrimeFaces pode ser acessado no link a seguir. Disponível em: <http://repository.primefaces.org/org/primefaces/themes//>. Acesso em: 20 ago. 2016.

Você verá cinco temas diferentes aos temas (*skins*) disponíveis para utilizar em sua aplicação: **black-tie**, **bluesky, bootstrap, casablanca** e **dark-hive**. Aplicando em seu projeto e ao carregar os temas, verá essa aparência:

1º Tema: black-tie **(Figura 30):**

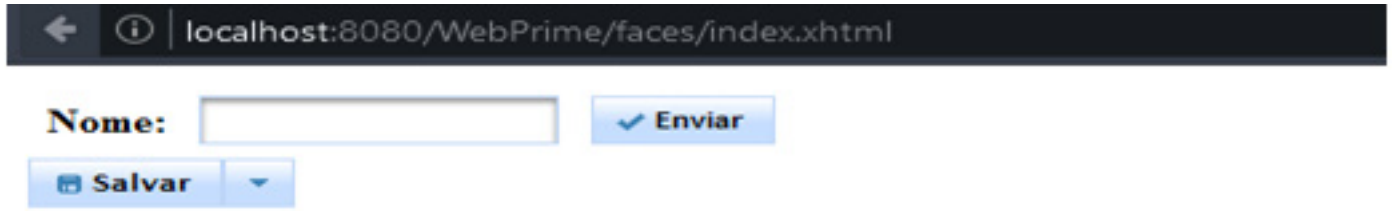
Figura 30 – Black-tie tema do PrimeFac



Fonte: Elaborada pelo autor.

2º Tema: bluesky **(Figura 31):**

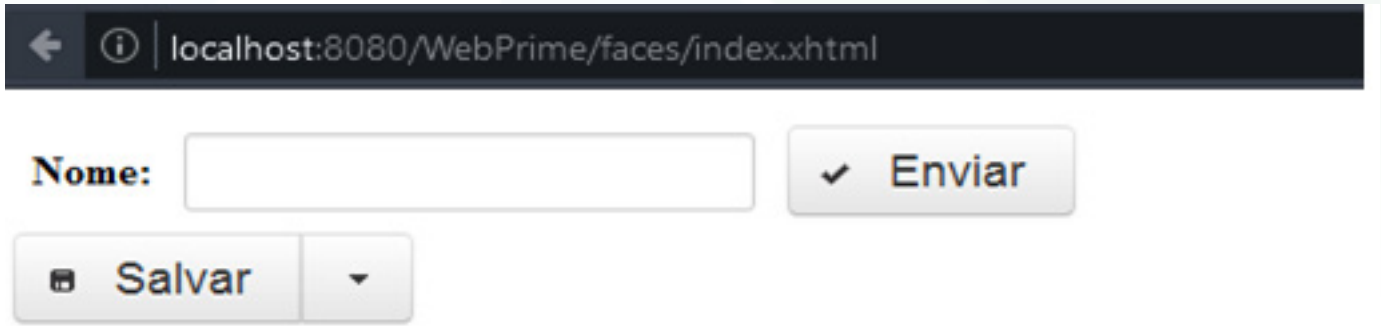
Figura 31 – Bluesky tema do PrimeFaces



Fonte: Elaborada pelo autor.

3º Tema: bootstrap **(figura 32):**

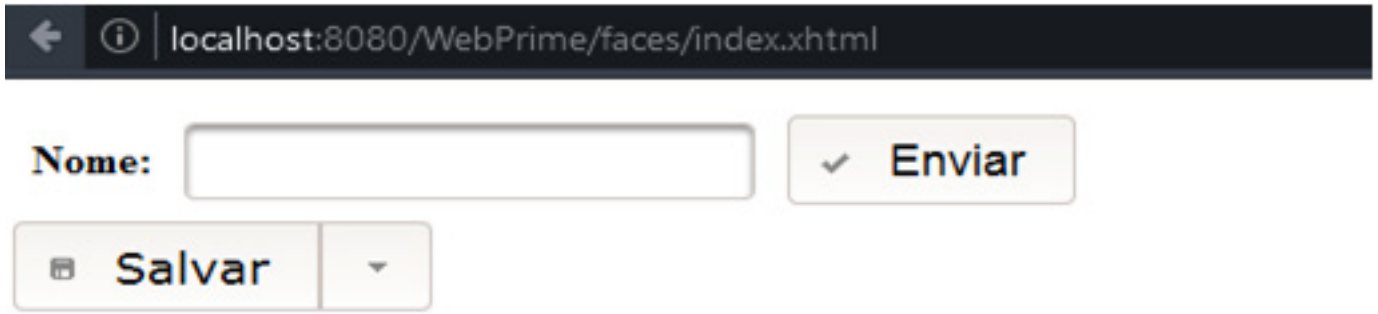
Figura 32 – Bootstrap tema do PrimeFaces



Fonte: Elaborada pelo autor.

4º Tema: casablanca **(Figura 33):**

Figura 33 – Casablanca tema do PrimeFaces



Fonte: Elaborada pelo autor.

5º Tema: dark-hive **(Figura 34):**

Figura 34 – Dark-hive tema do PrimeFaces



Fonte: Elaborada pelo autor.

Você observou uma grande diferença dependendo do tema que escolher para sua aplicação. Teste alguns **temas** para saber quais são os mais adequados para utilizar em algum projeto que tem em mente ou mesmo para praticar.

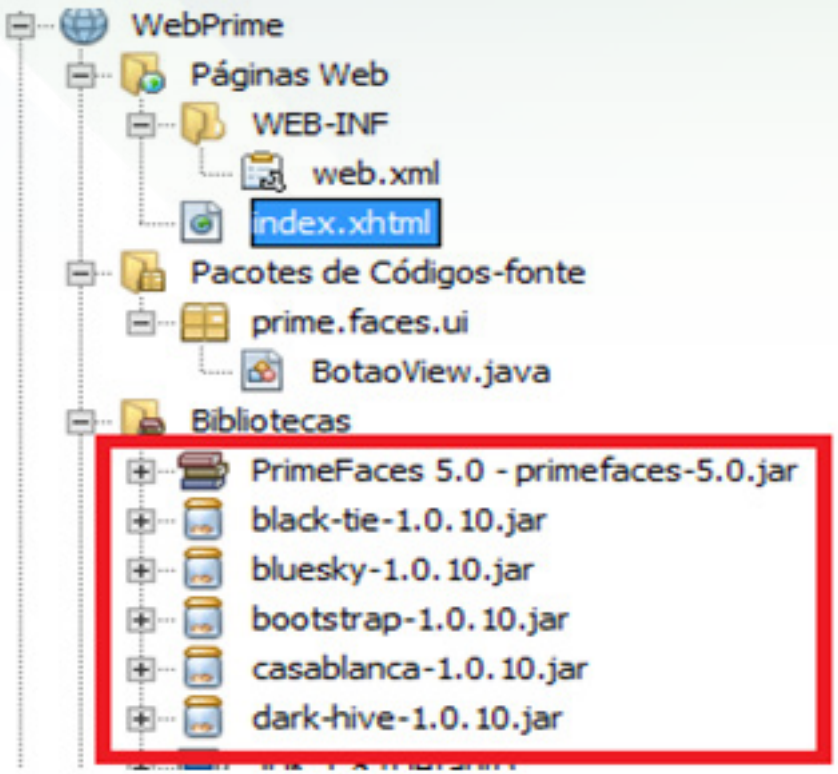
Para saber mais

Além dos temas disponíveis no repositório do PrimeFaces para utilizar em projetos, há outros modelos pagos. Disponível em: <http://www.primefaces.org/themes//>. Acesso em: 20 ago. 2016.

2 Utilizando temas em sua aplicação

Utilizar os temas em sua aplicação com JSF + PrimeFaces é uma tarefa relativamente fácil. Ao escolher o tema no repositório oficial do PrimeFaces, você precisa importar o arquivo jar para dentro de seu projeto como na **(Figura 35),** que ilustra os arquivos com os temas apresentados. Note que o framework do JSF está no mesmo local:

Figura 35 – Framework do JSF no projeto Netbeans



Fonte: Elaborada pelo autor.

Ao importar um ou mais **temas**, você pode mudar a aparência de sua aplicação assim

que desejar. Para configurar o tema a ser utilizado, abra o arquivo web.xml de sua aplicação e adicione as seguintes *tags* (<context-param></context-param>), como o código a seguir:

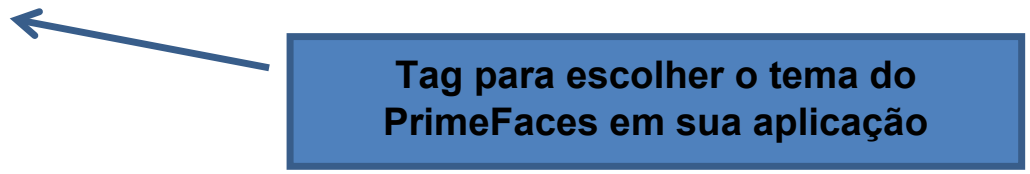
Para saber mais

É possível importar diversos temas em seu projeto e modificar sempre que precisar. Os temas são arquivos jars de fácil manipulação que trabalham com o JSF.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="3.1" xmlns="http://xmlns.jcp.org/xml/ns/javaee" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd">
  <context-param>
    <param-name>javax.faces.PROJECT_STAGE</param-name>
    <param-value>Development</param-value>
  </context-param>
  <servlet>
    <servlet-name>Faces Servlet</servlet-name>
```

```
    <servlet-class>javax.faces.webapp.FacesServlet</servlet-class>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>Faces Servlet</servlet-name>
    <url-pattern>/faces/*</url-pattern>
  </servlet-mapping>
  <session-config><session-timeout>30</session-timeout>
  </session-config>
  <welcome-file-list><welcome-file>faces/index.xhtml</welcome-file>
  </welcome-file-list>
  <context-param>
  <param-name>primefaces.THEME</param-name>
  <param-value>dark-hive </param-value>
  </context-param>

</web-app>
```



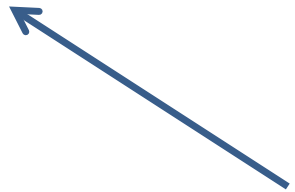
Observe no código anterior que o nome do tema que você escolheu deve ficar entre a tag (<param-value>seu tema<param-value>), e o nome a ser colocado é o mesmo do arquivo jar.

Depois de configurar o arquivo XML e escolher o tema de sua aplicação, adicione alguns componentes do JSF para testar a sua aplicação. No código a seguir, há uma caixa de texto e um botão dentro do corpo da página **index.xhtml** para verificar a mudança do tema.

```
<?xml version='1.0' encoding='UTF-8' ?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:h="http://xmlns.jcp.org/jsf/html"
    xmlns:p="http://primefaces.org/ui">
   <h:head>
      <title>Meu Tema</title>
   </h:head>
   <h:body>
    <h:form>
```



```
        <h:panelGrid columns="4" cellpadding="5">
            <h:outputLabel for="name" value="Nome:" style="font-weight:bold" />
            <p:inputText id="name" value="#{basicView.text}" />
            <p:commandButton value="Enviar" update="display" icon="ui-icon-check" />
            <h:outputText id="display" value="#{basicView.text}" />
        </h:panelGrid>
    </h:form>
  </h:body>
</html>
```



Componentes do PrimeFaces. A tag inicia com 'p' e em seguida o componente escolhido.

Ao carregar sua aplicação, você verá um resultado semelhante à Figura 22 com o tema **dark-hive**, que está declarado no arquivo XML. Escolha um tema e veja os resultados.

PrimeFaces é um framework moderno que apresenta facilidade no uso de aplicações Web. Há diversas possibilidades no uso do JSF + PrimeFaces, dentre elas: aplicações Web e Mobile.

É recomendado explorar o framework do PrimeFaces e seus componentes que possibilitam trazer uma qualidade no front-end de sua aplicação. Além disso, a integração de tecnologias como Ajax e outros frameworks com jQuery aumenta a possibilidade em melhorar o visual da aplicação.

Glossário

CSS: *Cascading Style Sheets.*

SKINS: temas.

STATEFUL: estado dos objetos.

Questão para reflexão

Como você escolheria o tema de sua aplicação? Quais seriam os critérios adotados?



Considerações Finais

- Temas;
- configurando temas;
- tipos de temas diferentes;
- aplicando temas.

Referências

BOOTSTRAP. Disponível em: <http://getbootstrap.com.br/>. Acesso em: 30 ago. 2016.

CORDEIRO, Gilliard. **Aplicações Java para a web com JSF e JPA.** São Paulo: Casa do Código, 2014.

JQUERY. Disponível em: <http://jqueryui.com/themeroller/>. Acesso em: 30 ago. 2016.

PRIMEFACES. Componentes do PrimeFaces. Disponível em: <http://www.primefaces.org>. Acesso em: 30 ago. 2016.

Assista a suas aulas



Aula 7 - Tema: Temas do primefaces, apresentação e utilização. Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/e50ca6d4f63f4b9cd23fdaccfb5068c4>.



Aula 7 - Tema: Temas do primefaces, apresentação e utilização. Bloco II

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/c4128b03c83d94db92fd739673cc05fb>.

Questão 1

1. Para que serve o ThemeRoller CSS framework?

a) Para facilitar o layout do JSP.

b) Adiciona temas do jQuery.

c) Adiciona temas do bootstrap.

d) Adiciona temas do HTML5.

e) Recurso de rotação de componentes.

Questão 2

2. O que é possível encontrar no showcase do PrimeFaces?

a) Diversos temas e componentes.

b) Apenas temas.

c) Apenas componentes.

d) É uma apresentação de temas.

e) É o local de informações e fórum.

Questão 3

3. O que é Stateful?

a) Estado dos objetos.

b) Estado do parâmetro.

c) Estado da variável.

d) Estado do JSF.

e) Estado do Servlet

Questão 4

4. Essa declaração xmlns:p=”http://primefaces.org/ui”> no escopo da JSF representa?

a) Estado do parâmetro.

b) Estado da variável.

c) Estado dos objetos.

d) Adição de layout.

e) Adição de componentes

Questão 5

5. Em qual tag deve se colocar o nome do tema do PrimeFaces dentro da <context-param>?

a) <param-name>

b) <param-value>

c) <url-pattern>

d) <session-config>

e) <welcome-file>

Gabarito

1. Resposta: B.

Recurso junto ao jQuery.

2. Resposta: A.

Há uma variação dos temas e componentes.

3. Resposta: A.

Referente ao objeto.

4. Resposta: E.

Declaração do framework para componentes.

5. Resposta: B.

Tag padrão de para declarar o tema escolhido.

Unidade 8

JSF com JPA e MVC apresentação e utilização

Objetivos

1. CRUD
2. ORM
3. JPA
4. DAO
5. MVC

Introdução

Nos módulos anteriores, você aprendeu como configurar o JSF com PrimeFaces e escolher os temas de sua aplicação; ele poderá ser usado com qualquer outro framework, inclusive com persistência.

No Tema 4, você aprendeu a configurar o Hibernate integrando com JSF e JPA Hibernate, fazendo uma pequena aplicação para verificar o funcionamento que faz uma consulta ao banco de dados.

Neste tema, o foco é a persistência que realizará as ações de CRUD (*Create, Read, Update e Delete*) no padrão MVC (*Model, View e Controller*). Além disso, colocar-se-á em prática a persistência utilizando padrão MVC com JSF.

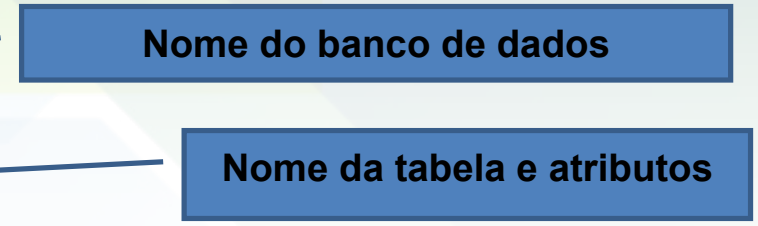
Portanto, o JPA será utilizado como framework de persistência e, antes de começar a desenvolver o projeto, verifique se o banco de dados de nome: **cadastro** com a tabela **users** utilizado em lições anteriores está disponível em seu Sistema Gerenciador de Banco de Dados **MySQL**:

Para saber mais

No site do MySQL, há diversas ferramentas gratuitas além do servidor. Uma delas é o Workbench MySQL, que auxilia no gerenciamento de seu SGDB. Disponível em: <https://www.mysql.com/products/workbench/>. Acesso em: 20 ago. 2016.

```
CREATE DATABASE cadastro;

CREATE TABLE users(
id_usuario int not null auto_increment,
login varchar(100) null,
senha varchar(100) null,
email varchar(160) null,
Primary key(id_usuario));
```



Para realizar a persistência, será utilizado o mesmo banco de dados nomeado de: **cadastro** com a tabela **users** do Tema 4. Inclua alguns registros na base de dados para serem apresentados em sua aplicação com JSF.

No Tema 4, você fez uma pequena aplicação com a integração das tecnologias do JSF com persistência, ao carregar o projeto pode entender como funciona a persistência de dados, conseguindo ter um melhor entendimento e ao clicar no botão na aplicação Web a programação trouxe os dados do primeiro registro da tabela.

Para melhor aprendizado, você fará uma aplicação do **CRUD** utilizando o **Netbeans**. O objetivo é utilizar o JPA. No caso, utilizará o EclipseLink para persistência, que é um dos frameworks semelhante ao Hibernate, com as mesmas especificações do JPA.

Para saber mais

CRUD são as operações realizadas em aplicações com banco de dados e esse nome é a sigla das operações SQL. Disponível em: <http://docs.jboss.org/tools/latest/en/seam/html/crud_database_application.html>. Acesso em: 20 ago. 2016.

Lembre-se de que o JPA é uma especificação do JAVA vinda do EJB e há diversos frameworks que auxiliam na persistência. A persistência com JPA auxiliará na criação do CRUD.

1 CRIANDO PROJETO JPA CRUD JSF

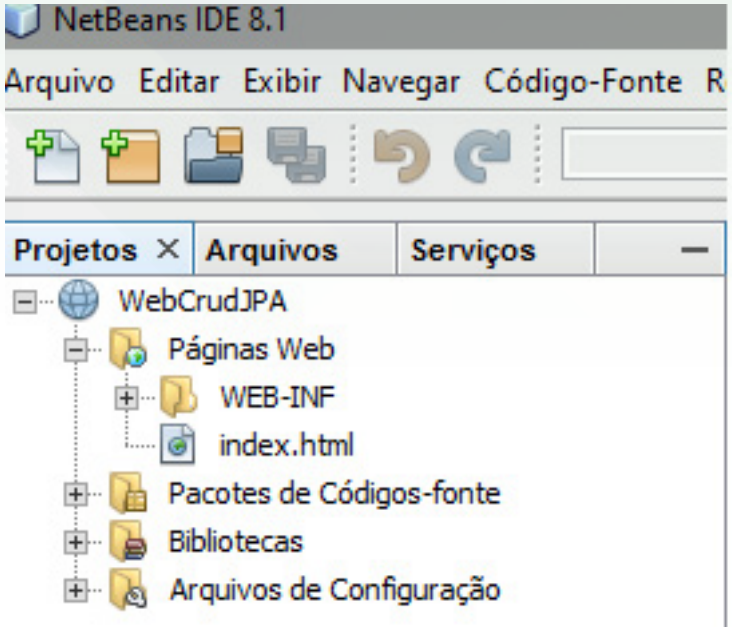
Primeiro, você deverá criar um projeto Java *Web* (JEE) no Netbeans com nome sugerido de **WebCrudJPA**, e depois escolha o servidor **GlassFish 4.x**, que é um servidor completo com todas as especificações do EJB.

Link

É possível utilizar o servidor GlassFish em qualquer IDE que disponibiliza suporte e pode ser obtido diretamente pelo site. Disponível em: <https://glassfish.java.net/download.html>. Acesso em: 20 ago. 2016.

Lembre-se de que o servidor TOMCAT não contém todas as especificações do JPA e é apenas um Web *container*. Ao criar o projeto Java *Web* (JEE), não escolha nenhum *framework*, apenas avance e, assim que concluir, verifique se seu projeto ficou com a aparência da **Figura 36:**

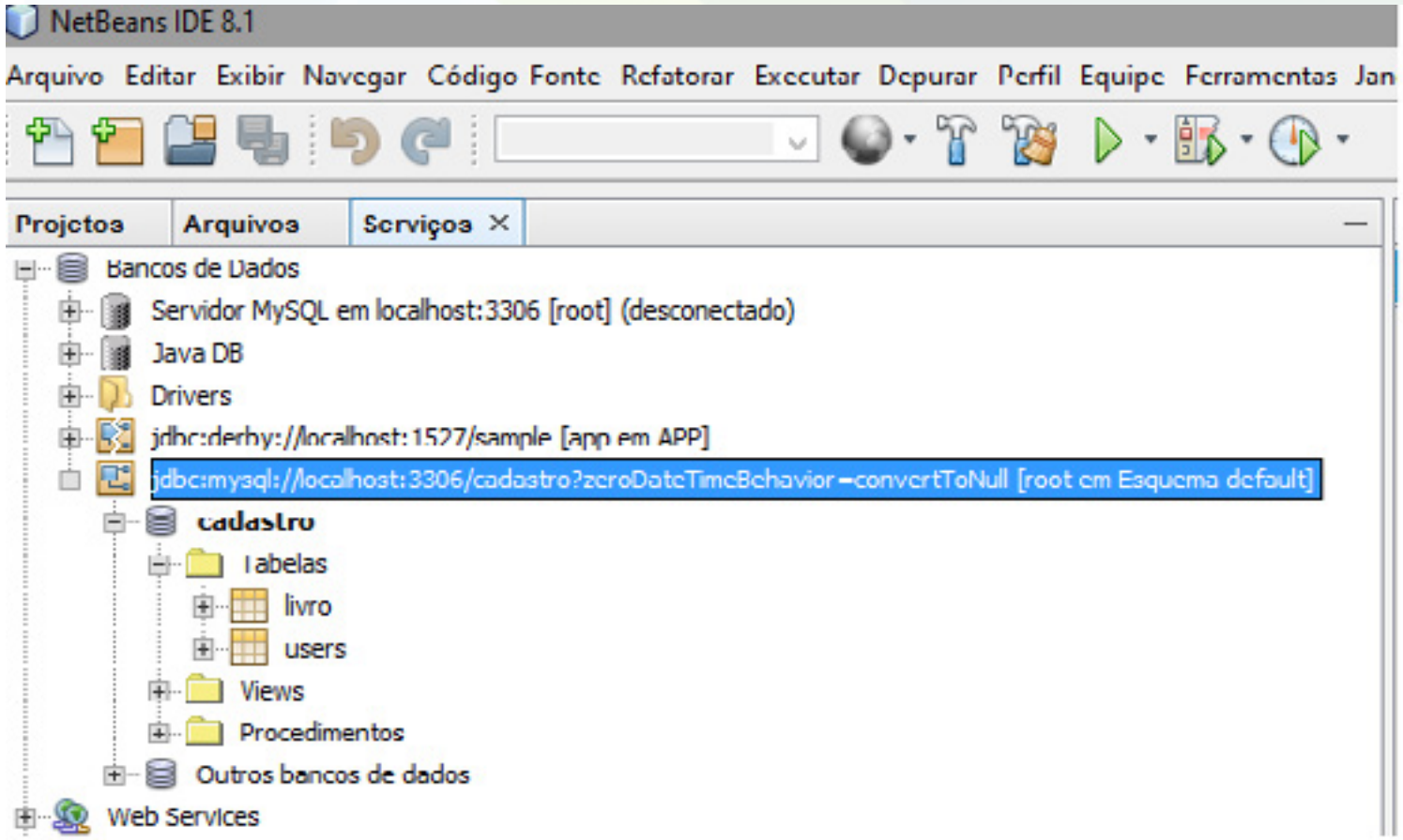
Figura 36 – Estrutura do projeto WebCrudJPA



Fonte: Elaborada pelo autor.

Agora que criou o projeto no Netbeans, deve ir à aba Serviços (***Database Explorer***) e fazer uma conexão com o banco de dados cadastro do MySQL, conforme a **Figura 37**, a seguir:

Figura 37 – Conexão com MySQL utilizando Netbeans



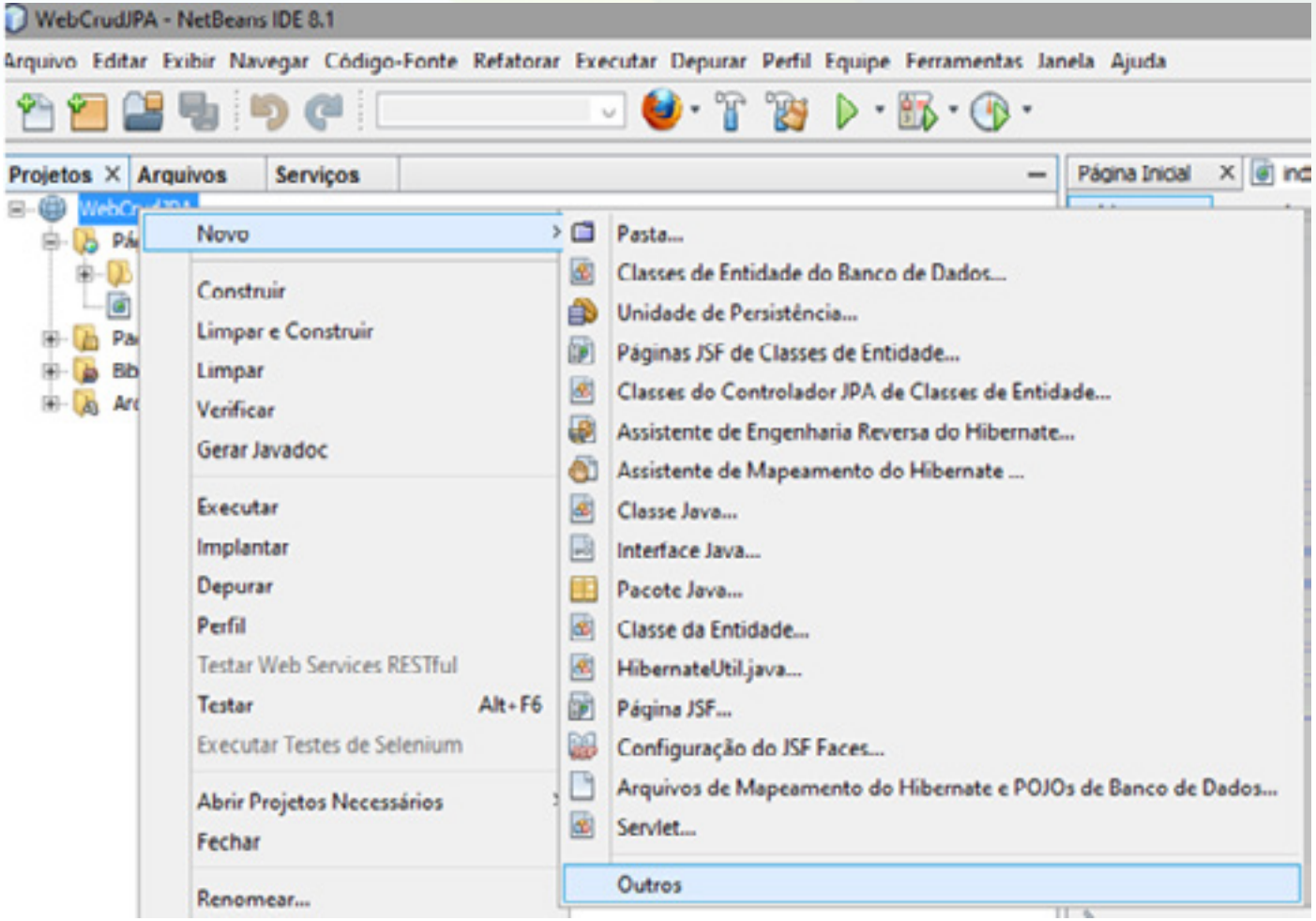
Fonte: Elaborada pelo autor.

Link

Há alguns recursos no *Database Explorer* do Netbeans que facilitam a manipulação de banco de dados. Veja mais no link a seguir. Disponível em: <http://www.devmedia.com.br/o-ide-netbeans-revista-easy-java-magazine-7-parte-2/21376>. Acesso em: 20 ago. 2016.

Com o botão direito do mouse, clique em sua aplicação Web e entre em outros **(Figura 38)** e escolha persistência:

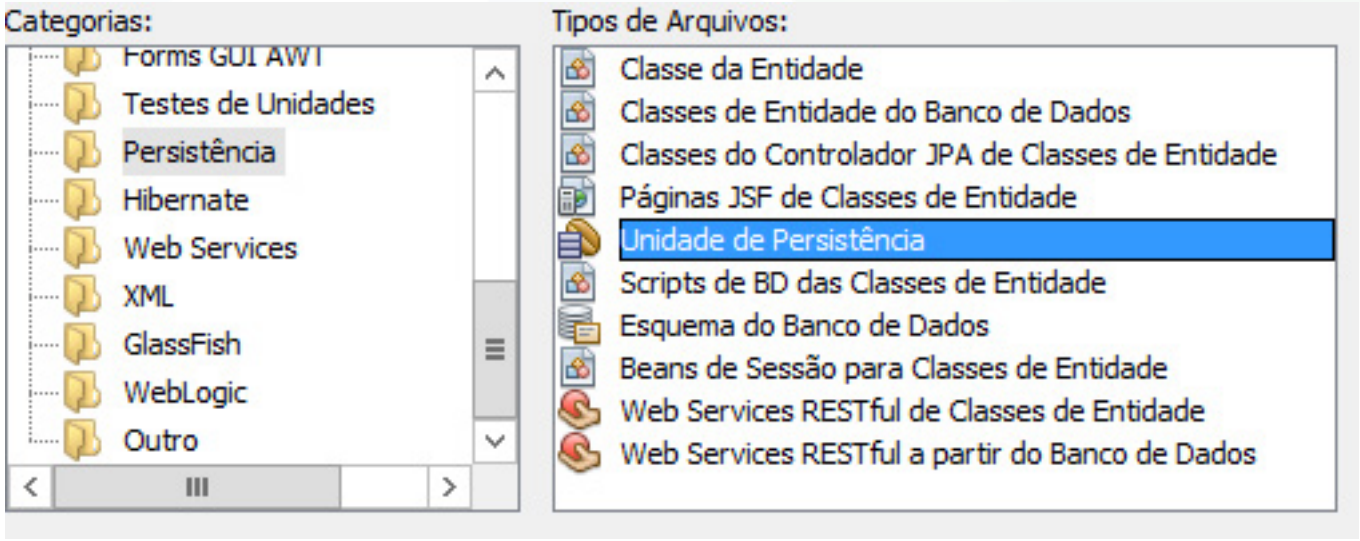
Figura 38 – Diretório para escolher aplicabilidade a ser utilizada



Fonte: Elaborada pelo autor.

Logo em seguida **(Figura 39)**, escolha Unidade de Persistência:

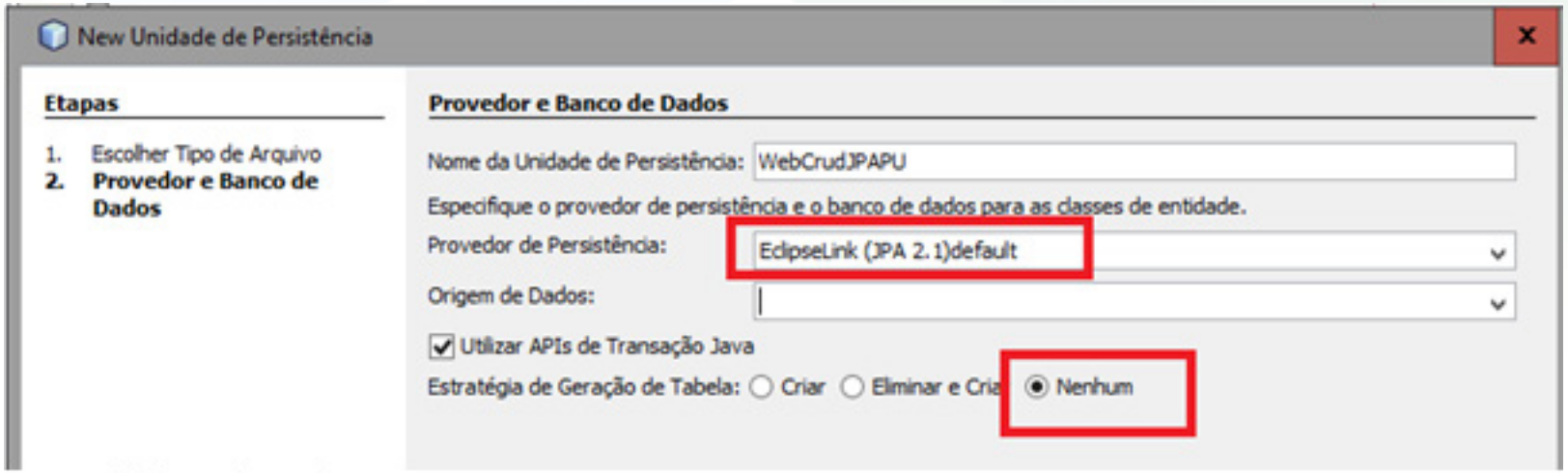
Figura 39 – Criação de unidade de persistência com JPA



Fonte: Elaborada pelo autor.

Ao abrir a próxima tela, escolha o **Provedor de Persistência**, no caso: EclipseLink (JPA 2.1) e na estratégia de tabela coloque nenhuma **(Figura 40),** O próximo passo é estabelecer a conexão com banco de dados cadastro no MySQL **(Figura 41):**

Figura 40 – Escolha de provedor de JPA

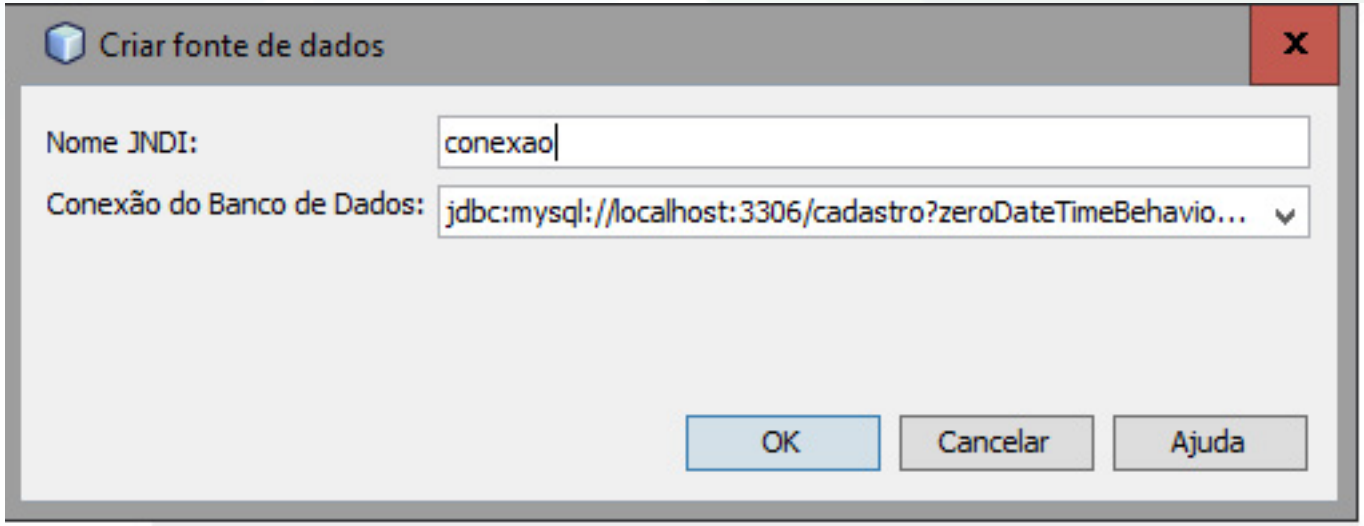


Fonte: Elaborada pelo autor.

Para saber mais

Há algumas opções para escolher como provedor de persistência em sua aplicação. Entre as mais utilizadas, estão: Hibernate, EclipseLink, TopLink e Apache OpenJPA. Conheça algumas semelhanças entre dois destes. Disponível em: <http://www.edsongoncalves.com.br/category/jpa-2-0/>. Acesso em: 20 ago. 2016.

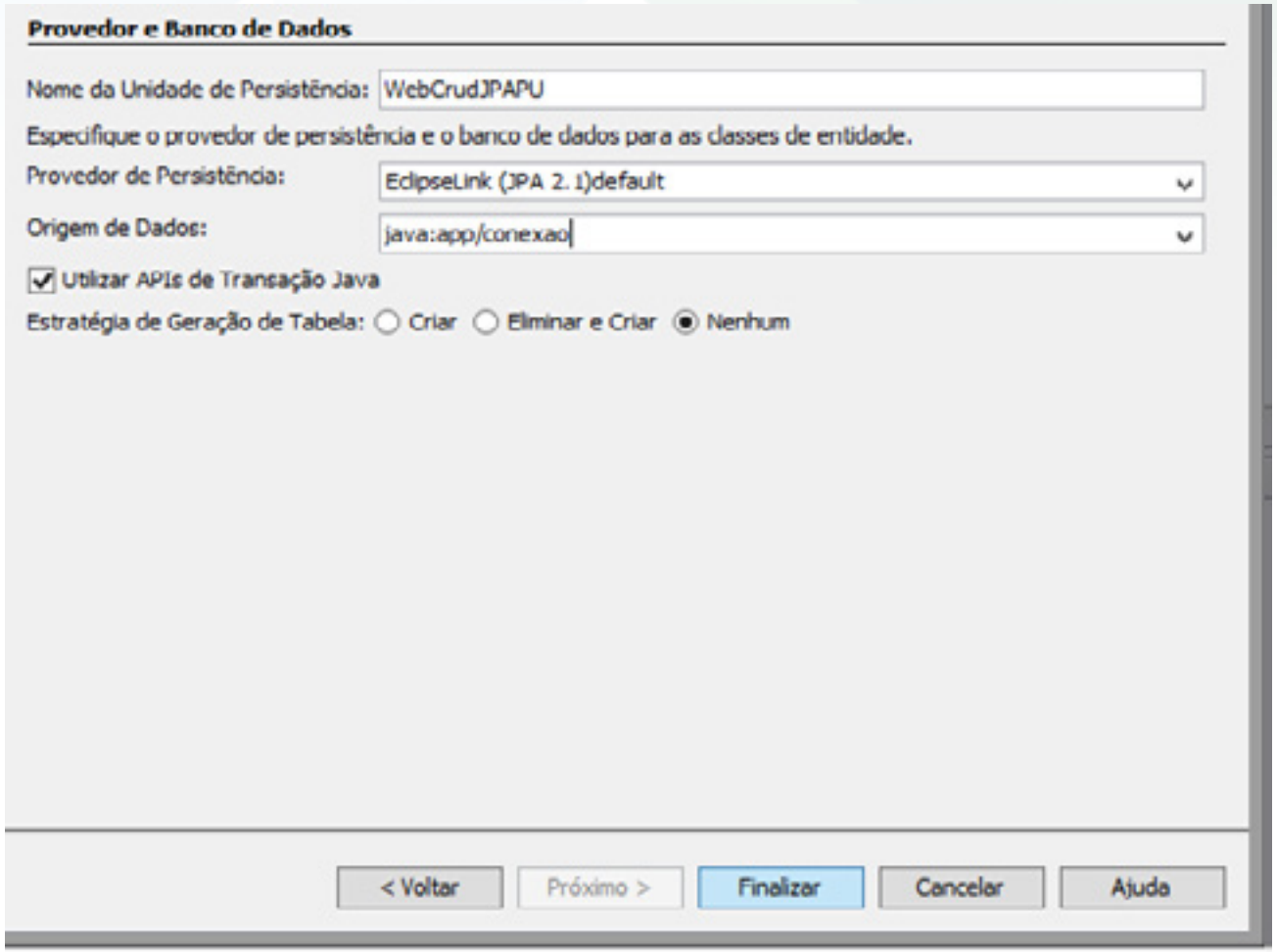
Figura 41 – Escolhendo conexão com base de dados para mapeamento do JPA



Fonte: Elaborada pelo autor.

Realizado o procedimento, ficará como na **Figura 42,** e em seguida finalize.

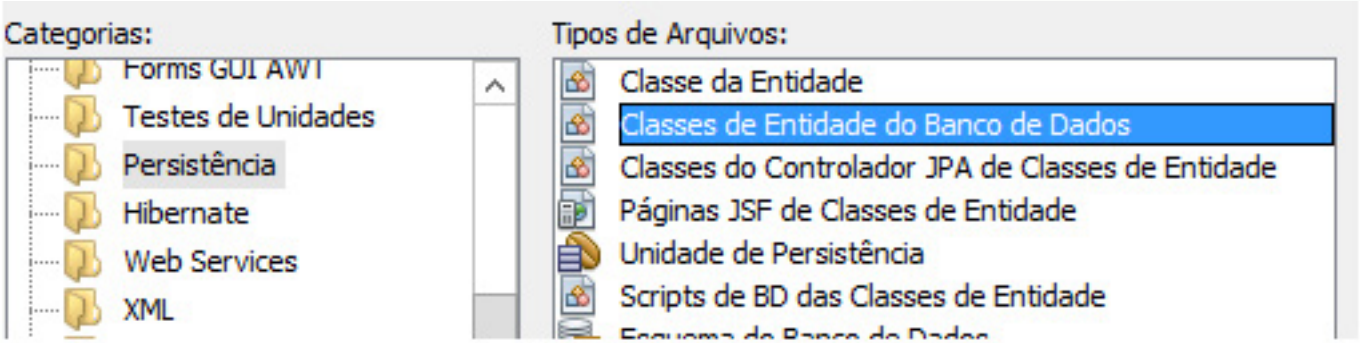
Figura 42 – Passos de configuração do JPA



Fonte: Elaborada pelo autor.

Pronto, há uma unidade de persistência em sua aplicação e o próximo passo é mapear uma classe do banco de dados. Para isso, vá em persistência, que está dentro de outros e escolha: Classes de Entidade de Banco de Dados **(Figura 43).**

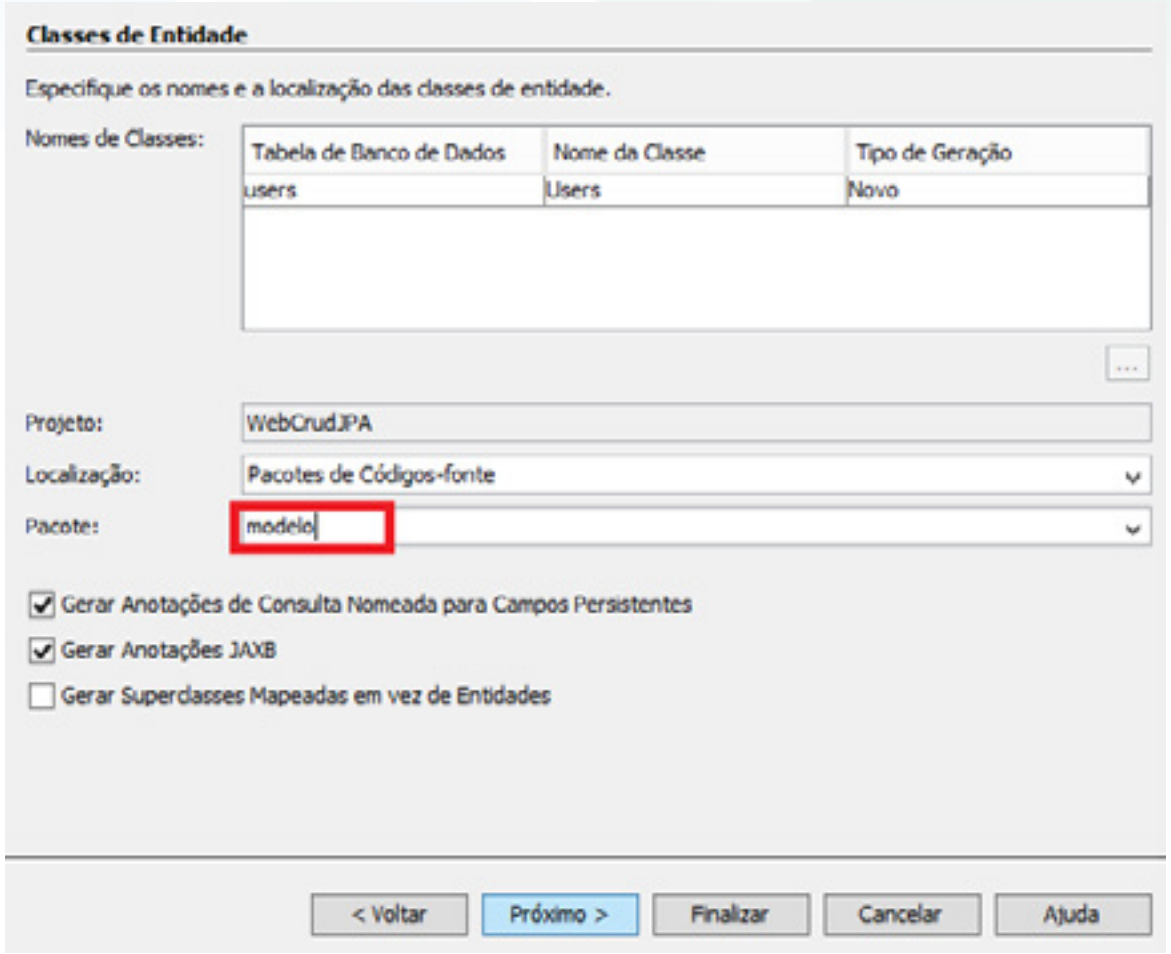
Figura 43 – Inserindo a classe de banco de dados



Fonte: Elaborada pelo autor.

Adicione a tabela que irá fazer o mapeamento. No banco de dados cadastro, há uma tabela **users** que você irá utilizar nessa aplicação. Em seguida, deverá dar um nome ao pacote que será criado a classe. Você deve colocar o nome de modelo e poderá avançar e finalizar **(Figura 44):**

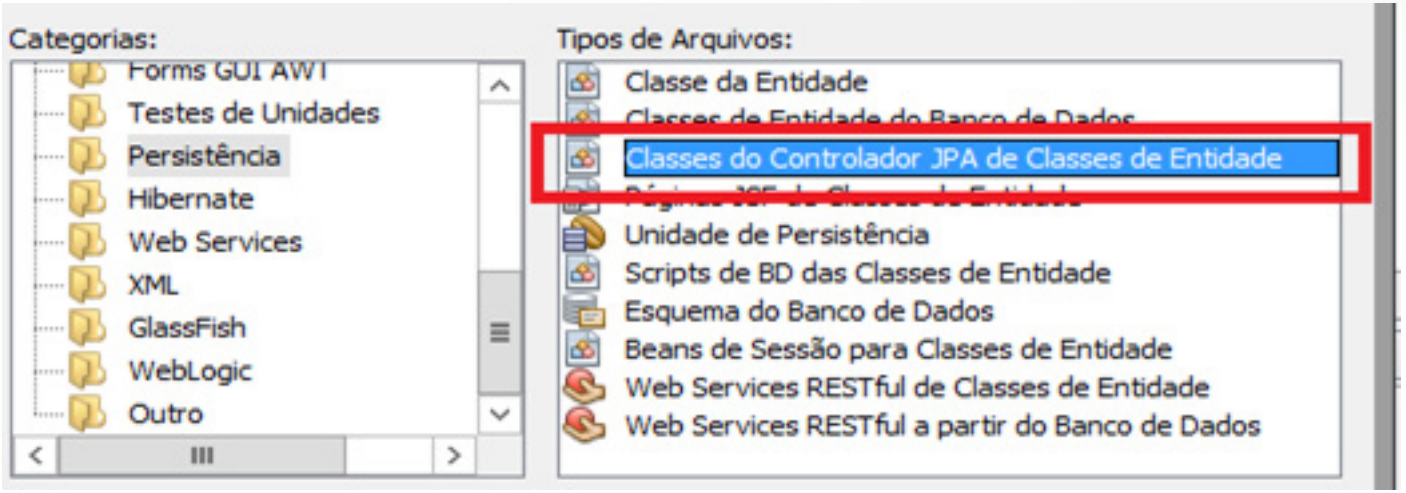
Figura 44 – Configurações da classe



Fonte: Elaborada pelo autor.

Agora, o projeto possui o mapeamento dos itens do banco de dados, que é um modelo para criar o CRUD. Em outros, na persistência deverá adicionar uma classe controladora JPA **(Figura 45):**

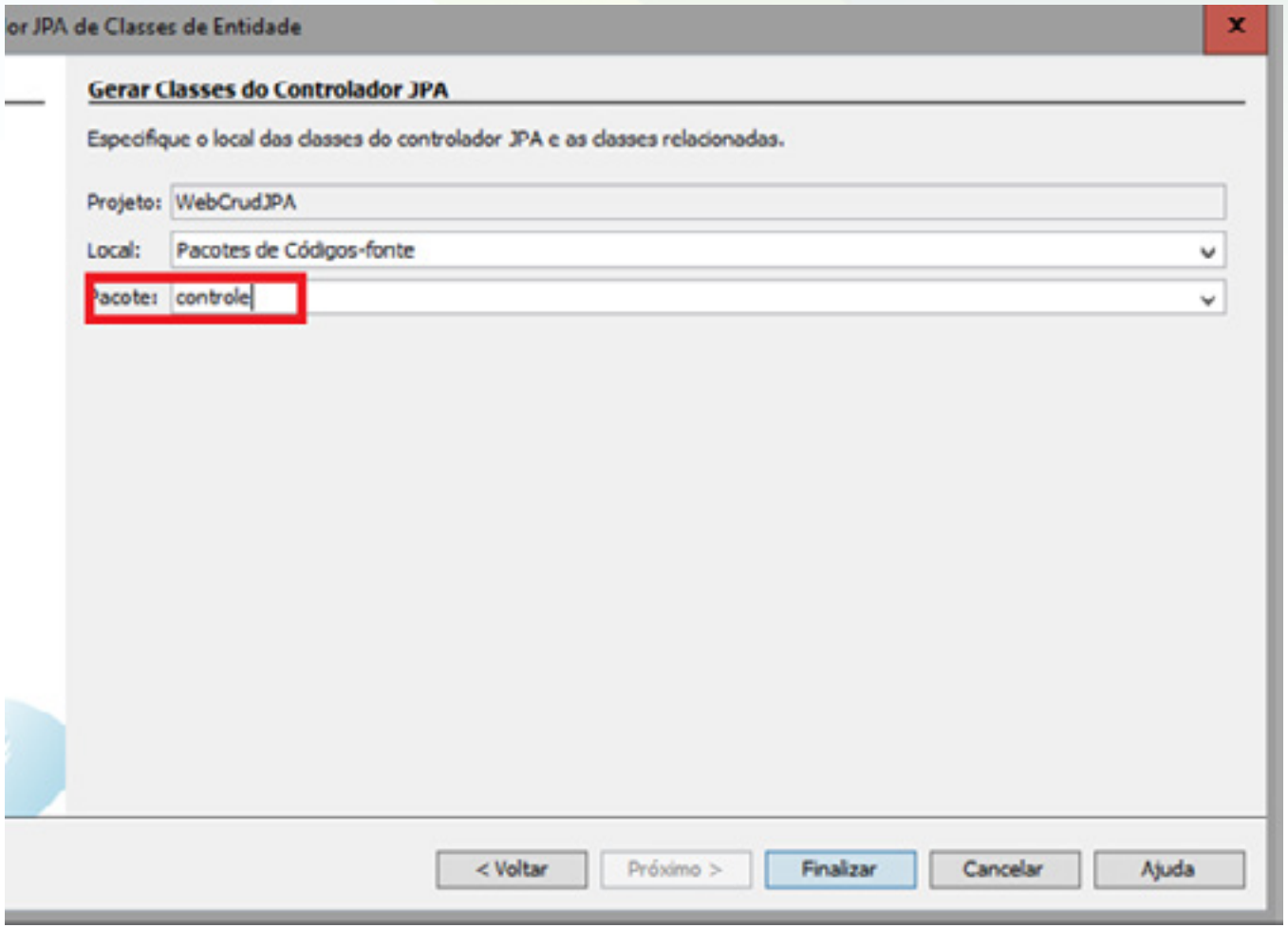
Figura 45 – Classe controladora



Fonte: Elaborada pelo autor.

Essa classe é responsável em utilizar os métodos do CRUD. Na sequência, escolha a classe que está dentro do modelo e, em seguida, troque o nome do pacote para: controle e finalize **(Figura 46):**

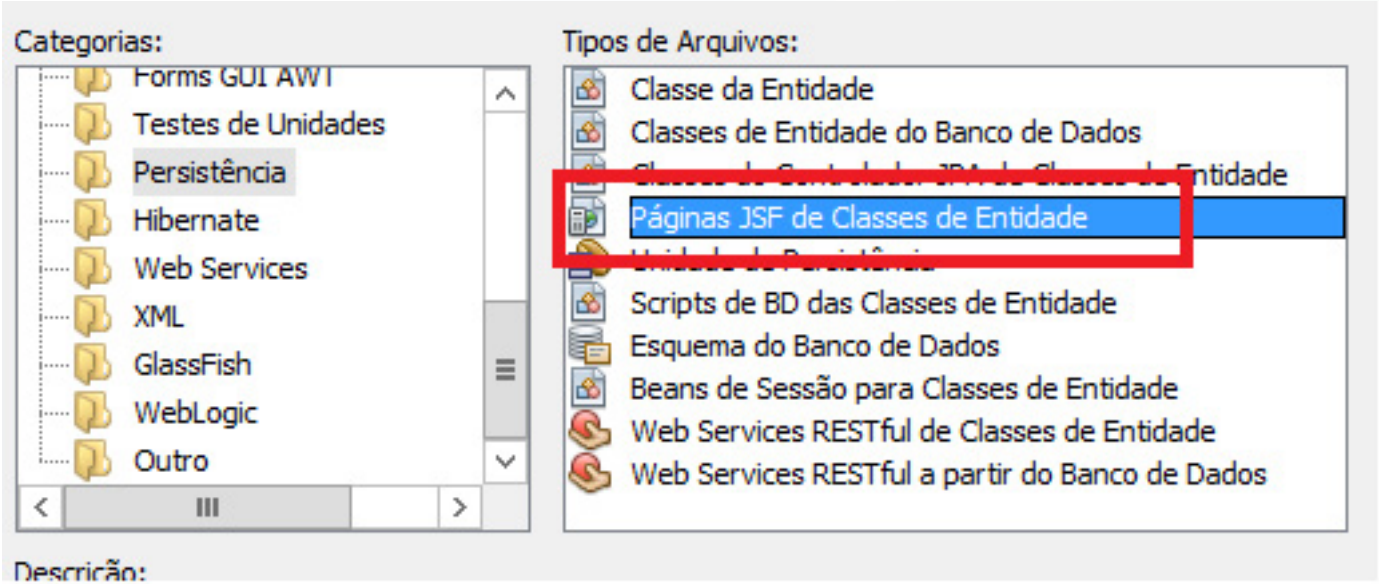
Figura 46 – Configuração para classe controladora



Fonte: Elaborada pelo autor.

O seu projeto contém as duas camadas do Model e *Controller* do padrão MVC. Você irá completar o CRUD com JSF, que é a *View* desse modelo. Entre em outros e em persistência escolha: Páginas JSF de classes de entidades **(Figura 47):**

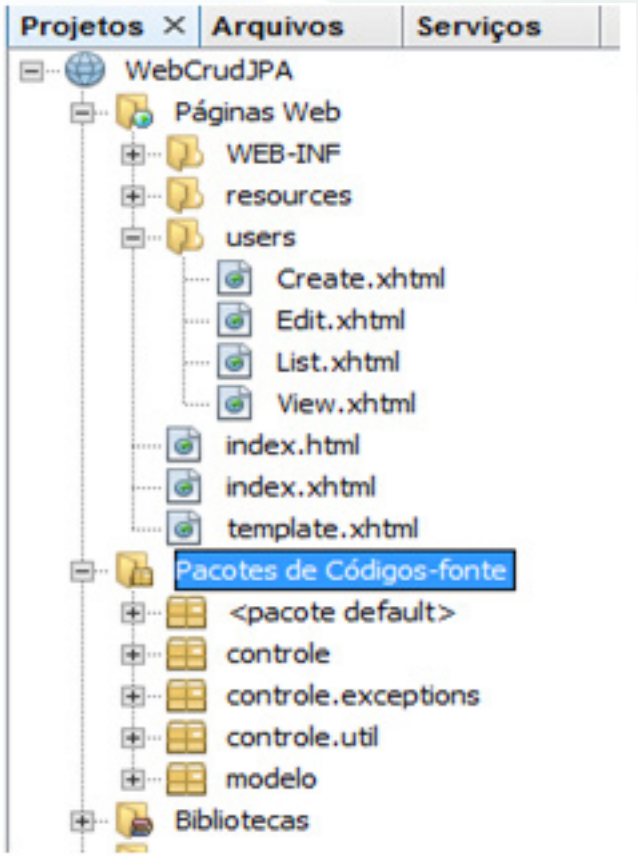
Figura 47 – classe de entidade do JSF



Fonte: Elaborada pelo autor.

Ao selecionar essa opção, escolha a classe dentro do modelo e depois avançar e concluir. Verá que a estrutura de seu projeto criará uma pasta chamada **users** com páginas JSFs com o CRUD **(Figura 48)** e as pastas do restante do MVC.

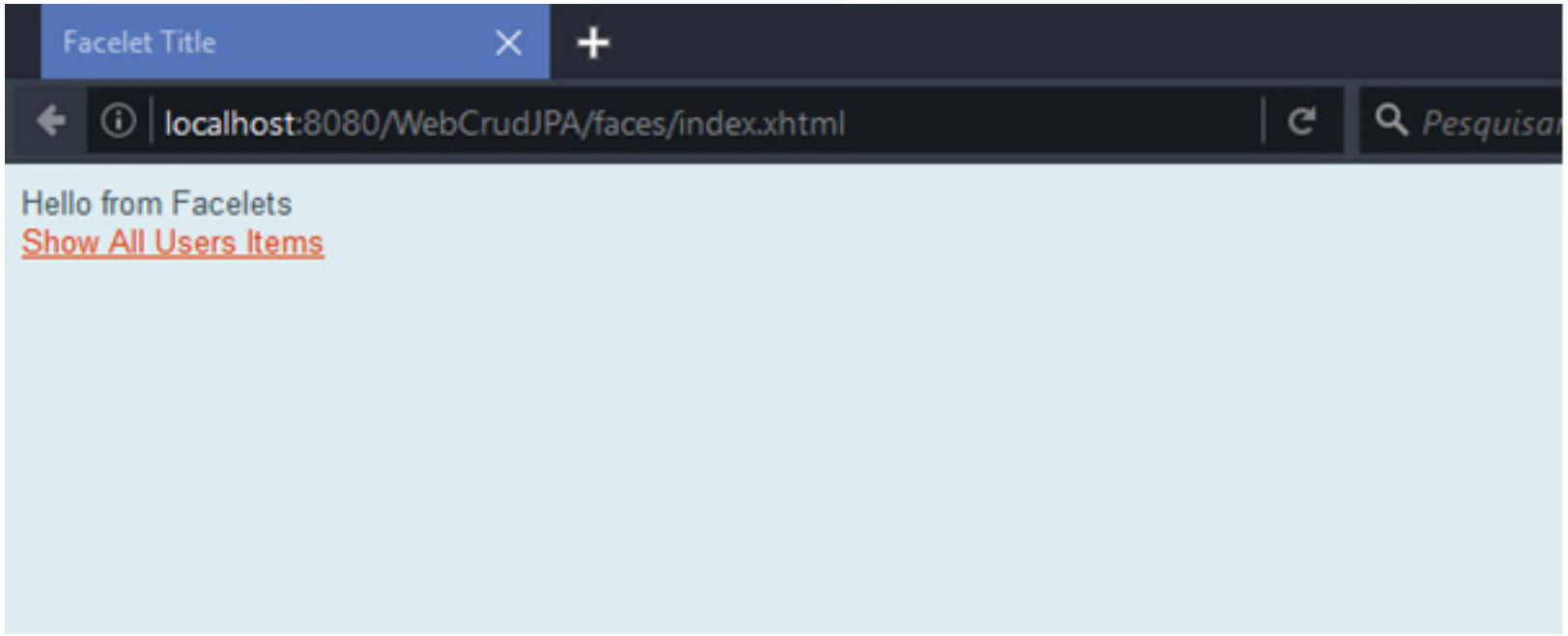
Figura 48 – Estrutura do projeto WebCrudJPA



Fonte: Elaborada pelo autor.

Carregue seu projeto, que deverá iniciar com a seguinte tela **(Figura 49):**

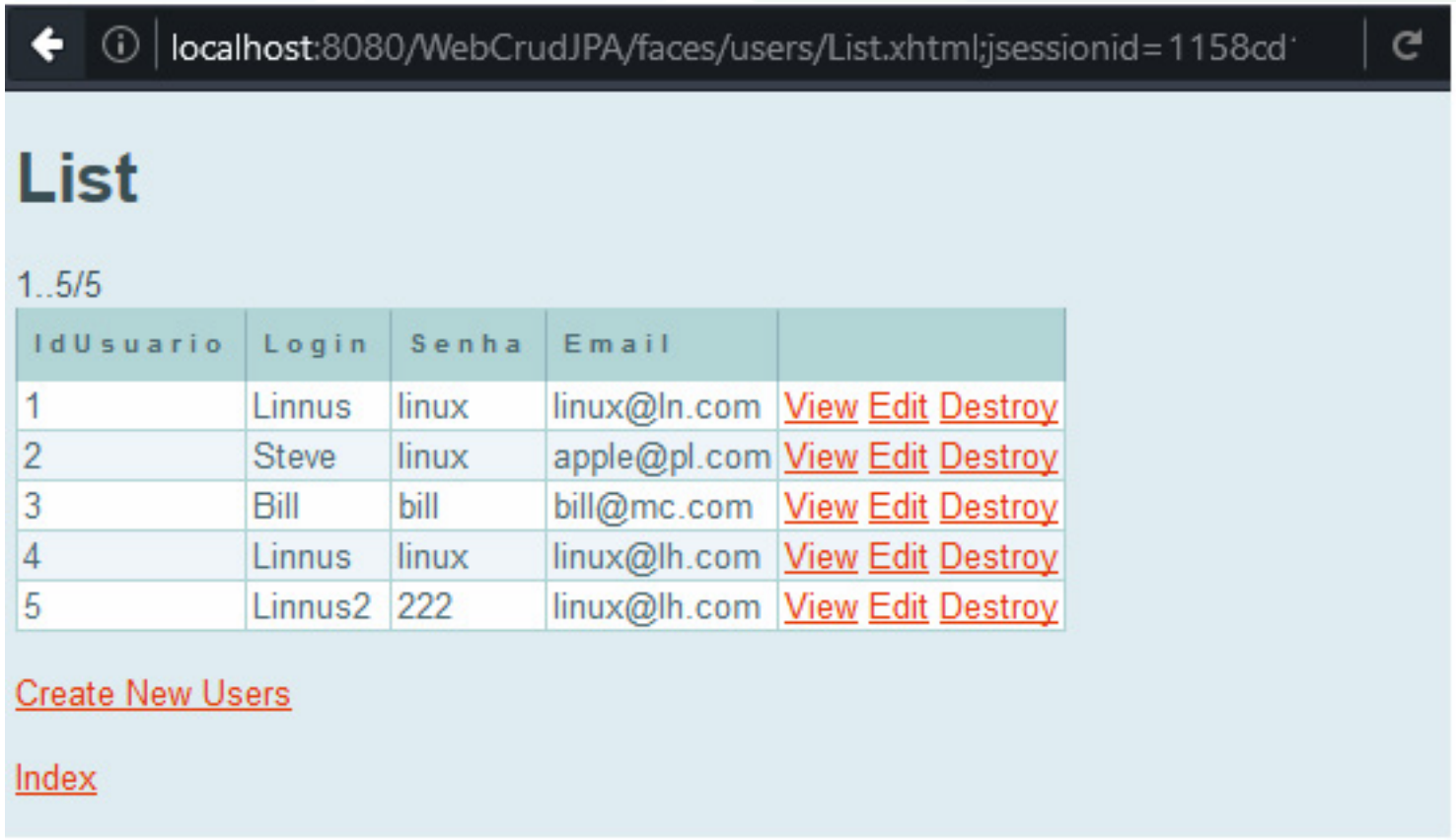
Figura 49 – Resultado da aplicação WebCrudJPA



Fonte: Elaborada pelo autor.

Ao clicar para mostrar todos os itens, você verá o quadro apresentado na **Figura 50, e, dentro dele,** é possível editar, ver e apagar o item desejado:

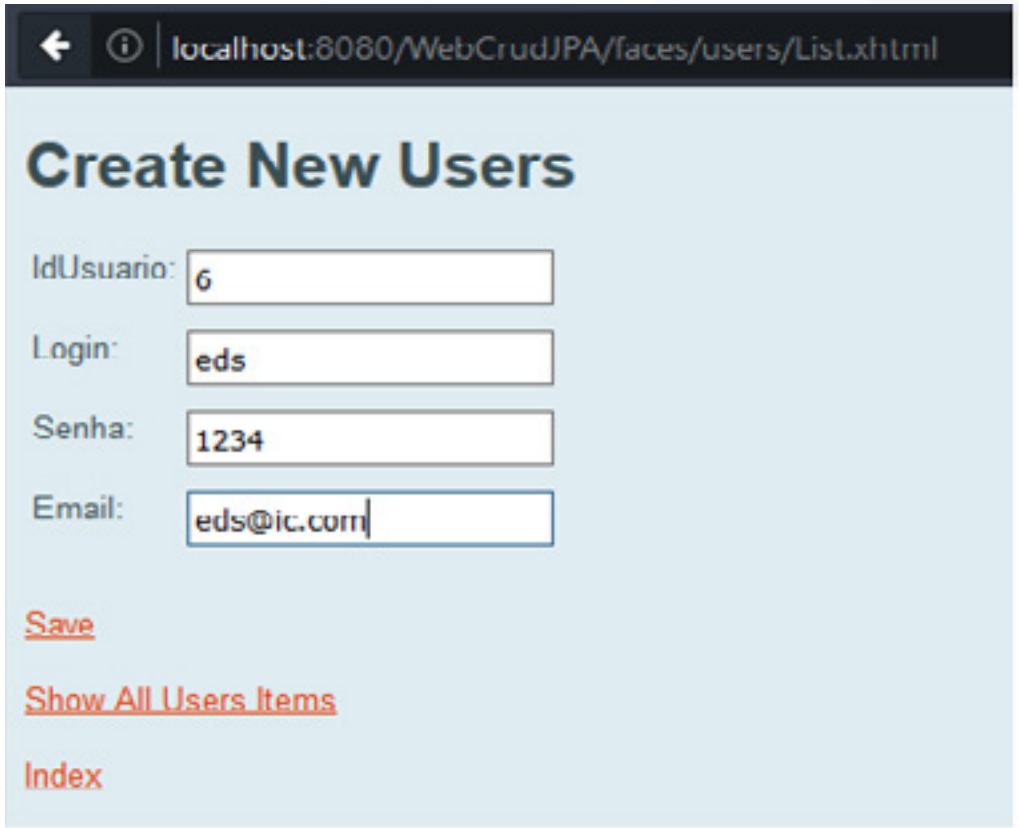
Figura 50 – Lista de itens cadastrados no banco de dados



Fonte: Elaborada pelo autor.

Para criar usuários, deve=se acessar o link **(Figura 51)** logo abaixo do quadro ou voltar para o início da aplicação:

Figura 51 – Tela de cadastro no banco de dados



Fonte: Elaborada pelo autor.

Portanto, essa é uma aplicação completa com padrões do MVC e ações do CRUD. É possível modificar os itens para português ou qualquer parte do programa. Essa é uma aplicação auxiliada pelo EclipseLink (JPA) e Netbeans, que facilita o desenvolvedor criar exemplos como esse apresentado.

O JPA e diversos recursos do JSF são ferramentas que permitirão aplicações Web melhor elaboradas e com tecnologias que ajudarão na performance como ferramentas do Ajax e também com JPA na persistência.

É possível utilizar diversos frameworks e padrões dentro de aplicações Web, que são abrangentes e com propósitos diferentes. A especificação do *Java Enterprise Edition* é adaptada e atualizada a diversas tecnologias existentes para *Web*, dentre elas: Ajax, RichFaces, PrimeFaces, JPA, HTML 5.

O **JEE** pode ser utilizado em aplicações Web e também para dispositivos móveis, como tablets e smartphones. Os servidores Web para JEE possuem uma tecnologia de processamento muito alta e garantem transações de dados e o uso de padrões e especificações.

Link

O JEE é uma tecnologia completa e utilizada como padrão de diversas aplicações com mais de nove milhões de desenvolvedores. Disponível em: <https://www.java.com/pt_BR/about/>. Acesso em: 20 ago. 2016.

A tecnologia JAVA está sempre inovando e muitas de suas atualizações são feitas por diversos colaboradores, dentre eles: Apache e o PrimeFaces. O JAVA, desde seu lançamento, vem conquistando muitos desenvolvedores por ser uma das tecnologias mais confiáveis e colaborativas.

Glossário

JPA: Java *Persistence* API.

DAO: Data Access Object.

MVC: Model View Controller.

Questão para reflexão

Como escolher um provedor JPA em aplicações Web com JEE?



Considerações Finais

- CRUD;
- PERSISTÊNCIA;
- FRAMEWORK;
- JPA;
- DAO;
- provedores JPA e EclipseLink;
- integração de tecnologias;
- aplicação JSF com JPA e DAO no padrão MVC;
- aplicação Web com CRUD completa;
- manipulação de dados na Web com JPA e MVC;
- servidor de aplicação para aplicação JPA/ORM.

Referências

DEVMEDIA. Netbeans. Disponível em: <http://www.devmedia.com.br/o-ide-netbeans-revista-easy-java-magazine-7-parte-2/21376>. Acesso em: 30 ago. 2016.

GONSALVES, Edson. JPA. Disponível em: <http://www.edsongoncalves.com.br/category/jpa-2-0/>. Acesso em: 30 ago. 2016.

GLASSFISH. Servidor Web. Disponível em: <https://glassfish.java.net/download.html>. Acesso em: 30 ago. 2016.

JBOSS. CRUD. Disponível em: <http://docs.jboss.org/tools/latest/en/seam/html/crud_database_application.html>. Acesso em: 30 ago. 2016.

Assista a suas aulas



Aula 8 - Tema: JSF com JPA e MVC apresentação e utilização. Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f-1d/54b0328e02317e52851a4f51c3b4b7c5>.



Aula 8 - Tema: JSF com JPA e MVC apresentação e utilização. Bloco II

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f-1d/60b7aed961ca4b7a1ca2dcd94eebd50c>.

Questão 1

1. O que é CRUD?

a) São aplicações realizadas no banco de dados.

b) Padrão Web de provedor.

c) Persistência de dados JPA.

d) Sigla de *design pattern*.

e) Referência ao MVC.

Questão 2

2. Qual dessas ferramentas é uma IDE completa de gerenciamento de banco de dados?

a) Eclipse.

b) JDeveloper.

c) Netbeans.

d) Workbench.

e) Visual Studio.

Questão 3

3. Para utilização do JPA, qual desses servidores não possui todas as especificações do EJB?

a) Glassfish.

b) JBoss.

c) WebSphere.

d) TOMCAT.

e) BEA.

Questão 4

4. Em qual dessas opções todos são provedores do JPA?

a) Hibernate, EclipseLink, DAO.

b) Hibernate, Eclipse, TopLink.

c) Hibernate, EclipseLink, TopLink.

d) Hibernate, EclipseJPA, TopLink.

e) Hibernate, EclipseLink, TopLinkJPA.

Questão 5

5. Na aplicação com Hibernate ou EclipseLink que são frameworks de persistência, quando há uma classe com o CRUD em qual pacote é aconselhável colocar a classe com os métodos de ação de acordo com o padrão MVC?

a) jpa.model.

b) jpa.controller.

c) jpa.view.

d) jpa.aplication.

e) jpa.provedor.

Gabarito

1. Resposta: A.

Referem-se aos comandos SQL feitos na tabela de seu banco de dados.

2. Resposta: D.

Apesar de o Netbeans fornecer algumas ferramentas, não é considerado completo como o Workbench.

3. Resposta: D.

O TOMCAT é apenas um Web *container* e não possui todas especificações do EJB.

4. Resposta: C.

Os nomes corretos de provedores com especificação JPA são Hibernate, EclipseLink e TopLink.

5. Resposta: B.

Nessa classe, há um controle com CRUD e é apropriado colocar no pacote jpa.controller.

